

# **Coding Interview Solutions Handbook: Essential Problems in C++**

Catalog of problems, solutions and implementations in C++

rfdavid, 2025

# INTRODUCTION

# Motivation

The tech industry hiring standard is based on algorithm and data structure.

There are plenty of free resources available around algorithms and data structures. The purpose of this project is to be a quick guide where you can learn and review algorithms and data structures.

Some of the intended **key features:**

- Non-verbose, short-structured, and easy to follow descriptions
- Slide-based, practical for reviewing
- Based on common problems
- Free and open-source

★ If you like, please add a star at [github.com/rfdavid/coding-interview-handbook-cpp](https://github.com/rfdavid/coding-interview-handbook-cpp)

# How do I created and use this document?

Creating this document was a great exercise to learn and review data structures and algorithms. I encourage anyone to create its own notes

- Have sections based on the type of the problems (string, array, tree, dynamic programming)
- Start solving the problems randomly on Leetcode
- Create one slide for each question, including: problem, solution, code
- Write with your own words
- If you are not confident you have fully captured a problem, write the problem and leave the solution and code to another day when you redo it

★ If you like, please add a star at [\*\*github.com/rfdavid/coding-interview-handbook-cpp\*\*](https://github.com/rfdavid/coding-interview-handbook-cpp)

# Some Useful Links

## **Tech Interview Handbook**

<https://www.techinterviewhandbook.org>

A very well-structured resource for interview preparation

## **Interviewing.io**

<https://interviewing.io>

Anonymous mock interviews with engineers from Amazon, Google, Meta, and other top companies

## **Blind 75 Leetcode Questions**

<https://leetcode.com/discuss/general-discussion/460599/blind-75-leetcode-questions>

Popular list of leetcode questions

# Common Problems

- The following list includes blind 75 (a popular list of algorithm problems that intends to cover the main data structures and patterns) + other random popular problems

## Array

- ✓ [Two Sum](#)
- ✓ [Contains Duplicate](#)
- ✓ [Product of Array Except Self](#)
- ✓ [Best Time to Buy and Sell Stock](#)
- ✓ [Maximum Subarray](#)
- ✓ [Maximum Product Subarray](#)
- ✓ [Find Minimum in Rotated Sorted Array](#)
- ✓ [Search in Rotated Sorted Array](#)
- ✓ [3 Sum](#)
- ✓ [Container With Most Water](#)
- ✓ [Missing Number](#)

## Binary

- ✓ [Sum of Two Integers](#)
- ✓ [Number of 1 Bits](#)
- ✓ [Counting Bits](#)
- ✓ [Reverse Bits](#)

## Dynamic Programming

- ✓ [Climbing Stairs](#)
- [Coin Change](#)
- [Longest Increasing Subsequence](#)
- ✓ [Longest Common Subsequence](#)
- [Word Break](#)
- [Combination Sum](#)
- ✓ [House Robber](#)
- [House Robber II](#)
- [Decode Ways](#)
- ✓ [Unique Paths](#)
- ✓ [Jump Game](#)

## Matrix

- ✓ [Set Matrix Zeroes](#)
- ✓ [Spiral Matrix](#)
- ✓ [Rotate Image](#)
- [Word Search](#)

# Common Problems

## Tree

- ✓ [Maximum Depth of Binary Tree](#)
- ✓ [Same Tree](#)
- ✓ [Invert/Flip Binary Tree](#)
- ✓ [Path Sum](#)
- ✓ [Binary Tree Level Order Traversal](#)
- ✓ [Serialize and Deserialize Binary Tree](#)
- ✓ [Subtree of Another Tree](#)
- ✓ [Construct Binary Tree from Preorder and Inorder Traversal](#)
- ✓ [Validate Binary Search Tree](#)
- ✓ [Kth Smallest Element in a Binary Search Tree](#)
- ✓ [Lowest Common Ancestor of Binary Search Tree](#)
- ✓ [Implement Trie \(Prefix Tree\)](#)

[Add and Search Word](#)

[Word Search II](#)

[Balanced Binary Tree](#)

## Heap

- ✓ [Top K Frequent Elements](#)
- [Find Median from Data Stream](#)

## String

- ✓ [Longest Substring Without Repeating Characters](#)
- ✓ [Longest Repeating Character Replacement](#)
- ✓ [Minimum Window Substring](#)
- ✓ [Valid Anagram](#)
- ✓ [Group Anagrams](#)
- ✓ [Valid Parentheses](#)
- ✓ [Valid Palindrome](#)
- [Longest Palindromic Substring](#)
- [Palindromic Substrings](#)
- [Simplify Path](#)
- [Encode and Decode Strings](#) ★

## Linked List

- ✓ [Reverse a Linked List](#)
- ✓ [Detect Cycle in a Linked List](#)
- ✓ [Merge Two Sorted Lists](#)
- ✓ [Merge K Sorted Lists](#)
- ✓ [Remove Nth Node From End Of List](#)
- ✓ [Reorder List](#)
- ✓ [LRU Cache](#)

## Graph

- ✓ [Keys and Rooms](#)
- ✓ [Clone Graph](#)
- ✓ [Course Schedule](#)
- ✓ [Pacific Atlantic Water Flow](#)
- ✓ [Number of Islands](#)
- ✓ [Longest Consecutive Sequence](#)
- ✓ [Permutations](#)
- [Alien Dictionary](#) ★
- ✓ [Graph Valid Tree](#) ★
- ✓ [Number of Connected Components In an Undirected Graph](#) ★

## Interval

- ✓ [Insert Interval](#)
- ✓ [Merge Intervals](#)
- ✓ [Non-overlapping Intervals](#)
- ✓ [Meeting Rooms](#) ★
- [Meeting Rooms II](#) ★

# Other problems

- ✓ Maximum Level Sum of a Binary Tree
- ✓ Minimum Number of Increments on Subarrays to Form a Target Array
- ✓ Leaf-Similar Trees
- ✓ Count Good Nodes in Binary Tree
- Min Cost Climbing Stairs
- Longest Palindromic Subsequence
- ✓ Minimum Cost for Tickets
- ✓ Webcrawler
- ✓ Network Delay Time
- ✓ Rotated Digits
- ✓ Ransom Note
- ✓ String to Integer (atoi)
- ✓ Middle of the Linked List



# Time Complexity

**If the input gets bigger, how many steps does the algorithm take?**

- Measure of how much the execution time of an algorithm grows relative to the size of its input (usually called  $n$ )
- Expressed in **Big-O notation** (e.g.  $O(1)$ ,  $O(n)$ ,  $O(n^2)$ , etc) to describe the **upper bound** of how fast the algorithm's runtime grows
- Asymptotic notations

**Big-O ( $O$ )** – Upper Bound (**Worst-case**): Describes the maximum amount of time/memory an algorithm could take.

**Theta  $\Theta$**  – Tight Bound (**Exact**): describes both the **upper** and **lower bound** (the **exact** growth rate)

**Omega ( $\Omega$ )** – Lower Bound (**Best-case**): describes the minimum time/space the algorithm needs.

# Time Complexity

## Examples

### $O(1)$

| Examples                                           | Problems                          |
|----------------------------------------------------|-----------------------------------|
| Accessing an array element ( <code>arr[i]</code> ) | Hash table lookups                |
| Swapping two variables                             | Checking if a number is even/odd  |
| Stack/Queue <code>push</code> or <code>pop</code>  | Returning first element of a list |

### $O(\log n)$

|                                                |                                        |
|------------------------------------------------|----------------------------------------|
| Binary Search                                  | Search in sorted array                 |
| Balanced BST insert/find (AVL, Red-Black Tree) | Find k-th smallest in BST              |
| Finding floor/ceil in sorted array             | Finding square root with binary search |

### $O(n)$

|                                 |                                      |
|---------------------------------|--------------------------------------|
| Linear Search                   | Maximum subarray sum (Kadane's algo) |
| Finding min/max in an array     | Counting frequencies with hash map   |
| Traversing linked list or array | One-pass string processing problems  |

# Time Complexity

## $O(n \log n)$

|                                       |                                  |
|---------------------------------------|----------------------------------|
| Merge Sort / Heap Sort                | Sorting an array                 |
| Efficient algorithms for Closest Pair | Finding inversion count in array |
| Heapify operations                    | Kth largest element with heap    |

## $O(n^2)$

|                              |                                    |
|------------------------------|------------------------------------|
| Bubble Sort / Insertion Sort | Two Sum (brute force)              |
| Checking all pairs in array  | Longest Palindromic Substring (DP) |
| Floyd-Warshall algorithm     | Edit Distance (DP)                 |

## $O(n^3)$

|                                 |                               |
|---------------------------------|-------------------------------|
| Matrix multiplication (naive)   | Boolean matrix multiplication |
| DP on subsequences of length 3  | Some DP path-finding problems |
| Floyd-Warshall for dense graphs | Counting triangles in graph   |

## $O(2^n)$

|                                            |                                  |
|--------------------------------------------|----------------------------------|
| Recursive Fibonacci (no memo)              | Subset sum (brute force)         |
| Backtracking for combinations/permutations | N-Queens                         |
| Traveling Salesman (brute force)           | All subsets of array (power set) |

# Time Complexity

## $O(n!)$

|                                       |                                      |
|---------------------------------------|--------------------------------------|
| Generating all permutations           | Traveling Salesman (brute force)     |
| Brute-force anagram check             | Word ladder with all transformations |
| Solving puzzles with all arrangements | Hamiltonian Path                     |

**V = vertices (nodes)**

**E = edges**

## $O(V + E)$

|                            |                            |
|----------------------------|----------------------------|
| DFS / BFS (adjacency list) | DFS / BFS (adjacency list) |
|----------------------------|----------------------------|

## $O(E \log V)$

|                              |                              |
|------------------------------|------------------------------|
| Dijkstra with priority queue | Dijkstra with priority queue |
|------------------------------|------------------------------|

## $O(VE)$

|              |
|--------------|
| Bellman-Ford |
|--------------|

## $O(V^3)$

|                |
|----------------|
| Floyd-Warshall |
|----------------|

## $O(E \log E)$

|                         |
|-------------------------|
| Kruskal's MST algorithm |
|-------------------------|

# Space Complexity

**As the input size  $n$  grows, how much extra memory does the algorithm need to run?**

- Measure of how much memory an algorithm uses relative to **input size**
- Expressed in **Big-O notation** (e.g.  $O(1)$ ,  $O(n)$ ,  $O(n^2)$ , etc)
- It includes auxiliary space (extra memory used by the algorithm, not counting the input itself) and sometimes considers input space depending on the context
- Count only extra space needed (**exclude output**)
- The space complexity of a recursive tree traversal is  **$O(h)$** , where  $h$  is the height of the tree. This is because each recursive call adds a frame to the call stack, and in the worst case, the maximum stack depth is proportional to the tree's height

# Space Complexity

## Examples

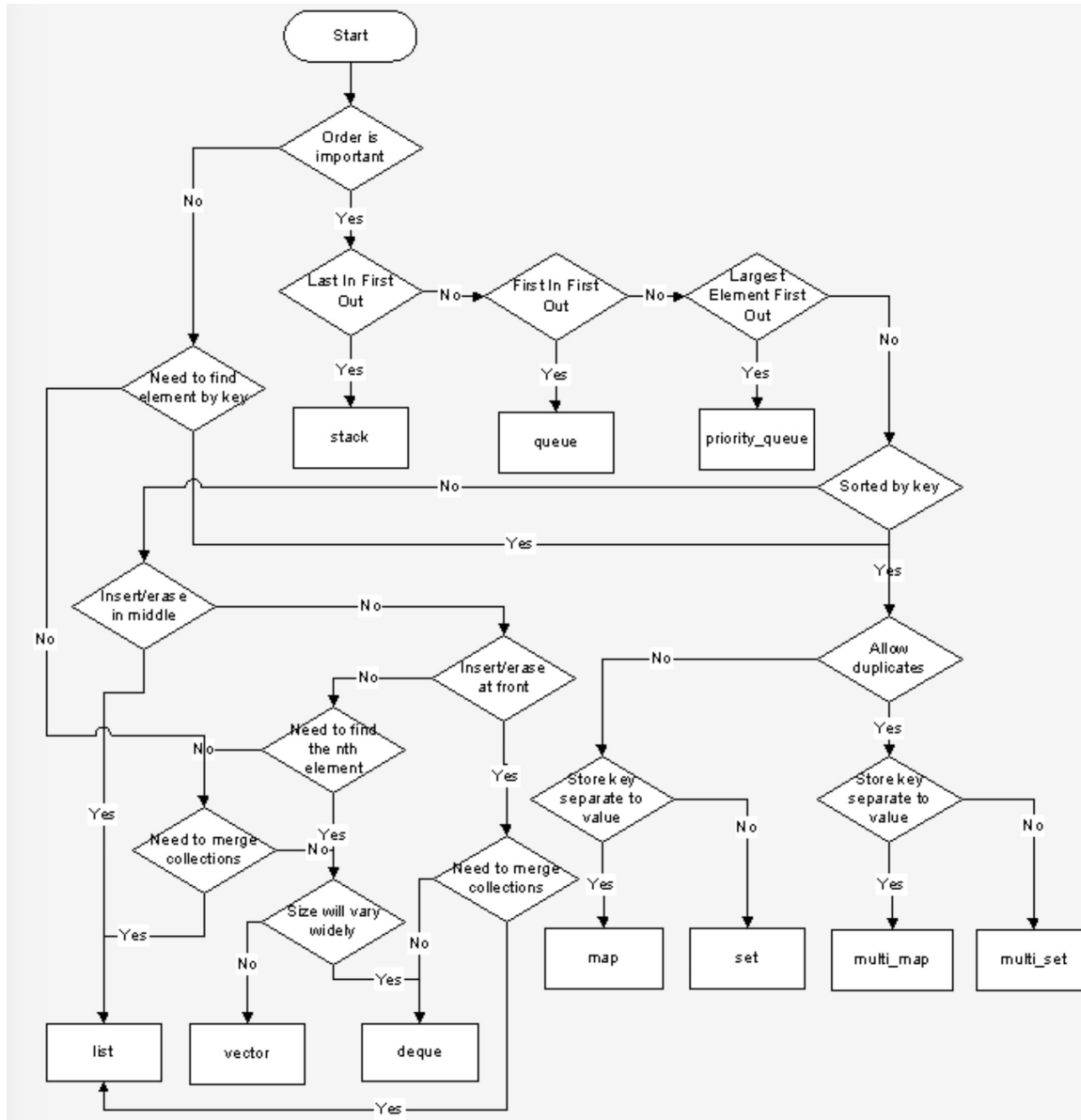
| Algorithm / Operation                          | Space Complexity | Explanation                                   |
|------------------------------------------------|------------------|-----------------------------------------------|
| Swap two integers                              | $O(1)$           | Only uses constant space                      |
| Iterate through array and sum values           | $O(1)$           | No extra memory used besides accumulator      |
| Store array copy                               | $O(n)$           | Needs space to store the copied array         |
| Recursive factorial (factorial(n))             | $O(n)$           | n stack frames in the call stack              |
| Binary search (recursive)                      | $O(\log n)$      | Recursive depth is $\log(n)$ for sorted array |
| Binary search (iterative)                      | $O(1)$           | No extra space beyond a few variables         |
| Merge sort                                     | $O(n)$           | Needs temp arrays to merge subarrays          |
| Quick sort (in-place)                          | $O(\log n)$      | Call stack for recursive calls                |
| Depth-first search (recursive) in tree         | $O(h)$           | h = height of the tree (stack frames)         |
| Breadth-first search (using queue)             | $O(n)$           | Stores all nodes at current level in queue    |
| DP with full 2D table (e.g., LCS)              | $O(m \cdot n)$   | Stores results of all subproblems             |
| Optimized Fibonacci with two variables         | $O(1)$           | Only tracks last two results                  |
| Memoized Fibonacci (top-down DP)               | $O(n)$           | Memoization table + recursion stack           |
| Using a hash map to count frequencies          | $O(n)$           | Stores one count per element                  |
| Storing all substrings of a string             | $O(n^2)$         | Total number of substrings is $\sim n^2$      |
| Adjacency list for graph with V nodes, E edges | $O(V + E)$       | One list per node, total edges stored         |

# **DATA STRUCTURES IN C++**

## **{Quick Recap}**

# Data Structure Decision Diagram

- The following diagram gives you the direction to which data structure to use in C++ according to the problem you are trying to solve



Note: I don't have the source of this diagram. If you know it, please drop me a msg so I can add it here.



# Arrays

- Fixed-size collection of elements of the same type
- Stored in **contiguous memory**
- Declared with syntax: **type arrayName[size]**

## Example:

```
int numbers[5]
```

- Can also be initialized at declaration:

```
int arr[3] = {1 ,2, 3}
```

- Cannot resize after declaration
- Size can be calculated by **sizeof(arr) / sizeof(arr[0])**
- stdlib provides **std::array<type, size>**
- Includes header: **#include <array>**

## Example:

```
std::array<int, 3> a = {1, 2, 3};
```

# Arrays (vectors)

- `std::vector` is a sequence container that encapsulates dynamic sized arrays\*
- Stored in **contiguous memory** (like arrays)
- Declared with syntax: **`std::vector<type> vectorName`**

## Example:

```
std::vector<int> numbers
```

- Can be initialized at declaration:

```
std::vector<int> vec = {1, 2, 3}
```

- **Can resize dynamically** during runtime
- Size accessed with **`.size()`** method
- Includes header: **`#include <vector>`**

## Example:

```
std::vector<int> v = {1, 2, 3};
```

```
v.push_back(4);
```

```
v.pop_back();
```

# Linked List

- Dynamic collection of elements connected by pointers (implemented as doubly-linked list in C++)
- Stored in **non-contiguous memory** locations
- Declared with syntax: **std::list<type> listName**

## Example:

```
std::list<int> numbers
```

- Can be initialized at declaration:

```
std::list<int> lst = {1, 2, 3}
```

- **Can resize dynamically** during runtime
- Size accessed with **.size()** method
- Includes header **#include <list>**

- **Example:**

```
std::list<int> l = {1, 2, 3};  
l.push_front(0);
```

# Stack

- **LIFO** (Last In, First Out) data structure
- Elements added and removed from the **top** only
- Declared with syntax:

**std::stack<type> stackName**

**Example:**

```
std::stack<int> numbers
```

- Cannot be initialized with list at declaration
- **Can resize dynamically** during runtime
- Size accessed with **.size()** method
- Includes header **#include <stack>**
- **Example:**

```
std::stack<int> s;
```

```
s.push(1);
```

```
s.pop();
```

# Queue

- **FIFO** (First In, First Out) data structure
- Elements added at **back** and removed from **front**
- Declared with syntax: **std::queue<type> queueName**

- **Example:**

```
std::queue<int> numbers
```

- Cannot be initialized with list at declaration
- **Can resize dynamically** during runtime
- Size accessed with **.size()** method
- Includes header **#include <queue>**

- **Example:**

```
std::queue<int> q;  
q.push(1);  
q.pop();
```

# Heap

- **Priority queue** data structure (max-heap by default)
- Elements automatically ordered by **priority/value**
- Declared with syntax:

**`std::priority_queue<type> heapName`**

- **Example:**

```
std::priority_queue<int> numbers
```

- Can be initialized from container: `std::priority_queue<int> pq(vec.begin(), vec.end())`
- **Can resize dynamically** during runtime
- Size accessed with **`.size()`** method
- Includes header **`#include <queue>`**

- **Example:**

```
std::priority_queue<int> h;
```

```
h.push(3); h.push(1);
```

```
h.top(); // returns 3
```

# Hash Table

- **Key-value pairs** with fast  $O(1)$  average lookup
- Uses **hash function** to map keys to indices
- Declared with syntax: **`std::unordered_map<keyType, valueType> mapName`**
- **Example:**  
`std::unordered_map<string, int> ages`
- Can be initialized at declaration: `std::unordered_map<string, int> map = {{"key", 1}}`
- **Can resize dynamically** during runtime
- Size accessed with **`.size()`** method
- Includes header **`#include <unordered_map>`**
- **Example:**  
`std::unordered_map<string, int> m;`  
`m["alice"] = 25;`  
`m.at("alice");`

# Tree

- **Hierarchical** data structure with nodes and edges
- Each node has **parent-child relationships** (except root)
- No built-in tree class - typically **implemented manually**

- **Example:**

```
struct TreeNode {  
    int val;  
    TreeNode* left;  
    TreeNode* right; };
```

- Cannot be initialized with simple syntax
- **Can resize dynamically** by adding/removing nodes
- Size calculated by **traversing all nodes**
- No standard header required (custom implementation)
- **Example:**

```
TreeNode* root = new TreeNode(1);  
root->left = new TreeNode(2);
```



# ARRAY

# Arrays

- **Memory layout:** hold values in a **contiguous** block of memory.
- **Fixed Size:** the size of an array is defined when it is created and cannot be changed.  
However, high-level languages have different implementations, making it dynamic.
- **Homogeneous elements:** all elements are of the same data type (int, float, char...)
- **Efficiency:** accessing elements by index is very efficient  $O(1)$ , since each index maps directly to a memory location. Also, range scans benefit from CPU cache lines since arrays are stored in contiguous blocks of memory.

# Problem – 1. Two Sum

Easy



LeetCode

[leetcode.com/problems/two-sum](https://leetcode.com/problems/two-sum)

## Problem

- Given an **array** of numbers and a **target**, example: **array** [2,7,11,15] and **target** 9
- Return indices of two numbers where they add up to **target**
- **Output:** [0,1]

$\text{array}[0] + \text{array}[1] = 2 + 7 = 9$

# Solution – 1. Two Sum

Easy



LeetCode

[leetcode.com/problems/two-sum](https://leetcode.com/problems/two-sum)

## Solution

- Iterative over each number in the array
- Calculate the difference between target and each number, example:  
 $\text{array}[0] = 2, \text{ target } 9, \text{ then } 9 - 2 = 7$
- Now we know we need the number **7** to sum up to **9**
- Check in a *hashmap* if we have 7 in some part of the array  
 $\text{hash}[7] \text{ exists?}$
- If yes, return the current index and the index of 7
- If not, store the index of the current number in the hashmap for future evaluation

$\text{hash}[2] = 0$

# Code – 1. Two Sum

Easy



LeetCode

[leetcode.com/problems/two-sum](https://leetcode.com/problems/two-sum)

## Code Time: $O(n)$ Space: $O(n)$

```
vector<int> twoSum(vector<int>& nums, int target) {
    std::unordered_map<int, int> numMap;
    // n being the size of nums
    for (int i = 0; i < nums.size(); i++) {
        // current number of the array
        int number = nums[i];
        int diff = target - number;

        // check if the difference is in some part of the array
        // by using a hashmap
        if (numMap.find(diff) != numMap.end()) {
            return { numMap[diff], i};
        }

        // register the current number index
        numMap[number] = i;
    }
    // no matches
    return {};
}
```

# Problem – 217. Contains Duplicate

Easy



LeetCode

[leetcode.com/problems/contains-duplicate](https://leetcode.com/problems/contains-duplicate)

## Problem

- You are given an array of numbers
- Return any value that appears at least twice

## Solution

- Loop through the array
- Check if the value is in a hash table
- Return **true** if the value exist
- The problem requires at least twice, but one modification may be having a specific count

# Code – 217. Contains Duplicate

Easy



LeetCode

[leetcode.com/problems/contains-duplicate](https://leetcode.com/problems/contains-duplicate)

**Code** Time:  $O(n)$  Space:  $O(n)$

```
bool containsDuplicate(vector<int>& nums) {
    unordered_map<int, int> seen;
    for (int i = 0; i < nums.size(); ++i) {
        if (seen[nums[i]] == 1) {
            return true;
        }
        seen[nums[i]]++;
    }
    return false;
}
```

**Another solution (less flexible)**

```
bool containsDuplicate(vector<int>& nums) {
    unordered_map<int, bool> seen;
    for (const auto& num : nums) {
        if (seen[num]) {
            return true;
        }
        seen[num] = true;
    }
    return false;
}
```

# Problem – 238. Product of Array Except Self

Medium



LeetCode

[leetcode.com/problems/product-of-array-except-self](https://leetcode.com/problems/product-of-array-except-self)

## Problem Statement

- You are given an integer array **nums**
- Return another array where each element is multiplied by all the elements except itself
- Example:

```
nums = [14,2,5,99]
```

```
nums[0] = 2 * 5 * 99 (all except 14)
```

```
nums[1] = 14 * 5 * 99 (all except 2)
```

```
nums[2] = 14 * 2 * 99
```

```
nums[3] = 14 * 2 * 5
```



# Solution – 238. Product of Array Except Self

Medium



LeetCode

[leetcode.com/problems/product-of-array-except-self](https://leetcode.com/problems/product-of-array-except-self)

## Solution

- Go over the array once and calculate the product of the left side. Example:

```
nums = [14,2,5]
```

```
left[0] = 1 (think of multiplying all elements before 14, so 1 because there is none)
```

```
left[1] = 14 (all elements from the left multiplied, except 2)
```

```
left[2] = 14 * 2 = 28 (all elements from the left multiplied, except 5)
```

```
left = [1, 14, 28]
```

- Using the same logic, do the same calculation but starting from the right

```
right[2] = 1 (no elements after 5)
```

```
right[1] = 5 (only 5 after 2)
```

```
right[0] = 2 * 5 = 10
```

```
right = [10, 5, 1]
```

- Multiply each element from left and right:

```
left ⊙ right = [1, 14, 28] ⊙ [10,5,1] = [10, 70, 28]
```

# Code – 238. Product of Array Except Self

Medium



LeetCode

[leetcode.com/problems/product-of-array-except-self](https://leetcode.com/problems/product-of-array-except-self)

**Code** Time:  $O(n)$  Space:  $O(n)$

```
vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> output(n, 1);
    vector<int> right(n, 1);

    // calculate left first
    for (int i = 1; i < n; ++i) {
        output[i] = nums[i - 1] * output[i - 1];
    }
    // calculate right
    for (int i = n - 1; i >= 0; --i) {
        right[i] = nums[i + 1] * right[i + 1];
        output[i] = output[i] * right[i];
    }

    /* or you can save some space using this logic,
       although I don't find it as intuitive as the previous one
    int right = 1;
    for (int i = n - 1; i >= 0; --i) {
        output[i] *= right;
        right *= nums[i];
    }
    */

    return output;
}
```

# Code – 238. Product of Array Except Self

Medium



LeetCode

[leetcode.com/problems/product-of-array-except-self](https://leetcode.com/problems/product-of-array-except-self)

**Code (better)** Time:  $O(n)$  Space:  $O(n)$

```
vector<int> productExceptSelf(vector<int>& nums) {
    vector<int> res;

    int prefix = 1;
    for (int i = 0; i < nums.size(); ++i) {
        res.push_back(prefix);
        prefix *= nums[i];
    }

    int suffix = 1;
    for (int i = nums.size() - 1; i >= 0; --i) {
        res[i] *= suffix;
        suffix *= nums[i];
    }

    return res;
}
```

# Problem – 53. Maximum Subarray

Medium



LeetCode

[leetcode.com/problems/maximum-subarray](https://leetcode.com/problems/maximum-subarray)

## Problem

- You are given an array `nums`
- Find the subarray with the largest sum

- **Example:**

`nums = [-2,1,-3,4,-1,2,1,-5,4]`

`output = 6`

The subarray `[4,-1,2,1]` has the largest sum 6.

# Problem – 53. Maximum Subarray

Medium



LeetCode

[leetcode.com/problems/maximum-subarray](https://leetcode.com/problems/maximum-subarray)

## Solution

- Use Kadane's algorithm to find the maximum sum of a contiguous subarray in linear time
- Core idea:
  - at each index, either:
    1. start a new subarray at **nums[i]** or
    2. extend the current one by adding **nums[i]**

# Arrays – Kadane's algorithm

- Kadane's algorithm is a dynamic programming algorithm to solve **maximum subarray sum**
- At every **index i**:  
start a new subarray at **i**  
extend the previous subarray to include **array[i]**

- **Algorithm**

## 1. Initialize:

```
int maxSoFar = array[0];  
int maxEndingHere = array[0];
```

## 2. Loop through the array

```
for (int i = 1; i < array.size(); ++i) {  
    maxEndingHere = max(array[i], maxEndingHere + array[i]);  
    maxSoFar = max(maxSoFar, maxEndingHere);  
}
```

## 3. Return maxSoFar;

# Problem – 53. Maximum Subarray

Medium



LeetCode

[leetcode.com/problems/maximum-subarray](https://leetcode.com/problems/maximum-subarray)

**Code** Time:  $O(n)$  Space:  $O(1)$

```
int maxSubArray(vector<int>& nums) {  
    int maxSum = nums[0];  
    int currentSum = nums[0];  
    for (int i = 1; i < nums.size(); ++i) {  
        currentSum = max(nums[i], currentSum + nums[i]);  
        maxSum = max(maxSum, currentSum);  
    }  
    return maxSum;  
}
```

# Problem – 152. Maximum Product Subarray

Medium



LeetCode

[leetcode.com/problems/maximum-product-subarray](https://leetcode.com/problems/maximum-product-subarray)

## Problem

- You are given an array **nums**
- Find a subarray that has the largest product and return the product
- The array may contain negative numbers

- **Example:**

`nums = [2, 3, -2, 4]`

`output = 6`

`[2,3]` has the largest 6 ( $2 * 3$ )



# Problem – 152. Maximum Product Subarray

Medium



LeetCode

[leetcode.com/problems/maximum-product-subarray](https://leetcode.com/problems/maximum-product-subarray)

## Solution

- Use a modified version of Kadane's algorithm
- Keep track of the minimum and maximum product
- Once the current number is negative, swap minimum product with maximum product
- Check the largest product between maximum product and the final result

# Problem – 152. Maximum Product Subarray

Medium



LeetCode

[leetcode.com/problems/maximum-product-subarray](https://leetcode.com/problems/maximum-product-subarray)

**Code** Time:  $O(n)$  Space:  $O(1)$

```
int maxProduct(vector<int>& nums) {
    int result = nums[0];
    int maxProd = nums[0];
    int minProd = nums[0];
    for (int i = 1; i < nums.size(); ++i) {
        if (nums[i] < 0) {
            swap(minProd, maxProd);
        }

        minProd = min(nums[i], nums[i] * minProd); // -2
        maxProd = max(nums[i], nums[i] * maxProd); // -30

        result = max(result, maxProd);
    }
    return result;
}
```

# Problem – 153. Find Minimum in Rotated Sorted Array

Medium

 [leetcode.com/problems/find-minimum-in-rotated-sorted-array](https://leetcode.com/problems/find-minimum-in-rotated-sorted-array)

## Problem

- You are given a sorted array but “rotated”
- Rotated means the elements are displaced in order
- Return the **minimum element**

- **Example:**

nums = [3,4,5,1,2]

output = 1 (minimum element)

# Solution – 153. Find Minimum in Rotated Sorted Array

Medium

 [leetcode.com/problems/find-minimum-in-rotated-sorted-array](https://leetcode.com/problems/find-minimum-in-rotated-sorted-array)

## Solution

- Perform an adapted binary search
- Example:

[3,4,5,1,2]

**left = 3, mid = 5, right = 2**

You find mid (5), but have to go right, so adjust left:

```
if (mid > right)
```

```
    left = mid + 1
```

```
else
```

```
    right = mid
```

# Code – 153. Find Minimum in Rotated Sorted Array

Medium



LeetCode

[leetcode.com/problems/find-minimum-in-rotated-sorted-array](https://leetcode.com/problems/find-minimum-in-rotated-sorted-array)

**Code** Time:  $O(\log n)$  Space:  $O(1)$

```
int findMin(vector<int>& nums) {
    int left = 0;
    int right = nums.size() - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] > nums[right]) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }
    return nums[left];
}
```

# Problem - Best Time to Buy and Sell Stock

Easy



LeetCode

[leetcode.com/problems/best-time-to-buy-and-sell-stock](https://leetcode.com/problems/best-time-to-buy-and-sell-stock)

## Problem Statement

- You are given an integer **array** of stock prices
- Choose a **price[i]** to buy and **price[i]** to sell where you achieve maximum profits

- **Example:**

`prices = [9, 1, 3, 4]`

- **Output:** [1,3]

`array[3] - array[1] = 4 - 1 = 3`

# Solution - Best Time to Buy and Sell Stock

Easy



LeetCode

[leetcode.com/problems/best-time-to-buy-and-sell-stock](https://leetcode.com/problems/best-time-to-buy-and-sell-stock)

## Solution

- Initialize **profit = 0**
- Initialize **lowestBuyPrice = prices[0]**
- Loop through the prices
- Track the lowest buy price → **min(lowestBuyPrice, prices[i])**
- Check if selling “today” will make the maximum profit and update profit:  
**max(prices[i] - buy > profit, profit)**
- Update profit  
max(prices[i] - buy

# Code - Best Time to Buy and Sell Stock

Easy



LeetCode

[leetcode.com/problems/best-time-to-buy-and-sell-stock](https://leetcode.com/problems/best-time-to-buy-and-sell-stock)

## Code (simplified) Time: $O(n)$ Space: $O(1)$

```
int maxProfit(vector<int>& prices) {  
    int profit = 0;  
    int buy = prices[0];  
    for (auto i = 1; i < prices.size(); i++) {  
        buy = min(buy, prices[i]);  
        profit = max(profit, prices[i] - buy)  
    }  
    return profit;  
}
```



# Code - Best Time to Buy and Sell Stock

Easy



LeetCode

[leetcode.com/problems/best-time-to-buy-and-sell-stock](https://leetcode.com/problems/best-time-to-buy-and-sell-stock)

## Code (optimized) Time: $O(n)$ Space: $O(1)$

- Same logic, but with better branch prediction and less computation

```
int maxProfit(vector<int>& prices) {  
    int profit = 0;  
    int buy = prices[0];  
    for (auto i = 1; i < prices.size(); i++) {  
        if (prices[i] < buy) {  
            buy = prices[i];  
        } else if (prices[i] - buy > profit) {  
            profit = prices[i] - buy;  
        }  
    }  
    return profit;  
}
```

# Problem - Best Time to Buy and Sell Stock II

Medium



LeetCode

[leetcode.com/problems/best-time-to-buy-and-sell-stock-ii](https://leetcode.com/problems/best-time-to-buy-and-sell-stock-ii)

## Problem

- You are given an integer **array** of stock prices
- Choose a **price[i]** to buy and **price[i]** to sell where you achieve maximum profits
- You can buy/sell **multiple times**, but only hold **at most one** transaction at a time
- Output is the **maximum profits**
- **Example:**

`prices = [9, 1, 3, 4]`

**Output:**  $2 + 1 = 3$

`buy (price = 1), sell (price = 3), profit = 2`

`buy (price = 3), sell (price = 4), profit = 1`

# Solution - Best Time to Buy and Sell Stock II

Medium

 LeetCode [leetcode.com/problems/best-time-to-buy-and-sell-stock-ii](https://leetcode.com/problems/best-time-to-buy-and-sell-stock-ii)

## Solution

- Loop through the array starting from index 1
- If current **price[i]** is lower than previous **price[i - 1]**, buy and sell

- **Example:**

`prices = [1, 8, 4]`      `prices[0] = 1, prices[1] = 8, prices[2] = 4`

`prices[0] < prices[1] → true, profit = 8 - 1 = 7`

`prices[2] < prices[1] → false, do nothing`

# Code - Best Time to Buy and Sell Stock II

Medium



LeetCode

[leetcode.com/problems/best-time-to-buy-and-sell-stock-ii](https://leetcode.com/problems/best-time-to-buy-and-sell-stock-ii)

**Code** Time:  $O(n)$  Space:  $O(1)$

```
int maxProfit(vector<int>& prices) {  
    int profit = 0;  
    for (int i = 1; i < prices.size(); ++i) {  
        if (prices[i] > prices[i-1]) {  
            profit += prices[i] - prices[i - 1];  
        }  
    }  
    return profit;  
}
```

# Problem – 33. Search in Rotated Sorted Array

Medium



LeetCode

[leetcode.com/problems/search-in-rotated-sorted-array](https://leetcode.com/problems/search-in-rotated-sorted-array)

## Problem

- Variation of *Find Minimum in Sorted Rotated Array* problem
- You are given a sorted **array** but “rotated” and a target number **n**
- Rotated means the elements are displaced in order
- Search the number **n** and return its index

- **Example**

`nums = [4,5,6,7,0,1,2], target = 0`

**Output:** 4

**4** is the index where the target number **0** is located

# Problem – 33. Search in Rotated Sorted Array

Medium



LeetCode

[leetcode.com/problems/search-in-rotated-sorted-array](https://leetcode.com/problems/search-in-rotated-sorted-array)

## Solution

- Perform a binary search with some modification
- One side is always sorted, so find which side (left or right)
- Check if the target is in the range of the sorted side and adjust mid

Example:

[2,4,5,6,7,0,1] target = 0

1. Find mid (6)
2. Find the sorted side (left) = [2,4,5]
3. Check if your target is in this side. Is **target** between 2 and 5?
4. Adjust mid to search at the other side if not, otherwise continue searching at the same side

# Problem – 33. Search in Rotated Sorted Array

Medium



LeetCode

[leetcode.com/problems/search-in-rotated-sorted-array](https://leetcode.com/problems/search-in-rotated-sorted-array)

**Code** Time:  $O(\log n)$  Space:  $O(1)$

```
int search(vector<int>& nums, int target) {
    int left = 0;
    int right = nums.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;

        // figure it out the sorted side
        if (nums[mid] >= nums[0]) {
            // left side is sorted
            // is target within this range?
            if (target >= nums[0] && target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else {
            // right side is sorted
            // is target within this range?
            if (target <= nums[right] && target > nums[mid]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }
    return -1;
}
```

# Problem – 15. 3Sum

Medium



LeetCode

[leetcode.com/problems/3sum](https://leetcode.com/problems/3sum)

## Problem

- You are given an array of integer **nums**
- Find distinct triples that the final sum is equal to zero
- **Example:**

`nums = [-1,0,1,2,-1,-4]`

### Output:

`[[-1,-1,2],[-1,0,1]]`

### Explanation:

$\text{nums}[0] + \text{nums}[1] + \text{nums}[2] = (-1) + 0 + 1 = 0.$

$\text{nums}[1] + \text{nums}[2] + \text{nums}[4] = 0 + 1 + (-1) = 0.$

$\text{nums}[0] + \text{nums}[3] + \text{nums}[4] = (-1) + 2 + (-1) = 0.$

The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`.





LeetCode

[leetcode.com/problems/3sum](https://leetcode.com/problems/3sum)

## Solution

- Use three pointers: **i**, **j** and **k**
- Sort the array. This is necessary to move the pointers **j** and **k**
- Pointer **i** starts at the beginning the array
- Pointer **j** starts at  $i + 1$  (second position)
- Pointer **k** starts at the end of the array
- Pointer **i** always move forward until the end of the array
- For each value of **i**, **j** and **k** will move either forward or backward, depending on the results of the sum
- Once find a sum == 0, add to a set to guarantee no duplicates

# Code – 15. 3Sum

Medium



LeetCode

[leetcode.com/problems/3sum](https://leetcode.com/problems/3sum)

**Code** Time:  $O(n^2 \log n)$  Space:  $O(n^2)$

```
vector<vector<int>> threeSum(vector<int>& nums) {
    set<vector<int>> triplets;
    sort(nums.begin(), nums.end());
    for (int i = 0; i < nums.size() - 2; ++i) {
        int j = i + 1;
        int k = nums.size() - 1;
        // it is a solution
        while (j < k) {
            int sum = nums[i] + nums[j] + nums[k];
            if (sum == 0) {
                triplets.insert({nums[i], nums[j], nums[k]});
                j++;
                k--;
            }
            if (sum < 0) {
                j++;
            } else {
                k--;
            }
        }
    }
    vector<vector<int>> result;
    // convert the solutions to the expected return
    for (const auto& t : triplets) {
        result.push_back(t);
    }

    return result;
}
```

Note: why not a unordered\_set which is hash-based?

# Problem – 11. Container With Most Water

Medium



LeetCode

[leetcode.com/problems/container-with-most-water](https://leetcode.com/problems/container-with-most-water)

## Problem Statement

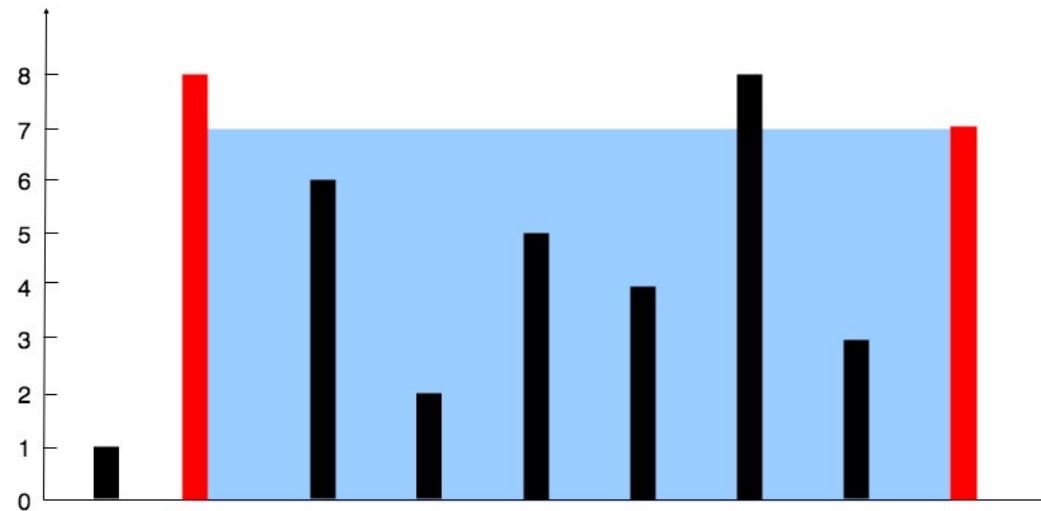
- You are given an integer array **height**
- Find two lines that together with x-axis form a container with most water
- Example:

### Input:

height = [1,8,6,2,5,4,8,3,7]

### Output:

49



# Solution – Container With Most Water

Medium



LeetCode

[leetcode.com/problems/container-with-most-water](https://leetcode.com/problems/container-with-most-water)

## Solution

- Initialize the maximum area **maxArea = 0**
- Initialize two pointers, **left = 0** and **right = height.size - 1**
- Loop while pointer **left < right**
- Calculate the area:  
**area = min(height[left], height[right]) \* (right - left)**
- Update the global maximum area:  
**maxArea = max(maxArea, area)**
- Move the smallest pointer (increment **left** or decrement **right**)
- Return **maxArea**

# Code – 11. Container With Most Water

Medium



LeetCode

[leetcode.com/problems/container-with-most-water](https://leetcode.com/problems/container-with-most-water)

**Code** Time:  $O(n)$  Space:  $O(1)$

```
int maxArea(vector<int>& height) {  
    // left and right = positions  
    int left = 0;  
    int right = height.size() - 1;  
    int maxArea = 0;  
    while (left < right) {  
        int area = min(height[left], height[right]) * (right - left);  
        maxArea = max(maxArea, area);  
        // adjust left and right based on 'greedy' algorithm  
        // move from the lowest height  
        if (height[left] < height[right]) {  
            left++;  
        } else {  
            right--;  
        }  
    }  
    return maxArea;  
}
```

# Problem – 268. Missing Number

Easy



LeetCode

<https://leetcode.com/problems/missing-number>

## Problem

- You are given an **array of nums** with **n** distinct numbers
- Return only the number in the range that is missing from the array

- **Example:**

nums = [3, 0, 1]

**Output:** 2

from the sequence 0,1,x,3 the missing number x is 2

# Solution – 268. Missing Number

Easy



LeetCode

<https://leetcode.com/problems/missing-number>

## Solution

- You have an array with **size n**. Example:

`nums = [3,0,1]`

`size = 3`

- It is expected the sum **0 + 1 + 2 + ... + n**:

`0 + 1 + 2 + 3 = 6`

- The **expected sum** can be calculated with **Gauss Formula**:  $\frac{n(n+1)}{2}$

`n = 3`

`n * (n + 1) / 2 = 3 * (3 + 1) / 2 = 6`

- Once you know the expected sum, calculate the sum from the elements of the array:

`sum = 3 + 0 + 1 = 4`

- Subtract from the expected value to find out the number:

`6 - 4 = 2 is the missing number`

# Code – 268. Missing Number

Easy



LeetCode

<https://leetcode.com/problems/missing-number>

**Code** Time:  **$O(n)$**  Space:  **$O(1)$**

```
int missingNumber2(vector<int>& nums) {
    int n = nums.size();
    // Gauss's formula: the sum of numbers 0 to n is n * (n + 1) / 2
    int expected = n * (n + 1) / 2;
    // calculate actual sum
    int actual = std::accumulate(nums.begin(), nums.end(), 0);
    // result is the expected - actual
    return expected - actual;
}
```



**STRING**

# Problem – 3. Longest Substring Without Repeating Characters

Medium



LeetCode

[leetcode.com/problems/longest-substring-without-repeating-characters](https://leetcode.com/problems/longest-substring-without-repeating-characters)

## Problem Statement

- You are given a string and the goal is to find the longest substring without repeating characters

- **Example**

**Input:** "abcbd"

**Output:** 4 (abcd since "b" is repeated)

# Solution – 3. Longest Substring Without Repeating Characters

Medium



LeetCode

[leetcode.com/problems/longest-substring-without-repeating-characters](https://leetcode.com/problems/longest-substring-without-repeating-characters)

## Solution

- Use sliding window algorithm (left and right)
- Loop through the string
- Try to find if the current character is already added by using unordered set or bitmap
- If added, remove from the set alongside with others using left pointer
- If not, add to the unordered set or bitmap
- Maximum length will be  $\text{right} - \text{left} + 1$

# Example – 3. Longest Substring Without Repeating Characters

Medium



LeetCode

[leetcode.com/problems/longest-substring-without-repeating-characters](https://leetcode.com/problems/longest-substring-without-repeating-characters)

## Example

- String: abcbd. Our goal is to return 3 (**abc**bd)
- Initialize **maxLength = 0**
- Loop through the string

**Iteration 1:** left = 0, right = 0, string[left] = 'a',

bitmap = ['a'] ('a' is not in bitmap, add), **maxLength = max(maxLength, right - left + 1) = 1**

**Iteration 2:** left = 0, right = 1, string[right] = 'b'

bitmap = ['a','b'], **maxLength = 2**

**Iteration 3:** left = 0, right = 2, string[right] = 'c'

bitmap = ['a','b','c'], **maxLength = 3**

**Iteration 4:** left = 0, right = 3, string[right] = 'b'

bitmap = ['a','b','c','b']

'b' is already in the bitmap. start "clearing" the character using left:

**Iteration 4a:** left = 0, string[left] = 'a' is different from 'b', so remove 'a'

bitmap = ['b','c','b']

**Iteration 4b:** left = 1, string[left] = 'b' is the same as the repeated one, remove

bitmap = ['c','b']

**Iteration 5:** left = 1, right = 4, string[right] = 'd'

bitmap = ['c','b','d']

# Code – 3. Longest Substring Without Repeating Characters

Medium

## Code (unordered\_set)

- Use unordered\_set when question requires unicode chars

```
int lengthOfLongestSubstring(string s) {
    int maxLength = 0;
    int left = 0, right = 0;
    // track the seen characters
    unordered_set<char> seen;
    for (right = 0; right < s.size(); ++right) {
        char currentChar = s[right];
        // if currentChar is in the set, clean
        // the character and everything from left of it
        // basically, reset the longest substring
        while (seen.count(currentChar)) {
            char c = s[left];
            seen.erase(c);
            left++;
        }
        // insert the current read character
        seen.insert(currentChar);
        // set max length
        maxLength = max(maxLength, right - left + 1);
    }
    return maxLength;
}
```

# Code – 3. Longest Substring Without Repeating Characters

Medium

## Code (bitmap)

- Using bitset: create a bitmask with 128 bits where each bit represent a character
- Optimal solution for ASCII since ASCII size is 127 characters
- Unicode / UTF-8 can represent over 1.1 million characters, so use **unordered\_set** approach instead

```
int lengthOfLongestSubstring(string s) {  
    std::bitset<128> bitmask;  
    uint32_t left = 0;  
    uint32_t maxLength = 0;  
  
    for (uint32_t right = 0; right < s.length(); ++right) {  
        uint32_t bitIndex = s[right];  
        // if char is already in the bitmask, move left until we reset the bits  
        while (bitmask.test(bitIndex)) {  
            bitmask.reset(s[left]);  
            ++left;  
        }  
  
        bitmask.set(bitIndex);  
        maxLength = std::max(maxLength, right - left + 1);  
    }  
    return maxLength;  
}
```

# Problem – 424. Longest Repeating Character Replacement

Medium

 [leetcode.com/problems/longest-repeating-character-replacement](https://leetcode.com/problems/longest-repeating-character-replacement)

## Problem

- You are given a **string s** and an **integer k**
- You can replace one character by any other uppercase English character **k** times
- Return the longest substring with the same character

- **Example:**

**Input:**

`s = "ABAB", k = 2`

**Output:** 4

Replace the two 'A's with two 'B's or vice versa.

# Problem – 424. Longest Repeating Character Replacement

Medium

 [leetcode.com/problems/longest-repeating-character-replacement](https://leetcode.com/problems/longest-repeating-character-replacement)

## Solution

- Start with two pointers: left and right
- Keep track of the frequencies of each letter in a **vector<int>** since we know there are 26 characters
- Initialize **maxFreq** to keep track of the letter with maximum frequency
- Initialize **maxLength** to keep track of the maximum substring
- Go over the string, and for each iteration:
  - calculate the windowSize
  - calculate the maximum frequency
  - check how many replacements is needed. That is, windowSize - maxFreq
  - if no replace can be done ( $k < \text{replaces}$ ) then move left pointer to the right



# Problem – 424. Longest Repeating Character Replacement

Medium

 [leetcode.com/problems/longest-repeating-character-replacement](https://leetcode.com/problems/longest-repeating-character-replacement)

**Code** Time:  $O(n)$  Space:  $O(1)$

```
int characterReplacement(string s, int k) {
    int left = 0;
    int maxLength = 0;
    int maxFreq = 0;
    vector<int> freq(26, 0);
    for (int right = 0; right < s.size(); ++right) {
        int index = s[right] - 'A';
        int windowSize = right - left + 1;
        // keep track of the frequencies
        freq[index]++;
        maxFreq = max(maxFreq, freq[index]);

        // check if the subwindow need to change
        int needReplace = windowSize - maxFreq;
        if (k < needReplace) {
            // need to move sub window
            int leftIndex = s[left] - 'A';
            freq[leftIndex]--;
            left++;
            windowSize = right - left + 1;
        }
        maxLength = max(maxLength, windowSize);
    }
    return maxLength;
}
```

# Problem – 76. Minimum Window Substring

Hard



LeetCode

[leetcode.com/problems/minimum-window-substring](https://leetcode.com/problems/minimum-window-substring)

## Problem

- You are given two strings **s** and **t** of lengths **m** and **n**
- Return the minimum window substring of **s** where every character in **t** is included in the window

- **Example:**

### Input

**s** = "ADOBECODEBANC"

**t** = "ABC"

### Output

"BANC"

- The minimum substring "BANC" includes A, B and C.

# Solution – 76. Maximum Window Substring

Hard



LeetCode

[leetcode.com/problems/minimum-window-substring](https://leetcode.com/problems/minimum-window-substring)

## Solution

- **Grow** → first valid window: move right until the window has every required char (use a need table and a have table plus **formed == distinctNeeded** to know this)
- **Prune** ← from left: while the window is still valid, drop **s[left]** and advance left-stop as soon as removing a char would break validity
- **Record** current window length as a candidate answer.
- **Resume** growing right, repeating the **grow → prune ← record** cycle until right reaches the end.
- Two pointers only move forward

# Code – 76. Maximum Window Substring

Hard



LeetCode

[leetcode.com/problems/minimum-window-substring](https://leetcode.com/problems/minimum-window-substring)

**Code** Time:  $O(|s| + |t|)$  Space:  $O(k)$  where  $|s|$  means the size of "s" and  $|t|$  the size of "t".  $k$  is the number of distinct characters in  $t$

```
string minWindow(string s, string t) {
    if (t.size() > s.size()) return "";

    // characters I need (t)
    unordered_map<char, int> need;
    // current window
    unordered_map<char, int> window;
    int left = 0;
    int right = 0;
    int start = 0;

    // number of valid characters
    int valid = 0;
    int minLength = INT_MAX;

    // populate need
    // need['A'] = 1
    // need['B'] = 1
    // need['C'] = 1
    for (const auto& c : t) {
        need[c]++;
    }

    // traverse the string
    while (right < s.size()) {
        // current char
        char c = s[right];
        // increase right
        right++;

        // do we need this character?
        if (need.count(c)) {
            // add to the current window
            window[c]++;
            // have we reached the number of characters we need?
            // then increase valid. It doesn't matter if have more,
            // what matters is exactly the number
            if (window[c] == need[c]) {
                valid++;
            }
        }
    }
}
```

# Code – 76. Maximum Window Substring

Hard



LeetCode

[leetcode.com/problems/minimum-window-substring](https://leetcode.com/problems/minimum-window-substring)

**Code (continue)** Time:  $O(|s| + |t|)$  Space:  $O(k)$  where  $|s|$  means the size of "s" and  $|t|$  the size of "t".  $k$  is the number of distinct characters in  $k$

```
// this will run once our window is now valid,
// meaning having all characters from need
// now we want to prune this because we want the minimum
// window substring
while (valid == need.size()) {
    int windowSize = right - left;
    // minLength hold the global minimum substring
    // current valid windowSize is smaller, update it
    if (windowSize < minLength) {
        minLength = windowSize;
        // we need to keep track where the substring starts
        start = left;
    }

    // prune substring
    // check if s[left] is needed
    // is the character I'm pruning, needed?
    char charToPrune = s[left];
    left++;
    if (need.count(charToPrune)) {
        // ok we need this character, and the amount we have is
        // exactly what we need (we don't have more to 'spare')
        if (window[charToPrune] == need[charToPrune]) {
            // invalidate. So break the while loop and
            // continue moving right
            valid--;
        }
        // character is removed
        window[charToPrune]--;
    }
}
}
```

```
// return
if (minLength == INT_MAX) return "";
return s.substr(start, minLength);
}
```

# Problem – 242. Valid Anagram

Easy



LeetCode

[leetcode.com/problems/valid-anagram](https://leetcode.com/problems/valid-anagram)

## Problem

- You are given two strings **s** and **t**
- Return true if **t** is an anagram of **s**

- **Example:**

t = word

s = dwor

**Output:** true

both have the same number of same characters

# Problem – 242. Valid Anagram

Easy



LeetCode

[leetcode.com/problems/valid-anagram](https://leetcode.com/problems/valid-anagram)

## Solution

- Initialize a vector of integers to keep track of the count of each letter
- Loop over **s** and increase the count of each character found
- Then, loop over **t** and decrease the count of each character found
- Finally, loop over the vector and if there is one count greater than 0, return false

# Problem – 242. Valid Anagram

Easy



LeetCode

[leetcode.com/problems/valid-anagram](https://leetcode.com/problems/valid-anagram)

**Code** Time: **O(n)** Space: **O(1)**

```
bool isAnagram(string s, string t) {  
    // count the number of characters in 's', store in a vector  
    // go over the vector and check if it's empty  
    vector<int> letters(26);  
    for (const auto& c : s) {  
        letters[c - 'a']++;  
    }  
    for (const auto& c : t) {  
        letters[c - 'a']--;  
    }  
    for (const auto& c : letters) {  
        if (c != 0) return false;  
    }  
    return true;  
}
```



# Problem – 49. Group Anagrams

Medium



LeetCode

[leetcode.com/problems/group-anagrams](https://leetcode.com/problems/group-anagrams)

## Problem

- You are given an array of strings, Example:  
`strs = ["eat", "tea", "tan", "ate", "nat", "bat"]`
- Group the anagrams together:  
`[["bat"], ["nat", "tan"], ["ate", "eat", "tea"]]`
- No anagram of "bat", where "nat" and "tan" are anagram so they're grouped together

# Solution – 49. Group Anagrams

Medium



LeetCode

[leetcode.com/problems/group-anagrams](https://leetcode.com/problems/group-anagrams)

## Solution

- Go over each word
- Sort the words

**Example:** ["eat", "tea", "tan", "ate", "nat", "bat"]

**After sorting:** ["aet", "aet", "ant", "aet", "ant", "abt"]

- Add the words in their respective buckets using a hashtable `unordered_map<string, vector<string>>`  
hash["aet"] = "eat", "tea", "ate"  
hash["ant"] = "tan", "nat"  
hash["abt"] = "bat"
- Go over the bucket and add to the results

# Code – 49. Group Anagrams

Medium



LeetCode

[leetcode.com/problems/group-anagrams](https://leetcode.com/problems/group-anagrams)

**Code** Time:  $O(n * k \log k)$  Space:  $O(n * k)$  where  $n$  is the number of strings in `strs` and  $k$  is the maximum length of a string in `strs`

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    // go over the strs
    // sort each of them, store it
    // ["eat","tea","tan","ate","nat","bat"]
    // ["aet","aet","ant","aet","ant","abt"]
    // hash["aet"] = ["eat","tea","ate"]
    unordered_map<string, vector<string>> hash;
    for (const auto& s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        hash[key].push_back(s);
    }
    // go over this hash map and push to the final output
    vector<vector<string>> result;
    for (const auto& [k, v] : hash) {
        result.push_back(v);
    }
    return result;
}
```

# Problem – Valid Parentheses

Easy



LeetCode

[leetcode.com/problems/valid-parentheses](https://leetcode.com/problems/valid-parentheses)

## Problem Statement

- You are given a string containing only the characters '(', ')', '{', '}', '[' and ']'
- A valid input have closed brackets by its own type

- **Example**

`()[]{} → valid`

`[]{}( → invalid`

`{()} → valid`

# Solution – Valid Parentheses

Easy



LeetCode

[leetcode.com/problems/valid-parentheses](https://leetcode.com/problems/valid-parentheses)

## Solution

- Loop through the string
- If **open** brackets (**{** push to a stack
- If **closed** brackets:
  - **pop** the last added bracket
  - **check** if the **closed** bracket corresponds to the **popped** bracket
  - if not, return false
- after the loop, **return true** if the **size** of the stack is empty (all brackets closed)

# Code – Valid Parentheses

Easy



LeetCode

[leetcode.com/problems/valid-parentheses](https://leetcode.com/problems/valid-parentheses)

## Code

Time:  $O(n)$  Space:  $O(n)$

```
bool isValid(string s) {
    // stack (LIFO)
    std::stack<char> brackets;
    // O(n)
    for (int i = 0; i < s.size(); ++i) {
        char bracket = s[i];
        if (bracket == '(' || bracket == '[' || bracket == '{') {
            brackets.push(bracket);
        } else {
            if (brackets.size() == 0) return false;
            char lastBracket = brackets.top();
            if (bracket == ')' && lastBracket != '(') return false;
            if (bracket == '}' && lastBracket != '{') return false;
            if (bracket == ']' && lastBracket != '[') return false;
            brackets.pop();
        }
    }
    // all brackets must be closed
    return brackets.size() == 0;
}
```

# Problem – 125. Valid Palindrome

Easy



LeetCode

[leetcode.com/problems/valid-palindrome](https://leetcode.com/problems/valid-palindrome)

## Problem

- You are given a **string s**
- Return **true** if it is a palindrome
- Note that the string may contain **non-alphanumeric characters** that should be ignored and **uppercase/lowercase** that must be considered the same

- **Example:**

input = "A man, a plan, a canal: Panama"

output = true

after removing non-alphanumeric characters (including spaces) and turning everything into lowercase (or uppercase), the resulting string is a palindrome

# Solution – 125. Valid Palindrome

Easy



LeetCode

[leetcode.com/problems/valid-palindrome](https://leetcode.com/problems/valid-palindrome)

## Solution

- **Remove non-alphanumeric** characters:

```
auto end = remove_if(s.begin(), s.end(), [](char& c) {  
    return !isalnum();  
});  
s.erase(end, s.end());
```

`remove_if` logically moves everything to the end of the string and return the iterator. Then, erase remove from the result of the iterator to the end of the strong

- **Transform** the string to lowercase:

```
transform(s.begin(), s.end(), s.begin(), [](char& c) {  
    return tolower(c);  
});
```

1<sup>st</sup> argument = beginning of string

2<sup>nd</sup> argument = end of the string

3<sup>rd</sup> argument = destination

4<sup>th</sup> argument = lambda function



# Solution – 125. Valid Palindrome

Easy



LeetCode

[leetcode.com/problems/valid-palindrome](https://leetcode.com/problems/valid-palindrome)

## Solution

- Have two pointers:

```
left = 0
```

```
right = s.size() - 1
```

- Loop incrementing left and decrementing right, checking the characters from both sides
- If they differ, **return false**
- At the end, **return true**

# Code – 125. Valid Palindrome

Easy



LeetCode

[leetcode.com/problems/valid-palindrome](https://leetcode.com/problems/valid-palindrome)

**Code** Time: **O(n)** Space: **O(1)**

```
bool isPalindrome(string s) {
    // transform everything into lowercase:
    // transform(begin, end, output begin)
    transform(s.begin(), s.end(), s.begin(), [](char& c) {
        return tolower(c);
    });

    // remove_if move everything that matches in the lambda
    // to the end of
    auto end = remove_if(s.begin(), s.end(), [](char& c) {
        return !isalnum(c);
    });
    s.erase(end, s.end());

    int left = 0;
    int right = s.size() - 1;

    while (left <= right) {
        if (s[left] != s[right]) return false;
        left++;
        right--;
    }
    return true;
}
```

# Problem – Minimum Number of Increments on Subarrays

Hard



LeetCode

[leetcode.com/problems/minimum-number-of-increments-on-subarrays-to-form-a-target-array](https://leetcode.com/problems/minimum-number-of-increments-on-subarrays-to-form-a-target-array)

## Problem Statement

- You are given an array of integers initialized with zeros (e.g. **[0,0,0,0]**)
- The goal is to reach some target (e.g. **[1, 2, 2, 3]**)
- The valid operations is to increment a subarray by one
- The output is the total number of operations

In this case:

**[1,1,1,1]** → increment the subarray starting from 0 to total size

**[1,2,2,2]** → increment the subarray starting from 1 to total size

**[1,2,2,3]** → increment the subarray starting and ending from the last element

**Output:** 3 (total number of operations)

# Solution – Minimum Number of Increments on Subarrays

Hard



LeetCode

[leetcode.com/problems/minimum-number-of-increments-on-subarrays-to-form-a-target-array](https://leetcode.com/problems/minimum-number-of-increments-on-subarrays-to-form-a-target-array)

## Solution

- Take this example:  
`target = [1000, 1, 1000]`
- The number of operations needed is equivalent to:
  - add 1 to each element: `[1,1,1]`
  - add 999 to the subarray `[0,0]`
  - add 999 to the subarray `[2,2]`
- Initialize total number of operations `totalOp = target[0] = 1000`  
This is the number of operations needed so far
- Loop through the array, ask if you need more operation or if the previous operation was enough:  
`target[1] > target[0] → 1 > 1000 → (false) can reuse so totalOp is still 1000`  
`target[2] > target[1] → 1000 > 1 → need more operation. Update totalOp:`  
`difference = 1000 - 1 = 999 (1000 more operations minus one operation already done previously)`  
`totalOp = 1000 + 999 = 1999 (sum the difference)`

# Code – Minimum Number of Increments on Subarrays

Hard



LeetCode

[leetcode.com/problems/minimum-number-of-increments-on-subarrays-to-form-a-target-array](https://leetcode.com/problems/minimum-number-of-increments-on-subarrays-to-form-a-target-array)

**Code** Time:  $O(n)$  Space:  $O(1)$

```
int minNumberOperations(vector<int>& target) {
    int totalOp = target[0];
    for (int i = 1; i < target.size(); ++i) {
        // can't reuse
        if (target[i - 1] < target[i]) {
            totalOp += target[i] - target[i - 1];
        }
    }
    return totalOp;
}
```

# Code [2] – Minimum Number of Increments on Subarrays

Hard



LeetCode

[leetcode.com/problems/minimum-number-of-increments-on-subarrays-to-form-a-target-array](https://leetcode.com/problems/minimum-number-of-increments-on-subarrays-to-form-a-target-array)

## Code (optimized)

```
int minNumberOperations(vector<int>& target) {  
    return target[0] +  
        inner_product(target.begin() + 1, target.end(),  
            target.begin(), 0,  
            plus<int>(),  
            [](int curr, int prev) { return max(curr - prev, 0); });  
}
```

this can be expressed using STL library inner\_product which is optimized.

Here is a good resource to explore it more:

Fast C++ by using SIMD Types with Generic Lambdas and Filters - Andrew Drakeford - CppCon 2022

<https://www.youtube.com/watch?v=sQvIPHuE9KY>

# Problem – 788. Rotated Digits

Medium



LeetCode

[leetcode.com/problems/rotated-digits](https://leetcode.com/problems/rotated-digits)

## Problem

- You are given a number **n**
- From the range between 1 to n, find “good” numbers
- A good number must meet **2 requirements**:
  - 1.** Be still valid after flipping: You physically “rotate” this number by 180 degrees, flip the number upside-down 2. The number can be either valid or invalid. For example, flipping **8** is still **8**, flipping **6** becomes **9**, but flipping **3**, becomes **ε** which is invalid.
  - 2.** Be a different digit after flipping. If you flip **1**, it is still a valid number but it is the same number (1), so it is not good. However, **16** is valid because it becomes a different number: **19**
- Return the the number of good numbers between **1** and **n**

# Problem – 788. Rotated Digits

Medium



LeetCode

[leetcode.com/problems/rotated-digits](https://leetcode.com/problems/rotated-digits)

## Solution

- The simplest and readable approach:
- Create a function to check if a number is good or not
- Go over the range  $(1, n)$  and check every number. If it is good, count as a valid
- Inside the function to check:
- Extract digit by digit from the number ( $\text{digit} = \text{num} \% 10$ )
- Check if the digit is valid (a.k.a “flippable”). In other words, return false if it is 3, 4 or 7.
- Now check the second condition (same number). So keep a bool “changed”, if you find a number that “changes”, mark changed as true. The numbers are 2, 5, 6 and 9, since when they flip they become different numbers
- Return “changed”



# Problem – 788. Rotated Digits

Medium



LeetCode

[leetcode.com/problems/rotated-digits](https://leetcode.com/problems/rotated-digits)

**Code** Time:  **$O(n \log n)$**  Space:  **$O(1)$**

For each number, we examine each of its digits:

- A number  $i$  has  $\log_{10}(i)$  digits  $\rightarrow$  in worst case:  $O(\log n)$  per number

```
int rotatedDigits(int n) {
    int count = 0;
    for (int i = 1; i <= n; ++i) {
        if (isGood(i)) count++;
    }
    return count;
}

bool isGood(int num) {
    bool changed = false;
    while (num > 0) {
        int digit = num % 10;
        if (digit == 3 || digit == 4 || digit == 7) return false;
        if (digit == 2 || digit == 5 || digit == 6 || digit == 9)
            changed = true;
        num /= 10;
    }
    return changed;
}
```

# Problem – 383. Ransom Note

Easy



LeetCode

[leetcode.com/problems/ransom-note](https://leetcode.com/problems/ransom-note)

## Problem

- You are given two strings: **magazine** and **ransomNote**
- Return true if **ransomNote** can be constructed by using letters from **magazine**
- A letter **cannot** be reused

- **Example:**

ransomNote = "aa", magazine = "ab"

**Output:** false (a letter from magazine cannot be used twice)

ransomNote = "aa", magazine = "aab"

**Output:** true

# Solution – 383. Ransom Note

Easy



LeetCode

[leetcode.com/problems/ransom-note](https://leetcode.com/problems/ransom-note)

## Solution

- Initialize an array with 26 characters (total letters in the English alphabet)
- Go over **magazine** string and count each character
- Go over **ransomNote** string and decrease each character
- If you get a negative number, return false

# Code – 383. Ransom Note

Easy



LeetCode

[leetcode.com/problems/ransom-note](https://leetcode.com/problems/ransom-note)

## Code

Time:  $O(n + m)$  Space:  $O(k)$  where  $n$  is the length of **magazine** and  $m$  the length of **ransomNote**, and  $k$  is the number of unique characters in magazine

```
bool canConstruct(string ransomNote, string magazine) {
    int count[26] = {0};
    for (const char& c : magazine) {
        count[c - 'a']++;
    }
    for (const char& c : ransomNote) {
        if (--count[c - 'a'] < 0) return false;
    }
    return true;
}
```

# Problem – 8. String to Integer (atoi)

Medium



LeetCode

[leetcode.com/problems/string-to-integer-atoi](https://leetcode.com/problems/string-to-integer-atoi)

## Problem

- You are given a string **s**
- Implement **myAtoi(string s)** using the following rules:
- Skip leading whitespace
- Determine sign (+ or -)
- Convert digits until non-digit or end of string
- Clamp result to 32-bit signed integer range  $[-2^{31}, 2^{31} - 1]$

# Solution – 8. String to Integer (atoi)

Medium



LeetCode

[leetcode.com/problems/string-to-integer-atoi](https://leetcode.com/problems/string-to-integer-atoi)

## Solution

- Initialize an index  $i$
- Position  $i$  to skip white spaces
- Check the sign and set a variable  $sign = -1$  or  $1$
- Go over the remaining of the string and use the following:  
 $digit = digit * 10 + s[i] - '0'$
- **Important:** use long long for the result and check overflows:  
 $if (sign == 1 \ \&\& \ result > INT\_MAX) \ return \ INT\_MAX;$   
 $if (sign == -1 \ \&\& \ -result < INT\_MIN) \ return \ INT\_MIN;$

# Problem – 8. String to Integer (atoi)

Medium



LeetCode

[leetcode.com/problems/string-to-integer-atoi](https://leetcode.com/problems/string-to-integer-atoi)

**Code** Time:  **$O(n)$**  Space:  **$O(1)$**

```
int myAtoi(string s) {
    int i = 0;
    int n = s.size();
    // skip leading whitespace
    while (i < n && s[i] == ' ') i++;

    // some sanity check
    if (i == n) return 0;

    // check the sign
    int sign = 1;
    if (s[i] == '-') {
        sign = -1;
        i++;
    } else if (s[i] == '+') {
        // keep sign = 1
        i++;
    }

    // convert
    long long result = 0;
    while (i < n && isdigit(s[i])) {
        result = result * 10 + s[i] - '0';
        if (sign == 1 && result > INT_MAX) return INT_MAX;
        if (sign == -1 && -result < INT_MIN) return INT_MIN;
        ++i;
    }
    return result * sign;
}
```

# Problem – 71. Simplify Path

Medium



LeetCode

[leetcode.com/problems/simplify-path](https://leetcode.com/problems/simplify-path)

## Problem

- You are given an **absolute path** for a Unix-style file system
  - Transform this **absolute path** into a simplified **canonical path**
  - The rules are:
    - “.” represents the **current directory**
    - “..” represents the **previous/parent directory**
    - “...”, “.....” or anything that doesn’t match “.” or “..” is a valid **directory / file**
- Canonical path** must **start** with a single slash ‘/’
- Directories** must be separated by one slash ‘/’
- The **path cannot end** with slash ‘/’



# Solution – 71. Simplify Path

Medium



LeetCode

[leetcode.com/problems/simplify-path](https://leetcode.com/problems/simplify-path)

## Solution

- Have a vector "**folders**" to keep track of all folders
- Split the **absolute path** string into multiple strings having '/' as divider
- Loop through each token:
  - If the folder is ".", just ignore (continue)
  - If the folder is not "." then push the name of the folder into the **vector of folders**
  - If the folder is "..", then pop back the last folder from the **vector of folders**
- After the loop, join the **folders** from the vector into a string

# Code – 71. Simplify Path

Medium



LeetCode

[leetcode.com/problems/simplify-path](https://leetcode.com/problems/simplify-path)

**Code** Time:  $O(n)$  Space:  $O(k)$  where  $n$  is the length of path and  $k$  is the number of valid folders in the simplified path

```
string simplifyPath(string path) {
    stringstream ss(path);
    string part;
    vector<string> folders;

    while(getline(ss, part, '/')) {
        if (part == "." || part.empty()) continue;
        if (part == "..") {
            if (!folders.empty())
                folders.pop_back();
        } else {
            folders.push_back(part);
        }
    }

    string result;
    for (const auto& s : folders) {
        result += "/" + s;
    }

    return result.empty() ? "/" : result;
}
```

# Problem – 14. Longest common Prefix

Easy



LeetCode

[leetcode.com/problems/longest-common-prefix](https://leetcode.com/problems/longest-common-prefix)

## Problem

- You are given an **array of strings**
- Find the **longest** common prefix amongst them
- Example:

```
strs = ["code", "colt", "cold"]
```

```
output = "co"
```

# Solution – 14. Longest common Prefix

Easy



LeetCode

[leetcode.com/problems/longest-common-prefix](https://leetcode.com/problems/longest-common-prefix)

## Solution

- You can solve in two nested loops
- ["code", "colt", "cold"]
- First loop through the first word "code"
- For each character, check if all other words match the same character in a second loop:  
`[c]ode == [c]olt → (true)`      `[c]ode == [c]old → (true)`  
`c[o]de == c[o]lt → (true)`      `c[o]de == c[o]ld → (true)`  
`co[d]de == co[l]t → (false)`
- When false, return the substring from 0 to the position that matches
- If all words are equal (no false conditions), return the first word

# Code – 14. Longest common Prefix

Easy



LeetCode

[leetcode.com/problems/longest-common-prefix](https://leetcode.com/problems/longest-common-prefix)

**Code** Time:  $O(n \times m)$  Space:  $O(1)$

```
string longestCommonPrefix(vector<string>& strs) {  
    if (strs.empty()) return "";  
  
    for (int i = 0; i < strs[0].size(); ++i) {  
        for (int j = 1; j < strs.size(); ++j) {  
            // Check length AND character in one condition  
            if (i >= strs[j].size() || strs[j][i] != strs[0][i]) {  
                return strs[0].substr(0, i);  
            }  
        }  
    }  
  
    return strs[0];  
}
```

**BINARY**

# Bit Manipulation in C

- **Operators**

**&** AND    **|** OR    **^** XOR    **~** NOT    **<<** LEFT SHIFT    **>>** RIGHT SHIFT

- **Common Operations**

**set bit:** `num |= (1 << pos)`

**clear bit:** `num &= ~(1 << pos)`

**toggle bit:** `num ^= (1 << pos)`

**check bit:** `(num & (1 << pos)) != 0`

**extract bit:** `(num >> pos) & 1`

**extract a range of bits:** `(num >> pos) & ((1 << length) - 1)`

- **Example**

```
void copyBit(int *dst, int src, int srcPos, int dstPos) {  
    int bit = (src >> srcPos) & 1; // extract bit  
    *dst &= ~(1 << dstPos); // clear destination bit  
    *dst |= (bit << dstPos); // set destination bit  
}
```

# Binary

- In C++, **std::bitset** represents a fixed-size sequence of N bits

- Example:

```
std::bitset<8> bitmap;
```

```
bitmap.reset(1)
```

```
bitmap.set(1)
```

```
if (bitmap.test(1)) { // true
```

```
...
```

- **reset** : set bit to false
- **set** : set a specific bit
- **test** : check a specific bit
- **count** : return the number of bits set to true
- **flip** : toggle the value of the bits (if true, set to false and vice-versa)



# Problem – 371. Sum of Two Integers

Medium



LeetCode

[leetcode.com/problems/sum-of-two-integers](https://leetcode.com/problems/sum-of-two-integers)

## Problem

- Sum two integer numbers **a** and **b**
- You can't use **+** or **-**

# Solution – 371. Sum of Two Integers

Medium



LeetCode

[leetcode.com/problems/sum-of-two-integers](https://leetcode.com/problems/sum-of-two-integers)

## Solution

- **Example:**

$a = 101$  and  $b = 110$

expected result =  $101 + 110 = 1011$

- **Find the position where carry occurs using and operation:**

$101 \& 110 = 100$

- **Sum without carry using XOR:**

$101 \wedge 110 = 011$

- **Shift left the carry:**

$100 \ll 1 = 1000$

- **Add the previous sum to the carry**

$011 \wedge 1000 = 1011$

- **Check the carry from the previous operation:**

$011 \wedge 1000 = 0000$

- **Now if it is zero, the result is 1011. Otherwise, repeat the process**

# Code – 371. Sum of Two Integers

Medium



LeetCode

[leetcode.com/problems/sum-of-two-integers](https://leetcode.com/problems/sum-of-two-integers)

**Code** Time: **O(1)** Space: **O(1)**

```
int getSum(int a, int b) {
    while (b != 0) {
        // find which bits produce a carry
        // e.g. 101 & 111 = 101
        int carry = static_cast<unsigned>(a & b);
        // adds without carry (XOR)
        // 101 ^ 111 = 010
        a = a ^ b;
        // move carry to the left
        // b = 110
        b = carry << 1;
    }
    return a;
}
```

# Problem – 191. Number of 1 Bits

Easy



LeetCode

[leetcode.com/problems/number-of-1-bits](https://leetcode.com/problems/number-of-1-bits)

## Problem

- You are given a positive integer  $n$
- Return the number of set bits
- **Example:**

$n = 11$

**output:** 3

Binary is 1011, hence three set bits "1"

# Solution – 191. Number of 1 Bits

Easy



LeetCode

[leetcode.com/problems/number-of-1-bits](https://leetcode.com/problems/number-of-1-bits)

## Solution

- Use Brian Kernighan's Bit Counting Algorithm
- removes the rightmost 1-bit from  $n$  in each iteration
- Example:

$n = 12$  (1100)

**First iteration:**  $n=1100$ ,  $n-1=1011$ , so  $1100 \& 1011 = 1000$  (removed rightmost 1)

**Second iteration:**  $n=1000$ ,  $n-1=0111$ , so  $1000 \& 0111 = 0000$  (removed last 1)

**Result:** 2 iterations = 2 ones counted

# Code – 191. Number of 1 Bits

Easy



LeetCode

[leetcode.com/problems/number-of-1-bits](https://leetcode.com/problems/number-of-1-bits)

## Code

```
// Brian Kernighan's algorithm
int hammingWeight(int n) {
    // return std::popcount(n);
    int count = 0;
    while (n) {
        n &= (n - 1);
        ++count;
    }
    return count;
}
```

# Problem – 338. Counting Bits

Easy



LeetCode

[leetcode.com/problems/counting-bits](https://leetcode.com/problems/counting-bits)

## Problem

- You are given an integer  $n$  representing  $n$  different numbers starting from 0
- Return an array with the number of 1 for each integer

- **Example:**

$n = 3$

$0 \rightarrow 0$  (zero '1's)

$1 \rightarrow 1$  (one '1')

$2 \rightarrow 10$  (one '1')

$3 \rightarrow 11$  (two '1's')

**Output:**  $[0, 1, 1, 2]$

# Solution – 338. Counting Bits

Easy



LeetCode

[leetcode.com/problems/counting-bits](https://leetcode.com/problems/counting-bits)

## Solution

- One solution is to code **Brian Kernighan's algorithm** and then call it **multiple times**; or
- **Use previously calculated results**: the bit count of any number  $i$  equals the bit count of  $i/2$  plus 1 if the last bit is 1
- `res[i] = res[i >> 1] + (i & 1)`
  - `i >> 1` removes the rightmost bit (divides by 2)
  - `i & 1` checks if the rightmost bit is 1



# Code – 338. Counting Bits

Easy



LeetCode

[leetcode.com/problems/counting-bits](https://leetcode.com/problems/counting-bits)

**Code** Time:  $O(n)$  Space:  $O(n)$

```
vector<int> countBits(int n) {
    vector<int> res(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
        // i >> 1 = i is divided by 2 (i with the last bit removed)
        // i & 1 is 1 if the last bit is set
        // number of set bits in i = number of set bits in i / 2 + 1 if last bit is 1.
        res[i] = res[i >> 1] + (i & 1);
    }
    return res;
}
```

# Problem – 190. Reverse Bits

Easy



LeetCode

[leetcode.com/problems/reverse-bits](https://leetcode.com/problems/reverse-bits)

## Problem

- You are given a **32 bits unsigned integer**
- **Reverse** its bits
- Example:  
00110 → 01100
- **Do not confuse this with “bit flipping”!**

# Solution – 190. Reverse Bits

Easy



LeetCode

[leetcode.com/problems/reverse-bits](https://leetcode.com/problems/reverse-bits)

## Solution

- Initialize a 32 bits integer **result** with 0
- As you know it is 32 bits, loop 32 times, and inside the loop:
- **Shift** all bits of **result** to the left (makes room for the new bit)
- Extract the right-most bit from the input
- Set the extracted bit as the new right-most bit of **result**
- Shift the input bits to the left

# Code – 190. Reverse Bits

Easy



LeetCode

[leetcode.com/problems/reverse-bits](https://leetcode.com/problems/reverse-bits)

**Code** Time:  $O(-)$  Space:  $O(-)$

```
uint32_t reverseBits(uint32_t n) {
    int result = 0;
    for (int i = 0; i < 32; ++i) {
        // shift to left
        result = result << 1;
        // extract the right-most bit from n
        int bit = n & 1;
        // set bit
        result = result | bit;
        // shift to right to check the next bit
        n = n >> 1;
    }
    return result;
}
```

# Negabinary

- Non-standard positional numeral system that uses base of -2
- Allow representing negative numbers in binary
- Example:

$1101_{-2}$

$$(-2)^3 + (-2)^2 + 0 + (-2)^0 = -8 + 4 + 0 + 1 = -3$$

## Summing Negabinary

- Add as a regular binary number, but with **negative carry**

$$0 + 0 = 0$$

$$1 + 0 = 1$$

$$1 + 1 = 0 \quad \text{with a negative carry 1}$$

$$\mathbf{1} + 1 = 0 \quad (\text{subtract})$$

$$\mathbf{1} + 0 = 1 \quad \text{with a positive carry 1}$$

# Negabinary

## Example 1

$$\begin{array}{r} 11\ 1 \\ 1011 \\ + 1110 \\ \hline = 110001 \end{array}$$

$$1 + 0 = 1$$

$$1 + 1 = 0 \text{ with negative carry } 1$$

$$1 + 1 = 0$$

$$1 + 1 = 0 \text{ with negative carry } 1$$

$$1 + 0 = 1 \text{ with positive carry } 1$$

$$1 + 0 = 1$$

red 1 = negative carry

green 1 = regular carry

## Example 2

$$\begin{array}{r} 1111 \\ 101010 \\ + 101100 \\ \hline = 11110110 \end{array}$$

## Reference

<https://math.stackexchange.com/questions/3251605/how-to-add-negabinary-numbers>

# GRAPH (BFS/DFS)

# Tips

- When to build an **adjacency list** from the input data?
  - When the graph is **sparse**
  - When working with **non-grid graphs**
  - When you want to perform multiple or complex traversals and need fast neighbour lookups.



# DFS Boilerplate

## Recursive DFS

Time Complexity:  $O(N + E)$     Space Complexity:  $O(N)$

```
void dfs(int node, const vector<vector<int>>& graph,
        vector<bool>& visited) {
    if (visited[node]) return;
    visited[node] = true;

    for (int neighbor : graph[node]) {
        dfs(neighbor, graph, visited);
    }
}

void runDFS(int n, const vector<vector<int>>& graph) {
    vector<bool> visited(n, false);
    dfs(0, graph, visited); // start from node 0
}
```

**N** = Number of nodes  
**E** = Number of edges

## Iterative DFS

Time Complexity:  $O(N + E)$     Space Complexity:  $O(N)$

```
void runDFS(int n, const vector<vector<int>>& graph) {
    vector<bool> visited(n, false);
    stack<int> stk;
    stk.push(0); // start from node 0

    while (!stk.empty()) {
        int node = stk.top();
        stk.pop();

        if (visited[node]) continue;
        visited[node] = true;

        for (int neighbor : graph[node]) {
            if (!visited[neighbor]) {
                stk.push(neighbor);
            }
        }
    }
}
```

# BFS Boilerplate

## Iterative BFS

Time Complexity:  $O(N + E)$     Space Complexity:  $O(N)$

```
void runBFS(int n, const vector<vector<int>>& graph) {  
    vector<bool> visited(n, false);  
    queue<int> q;  
    q.push(0); // start from node 0  
    visited[0] = true;  
  
    while (!q.empty()) {  
        int node = q.front();  
        q.pop();  
  
        for (int neighbor : graph[node]) {  
            if (!visited[neighbor]) {  
                visited[neighbor] = true;  
                q.push(neighbor);  
            }  
        }  
    }  
}
```

**N = Number of nodes**  
**E = Number of edges**



### Time Complexity notes

- Every node is enqueued/dequeued once
- Every edge is checked once

# Problem – 841. Keys and Rooms

Medium



LeetCode

[leetcode.com/problems/keys-and-rooms](https://leetcode.com/problems/keys-and-rooms)

## Problem

- You are given an **array of array rooms**, example:  
[[1,2], [2],[0]]
- **Each element** in the **outer array** represents a **room**
- **Each array** inside the array represents a **set of keys** that open the rooms
- **Rooms** are the **index of the array**
- In the example, **room 0** have the keys for **room 1 and 2**
- **Room 1** have the keys for **room 2**
- **Room 2** have the key for **room 0**
- **Room 0** is the only room unlocked. You start by visiting room 0, grab the key and unlock other rooms
- **Return true** if you can unlock all rooms

# Solution – 841. Keys and Rooms

Medium



LeetCode

[leetcode.com/problems/keys-and-rooms](https://leetcode.com/problems/keys-and-rooms)

## Solution

- This is a graph problem that can be solved using DFS
- Treat each room as a node
- Treat each set of keys as edges that goes from room A to B
- Visit room 0, and then start visiting the neighbours
- Once you visited all rooms/nodes, check if the visited size is the same as the number of existing rooms

# Code – 841. Keys and Rooms

Medium



LeetCode

[leetcode.com/problems/keys-and-rooms](https://leetcode.com/problems/keys-and-rooms)

**Code** Time:  $O(N + E)$  Space:  $O(N)$  where N is the number of rooms and E the number of keys. Space complexity is N due to the visited set and call stack

```
void dfs(int room, vector<vector<int>>& rooms, unordered_set<int>& visited) {
    if (visited.count(room)) return;
    visited.insert(room);
    for (const auto& roomNumber : rooms[room]) {
        dfs(roomNumber, rooms, visited);
    }
}

bool canVisitAllRooms(vector<vector<int>>& rooms) {
    unordered_set<int> visited;
    dfs(0, rooms, visited);
    return rooms.size() == visited.size();
}
```

# Problem – Clone Graph

Medium



LeetCode

<https://leetcode.com/problems/clone-graph>

## Problem Statement

- Given a node reference, create a deep copy of the graph
- The class node has two variables: val and neighbours

```
class Node {  
    public int val;  
    public List<Node> neighbors;  
}
```

- **Output** is the node reference of the copy



LeetCode

<https://leetcode.com/problems/clone-graph>

## Solution

- First check the edge cases (is the node null?)
- Create a hash map to store the nodes that is already created  
`unordered<int, Node*> graph;`
- Check if the current node already exists in the graph
- If not, create a new Node object and store in the hashmap
- Visit all the neighbors and add the neighbors to this current node

# Code – Clone Graph

Medium



LeetCode

<https://leetcode.com/problems/clone-graph>

```
std::unordered_map<int, Node*> graph;

Node* cloneGraph(Node* node) {
    if (node == NULL) {
        return NULL;
    }
    // does this node object exists?
    if (graph.find(node->val) == graph.end()) {
        // node wasn't visited yet, store in the hashmap
        graph[node->val] = new Node(node->val);
        // visit all neighbours
        for (const auto& n : node->neighbors) {
            graph[node->val]->neighbors.push_back(cloneGraph(n));
        }
    }
    return graph[node->val];
}
```



# Problem – 207. Course Schedule

Medium



LeetCode

[leetcode.com/problems/course-schedule](https://leetcode.com/problems/course-schedule)

## Problem

- You are given the number of courses and a course pre-requisite array
- Course pre-requisite indicates the dependency between courses

- **Example:**

`numCourses = 2, prerequisites = [[1,0],[0,1]]`

To take course 1, you must take course 0 first

to take course 0, you must take course 1 first

- In this example, this schedule is not possible since one course depends on the other
- Return if the schedule is valid or not

# Solution – 207. Course Schedule

Medium



LeetCode

[leetcode.com/problems/course-schedule](https://leetcode.com/problems/course-schedule)

## Solution

- Model as a graph problem
- Create a dependency graph between courses: **course A** depends on **course B**
- If there is a cycle, **A** to **B** and **B** to **A**, the schedule is invalid
- For the implementation: first convert the schedule to adjacency list
- Use DFS and track two status: **VISITING** and **VISITED**
- Go over each node in the adjacency list, and perform a DFS
- Once you find a node which status is **VISITING**, you've detected a cycle

# Code – 207. Course Schedule

Medium



LeetCode

[leetcode.com/problems/course-schedule](https://leetcode.com/problems/course-schedule)

**Code** Time:  $O(n + p)$  Space:  $O(n + p)$  where  $n$  is the number of courses and  $p$  the number of edges in the graph

```
enum class VisitState {
    NOT_VISITED,
    VISITING,
    VISITED
};

bool hasCycle(int node, const unordered_map<int, vector<int>>& adjList,
              unordered_map<int, VisitState>& visited) {
    // if we are revisiting a node in the current path, there's a cycle
    if (visited[node] == VisitState::VISITING) return true;

    // if we've already completed visiting this node, no need to check again
    if (visited[node] == VisitState::VISITED) return false;

    // mark the node as being visited
    visited[node] = VisitState::VISITING;

    for (int neighbor : adjList.at(node)) {
        if (hasCycle(neighbor, adjList, visited)) return true;
    }

    // mark the node as fully visited
    visited[node] = VisitState::VISITED;
    return false;
}
```

```
bool canFinish(int numCourses, const vector<vector<int>>& prerequisites) {
    // build the adjacency list: course -> list of its prerequisites
    unordered_map<int, vector<int>> adjList;
    for (const auto& dependencyPair : prerequisites) {
        int course = dependencyPair[0];
        int prerequisite = dependencyPair[1];
        adjList[course].push_back(prerequisite);
    }

    unordered_map<int, VisitState> visited;

    // check each course for cycles
    for (int course = 0; course < numCourses; ++course) {
        if (adjList.count(course)) {
            if (hasCycle(course, adjList, visited)) return false;
        }
    }

    return true; // no cycles detected
}
```

# Code – 207. Course Schedule

Medium



LeetCode

[leetcode.com/problems/course-schedule](https://leetcode.com/problems/course-schedule)

**Code (simplified)** Time:  $O(n + p)$  Space:  $O(n + p)$  where  $n$  is the number of courses and  $p$  the number of edges in the graph

```
bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {  
    // construct the graph  
    vector<vector<int>> adjList(numCourses);  
    // states:  
    // unvisited = 0, visiting = -1, visited = 1  
    vector<int> visited(numCourses, 0);  
    for (const auto& pre : prerequisites) {  
        adjList[pre[1]].push_back(pre[0]);  
    }  
    for (int n = 0; n < adjList.size(); ++n) {  
        if (hasCycle(adjList, visited, n /* starting node */)) {  
            return false;  
        }  
    }  
    return true;  
}  
  
bool hasCycle(vector<vector<int>>& adjList, vector<int>& visited, int node) {  
    if (visited[node] == -1) return true;  
    if (visited[node] == 1) return false;  
  
    // visiting  
    visited[node] = -1;  
    for (const auto& n: adjList[node]) {  
        if (hasCycle(adjList, visited, n)) {  
            return true;  
        }  
    }  
    // already visited  
    visited[node] = 1;  
}
```

# Problem – 417. Pacific Atlantic Water Flow

Medium



LeetCode

[leetcode.com/problems/pacific-atlantic-water-flow](https://leetcode.com/problems/pacific-atlantic-water-flow)

## Problem

- You are given a **matrix heights[m][n]** representing elevations on an island
- **Pacific Ocean** borders the **top and left**, **Atlantic Ocean** borders the **bottom and right**
- **Rainwater can flow** from a cell to its **north, south, east, or west** neighbor **if the neighbor's height is  $\leq$  current**
- Find all coordinates **(r, c)** from which **water can flow to both oceans**
- Return a list of such coordinates: `[[r1, c1], [r2, c2], ...]`

| Pacific Ocean  |   |   |   |   |   |
|----------------|---|---|---|---|---|
| Pacific Ocean  | 1 | 2 | 2 | 3 | 5 |
|                | 3 | 2 | 3 | 4 | 4 |
|                | 2 | 4 | 5 | 3 | 1 |
|                | 6 | 7 | 1 | 4 | 5 |
|                | 5 | 1 | 1 | 2 | 4 |
| Atlantic Ocean |   |   |   |   |   |

# Solution – 417. Pacific Atlantic Water Flow

Medium



LeetCode

[leetcode.com/problems/pacific-atlantic-water-flow](https://leetcode.com/problems/pacific-atlantic-water-flow)

## Solution 1 (inefficient)

- Iterate the whole matrix and perform a DFS
- Visit the neighbour if the value is less or equal the current value
- If it reaches some ocean, mark either atlantic or pacific as true
- Return true once both are true or false after finishing DFS

# Code – 417. Pacific Atlantic Water Flow

Medium



LeetCode

[leetcode.com/problems/pacific-atlantic-water-flow](https://leetcode.com/problems/pacific-atlantic-water-flow)

**Code** Time:  $O((m \times n)^2)$  Space:  $O(m \times n)$

```
private:
vector<vector<int>> directions = {{0,1},{0,-1},{1,0},{-1,0}};

public:
vector<vector<int>> pacificAtlantic(vector<vector<int>>& heights) {
    int m = heights.size();
    int n = heights[0].size();

    vector<vector<int>> result;

    for (int row = 0; row < m; ++row) {
        for (int col = 0; col < n; ++col) {
            bool pacific = false;
            bool atlantic = false;
            vector<vector<bool>> visited(m, vector<bool>(n, false));

            if (dfs(make_pair(row, col), heights, visited, pacific,
atlantic)) {
                result.push_back({row, col});
            }
        }
    }
    return result;
}
```

```
bool dfs(pair<int, int> coordinates, vector<vector<int>>& heights,
        vector<vector<bool>>& visited, bool& pacific, bool& atlantic) {
    if (pacific && atlantic) return true;
    int row = coordinates.first;
    int col = coordinates.second;
    if (visited[row][col]) return false;

    if (col == 0 || row == 0) pacific = true;
    if (col == heights[0].size() - 1 || row == heights.size() - 1)
        atlantic = true;

    visited[row][col] = true;

    for (const auto& d : directions) {
        int nextRow = row + d[0];
        int nextCol = col + d[1];
        if (nextRow < 0 || nextCol < 0 || nextRow >= heights.size() ||
            nextCol >= heights[0].size()) continue;
        if (heights[nextRow][nextCol] <= heights[row][col]) {
            if (dfs(make_pair(nextRow, nextCol), heights,
                visited, pacific, atlantic)) {
                return true;
            }
        }
    }
    return pacific && atlantic;
}
```

# Solution – 417. Pacific Atlantic Water Flow

Medium



LeetCode

[leetcode.com/problems/pacific-atlantic-water-flow](https://leetcode.com/problems/pacific-atlantic-water-flow)

## Solution 2 (better)

- **Start** from the **border of pacific ocean** and find all reachable squares using DFS
- Then, **start** from **the border of atlantic ocean** and find all reachable squares using DFS
- Check the squares where both are accessible



# Code – 417. Pacific Atlantic Water Flow

Medium



LeetCode

[leetcode.com/problems/pacific-atlantic-water-flow](https://leetcode.com/problems/pacific-atlantic-water-flow)

**Code** Time:  $O(m \times n)$  Space:  $O(m \times n)$

```
private:
vector<vector<int>> directions = {{0,1},{0,-1},{1,0},{-1,0}};
public:
vector<vector<int>> pacificAtlantic(vector<vector<int>>& heights) {
    int m = heights.size();
    int n = heights[0].size();

    // squares where pacific is reachable
    vector<vector<bool>> pacific(m, vector<bool>(n, false));
    // squares where atlantic is reachable
    vector<vector<bool>> atlantic(m, vector<bool>(n, false));

    // check pacific and atlantic
    for (int row = 0; row < m; ++row) {
        dfs(row, 0, heights, pacific);
        dfs(row, n - 1, heights, atlantic);
    }
    for (int col = 0; col < n; ++col) {
        dfs(0, col, heights, pacific);
        dfs(m - 1, col, heights, atlantic);
    }
    // check where both are reachable
    vector<vector<int>> result;
    for (int row = 0; row < m; ++row) {
        for (int col = 0; col < n; ++col) {
            if (pacific[row][col] && atlantic[row][col]) {
                result.push_back({row,col});
            }
        }
    }
    return result;
}
```

```
void dfs(int row, int col, vector<vector<int>>& heights,
vector<vector<bool>>& visited) {
    // check bounds
    if (visited[row][col]) return;

    visited[row][col] = true;

    for (const auto& d : directions) {
        int nextRow = d[0] + row;
        int nextCol = d[1] + col;
        if (nextRow < 0 || nextCol < 0 ||
            nextRow >= heights.size() || nextCol >= heights[0].size())
            continue;
        if (heights[nextRow][nextCol] >= heights[row][col]) {
            dfs(nextRow, nextCol, heights, visited);
        }
    }
}
```

# Problem – 200. Number of Islands

Medium



LeetCode

[leetcode.com/problems/number-of-islands](https://leetcode.com/problems/number-of-islands)

## Problem

- You are given a **matrix m x n**
- The **matrix** represents a map where "1" is land and "0" is water
- **Return** the total number of island: "connected" ones form one island
- Example:

### Input:

```
grid =  {{ "1", "1", "1", "1", "0"},
          { "1", "1", "0", "1", "0"},
          { "1", "1", "0", "0", "1"},
          { "0", "0", "0", "1", "1" }}
```

**Output:** 2

# Solution – 200. Number of Islands

Medium



LeetCode

[leetcode.com/problems/number-of-islands](https://leetcode.com/problems/number-of-islands)

## Solution

- **Loop** through each element in the matrix
- Once you find "1", count the land and;
- Perform a **DFS (or BFS)** around each '1' cell
- **When traversing**, mark "0" after visited
- **Continue** until you finish processing all elements in the matrix

# Code – 200. Number of Islands

Medium



LeetCode

[leetcode.com/problems/number-of-islands](https://leetcode.com/problems/number-of-islands)

**Code** Time:  $O(m \times n)$  Space:  $O(m \times n)$  As we visit each cell only once, time complexity is the size of the matrix. The space complexity comes from the DFS recursion stack.

```
int numIslands(vector<vector<char>>& grid) {
    int m = grid.size(), n = grid[0].size();
    int count = 0;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == '1') {
                count++;
                dfs(grid, i, j);
            }
        }
    }
    return count;
}

void dfs(vector<vector<char>>& grid, int i, int j) {
    int m = grid.size(), n = grid[0].size();

    if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] == '0')
        return;

    grid[i][j] = '0';

    dfs(grid, i + 1, j);
    dfs(grid, i - 1, j);
    dfs(grid, i, j + 1);
    dfs(grid, i, j - 1);
}
```

# Problem – 128. Longest Consecutive Sequence

Medium



LeetCode

[leetcode.com/problems/longest-consecutive-sequence](https://leetcode.com/problems/longest-consecutive-sequence)

## Problem

- You are given an unsorted **array of integers** `nums`
- **Return the length** of the **longest consecutive** elements sequence
- Algorithm **must run** on  **$O(n)$**  time

# Solution – 128. Longest Consecutive Sequence

Medium



LeetCode

[leetcode.com/problems/longest-consecutive-sequence](https://leetcode.com/problems/longest-consecutive-sequence)

## Solution

- **Create** a set with the same elements of **nums**
- Loop through this set and check if it is the beginning of a sequence

- **Example:**

[4, 100, 3, 2, 101]

**4**: Look up 3 (found) → 4 is **NOT** the beginning (skip it)

**100**: Look up 99 (not found) → 100 **IS** the beginning → count 100, 101 → length = 2

**3**: Look up 2 (found) → 3 is **NOT** the beginning (skip it)

**2**: Look up 1 (not found) → 2 **IS** the beginning → count 2, 3, 4 → length = 3

**101**: Look up 100 (found) → 101 is **NOT** the beginning (skip it)

- Track the **maximum length**

# Code – 128. Longest Consecutive Sequence

Medium



LeetCode

[leetcode.com/problems/longest-consecutive-sequence](https://leetcode.com/problems/longest-consecutive-sequence)

**Code** Time:  $O(n)$  Space:  $O(n)$

```
int longestConsecutive(vector<int>& nums) {
    unordered_set<int> seq(nums.begin(), nums.end());
    int maxStreak = 0;
    for (const auto& n : seq) {
        // is beginning of sequence?
        // previous exist?
        int streak = 0;
        if (seq.find(n - 1) == seq.end()) {
            // check next
            int currNum = n;
            while (seq.find(currNum) != seq.end()) {
                streak++;
                currNum++;
            }
        }
        maxStreak = max(maxStreak, streak);
    }

    return maxStreak;
}
```

# Problem – 323. Number of Connected Components

Medium

 [leetcode.com/problems/number-of-connected-components-in-an-undirected-graph](https://leetcode.com/problems/number-of-connected-components-in-an-undirected-graph)

## Problem

- You are given a graph of **n** nodes, and an array of **edges (source, destination)**
- Edges indicates the edge between node **source** and **destination**
- Find the total number of isolated components (subgraphs)

- **Example:**

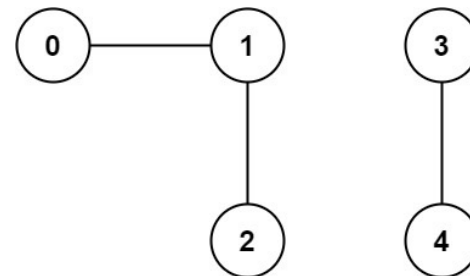
Input:

`n = 5, edges = [[0,1],[1,2],[3,4]]`

Output: 2

0 is connected to 1, 1 is connected to 2

3 is a new subgraph connected to 4





# Solution – 323. Number of Connected Components

Medium

 [leetcode.com/problems/number-of-connected-components-in-an-undirected-graph](https://leetcode.com/problems/number-of-connected-components-in-an-undirected-graph)

## Solution

- Build an adjacency list from edges
- From each node, check if its visited
- If it is not visited, mark as a new “component” or subgraph
- Perform a DFS from that node

# Code – 323. Number of Connected Components

Medium

 [leetcode.com/problems/number-of-connected-components-in-an-undirected-graph](https://leetcode.com/problems/number-of-connected-components-in-an-undirected-graph)

**Code** Time:  $O(n + E)$  Space:  $O(n + E)$  where  $n$  is the number of nodes and  $E$  is edges size

```
int countComponents(int n, vector<vector<int>>& edges) {
    vector<bool> visited(n, false);
    vector<vector<int>> adjList(n);

    // build adjacency list
    for (const auto& edge: edges) {
        adjList[edge[0]].push_back(edge[1]);
        adjList[edge[1]].push_back(edge[0]);
    }
    int totalComponents = 0;
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            dfs(adjList, visited, i);
            totalComponents++;
        }
    }

    return totalComponents;
}

void dfs(vector<vector<int>>& adjList, vector<bool>& visited, int node) {
    if (visited[node]) return;
    visited[node] = true;
    for (const auto& neighbour : adjList[node]) {
        dfs(adjList, visited, neighbour);
    }
}
```

# Problem – Maximum Level Sum of a Binary Tree

Medium

 <https://leetcode.com/problems/maximum-level-sum-of-a-binary-tree>

## Problem Statement

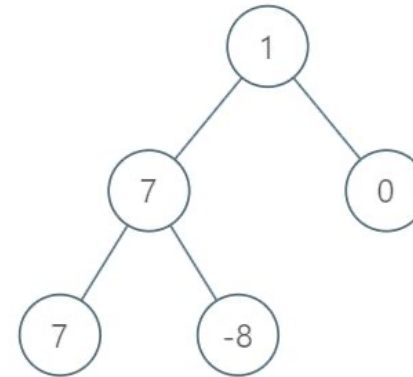
- Given the root of a binary tree, find the smallest level with the maximum sum
- For example, the tree below has the follow sums for each level:

level 1 (root) = 1

**level 2 = 7 + 0 = 7**

level 3 = 7 - 8 = -1

- Therefore, **level 2** has the maximum sum



# Solution – Maximum Level Sum of a Binary Tree

Medium

 LeetCode <https://leetcode.com/problems/maximum-level-sum-of-a-binary-tree>

## Solution

- Have a queue with the nodes for the current level
- Sum the values from that level by taking the nodes from the queue
- Example, we know that level 1 has one node. Hence, pop the first node from the queue  
If level 2 has 2 nodes, pop two nodes, sum the values
- In addition, add left and right to the end of the queue to process the next level

# Code – Maximum Level Sum of a Binary Tree

Medium

 <https://leetcode.com/problems/maximum-level-sum-of-a-binary-tree>

```
int maxLevelSum(TreeNode* root) {
    std::queue<TreeNode*> nodes;
    int currentLevel = 0;
    int maxLevel = 1;
    int maxSum = INT_MIN;

    nodes.push(root);

    // traverse the graph
    while(!nodes.empty()) {
        int levelSum = 0;
        int levelSize = nodes.size();
        currentLevel++;

        // sum the values in current level
        for (int i = 0; i < levelSize; ++i) {
            TreeNode* node = nodes.front();
            levelSum += node->val;
            nodes.pop();

            if (node->left) nodes.push(node->left);
            if (node->right) nodes.push(node->right);
        }

        if (levelSum > maxSum) {
            maxLevel = currentLevel;
            maxSum = levelSum;
        }
    }

    return maxLevel;
}
```

# Problem – 1236. Web Crawler

Medium



LeetCode

<https://leetcode.com/problems/web-crawler>

## Problem

- You are given a starting URL `startURL` and an interface `HtmlParser` with a method `getUrls(url)`
- `getUrls(url)` returns a vector of strings with the URLs found on the given page
- Start crawling from `startUrl` and recursively visit all reachable URLs
- Only visit URLs that share the same hostname as `startUrl`
- Return a list of all visited URLs (in any order)



LeetCode

<https://leetcode.com/problems/web-crawler>

## Solution

- This is a graph problem framed as an object
- Both BFS and DFS are valid options
- Each url represent a node, and `getUr1s` retrieve the neighbours
- Visit each node and add to the result if they have the same hostname



LeetCode

<https://leetcode.com/problems/web-crawler>

**Code (BFS)** Time:  $O(n + m)$  Space:  $O(n + w)$  where  $n$  is the number of unique URLs,  $m$  the number of links (edges),  $w$  is the explicit queue

```
vector<string> crawl(string startUrl, HtmlParser htmlParser) {
    string hostname = getHostname(startUrl);
    queue<string> urls;
    unordered_set<string> visited;
    vector<string> result;

    urls.push(startUrl);
    visited.insert(startUrl);
    result.push_back(startUrl);

    while(!urls.empty()) {
        string url = urls.front();
        urls.pop();
        for (const auto& u : htmlParser.getUrls(url)) {
            // is it the same hostname?
            // have I already visited this one?
            if (visited.count(u)) continue;
            if (getHostname(u) != hostname) continue;
            urls.push(u);
            result.push_back(u);
            visited.insert(u);
        }
    }
    return result;
}
```

```
string getHostname(const string& url) {
    int start = url.find("://") + 3;
    int end = url.find("/", start);
    return url.substr(start, end - start);
}
```



# Problem – 1236. Web Crawler

Medium



LeetCode

<https://leetcode.com/problems/web-crawler>

**Code (DFS)** Time:  $O(n + m)$  Space:  $O(n + h)$  where  $n$  is the number of unique URLs,  $m$  the number of links (edges),  $h$  is the recursive stack

```
string getHostname(const string& url) {
    int start = url.find("://") + 3;
    int end = url.find("/", start);
    return url.substr(start, end - start);
}

void dfs(const string& hostname, const string& url, HtmlParser& htmlParser,
unordered_set<string>& visited, vector<string>& result) {
    if (visited.count(url)) return;
    result.push_back(url);
    visited.insert(url);

    for (const auto& u: htmlParser.getUrls(url)) {
        if (getHostname(u) == hostname) {
            dfs(hostname, u, htmlParser, visited, result);
        }
    }
}

vector<string> crawl(string startUrl, HtmlParser htmlParser) {
    string hostname = getHostname(startUrl);
    unordered_set<string> visited;
    vector<string> result;
    dfs(hostname, startUrl, htmlParser, visited, result);
    return result;
}
```

# Problem – 994. Rotting Oranges

Medium

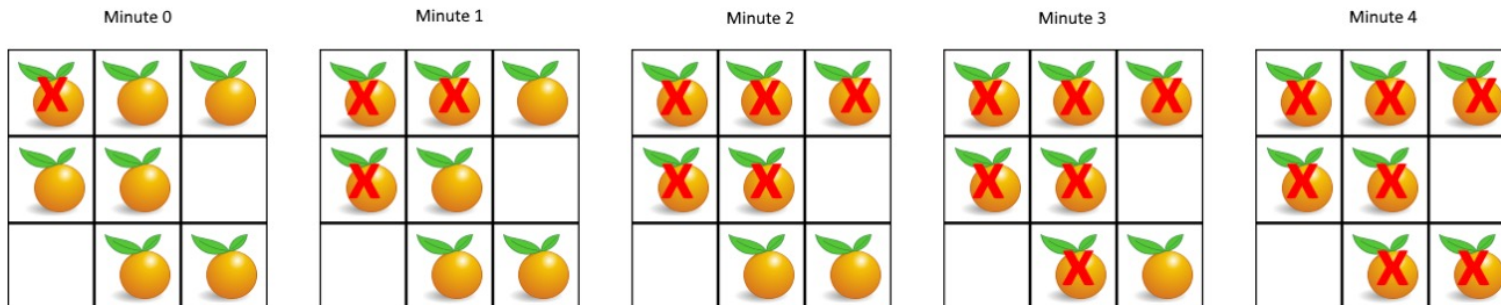


LeetCode

[leetcode.com/problems/rotting-oranges](https://leetcode.com/problems/rotting-oranges)

## Problem

- You are given a **m x n** grid
- Each cell represents the following:
  - **0** is an empty cell
  - **1** is a fresh orange
  - **2** is a rotten orange
- Every minute (snapshot), any fresh orange is contaminated by adjacent oranges
- Return the **minimum number of minutes** required for all fresh oranges to become rotten
- If it is **impossible** to rot all fresh oranges, return -1



# Solution – 994. Rotting Oranges

Medium



LeetCode

[leetcode.com/problems/rotting-oranges](https://leetcode.com/problems/rotting-oranges)

## Solution

- This is another BFS problem: for each rotten orange, visit adjacent fresh oranges
- Start by finding all rotten oranges in the grid. No need to convert grid to adjacent list
- Initialize a `queue<pair<int, int>>` to perform the BFS. Add the rotten oranges to this queue
- Start the traversal  
`while (!q.empty()) { ... }`
- **This part is important!** you want to calculate the “minutes”. So you have to first go over the current size of the queue and “process” all the elements, meaning, rot the adjacent fresh oranges
- Use directions vector to calculate the adjacent positions:  
`const vector<pair<int, int>> directions = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};`
- Keep track of the number of fresh oranges
- By the end, check if the number of fresh oranges is zero. If so, return minutes, or -1 otherwise.

# Code – 994. Rotting Oranges

Medium



LeetCode

leetcode.com/problems/rotting-oranges

**Code** Time:  $O(m * n)$  Space:  $O(m * n)$

```
int orangesRotting(vector<vector<int>>& grid) {
    // go over the grid, find the rotten ones
    // count the number of fresh oranges
    // add to a queue
    // queue should contain the positions x,y
    int fresh = 0;
    // we'll increase the minutes before visiting
    int minutesElapsed = -1;
    queue<pair<int, int>> q;

    int m = grid.size();
    int n = grid[0].size();

    for (int row = 0; row < m; ++row) {
        for (int col = 0; col < n; ++col) {
            if (grid[row][col] == 1) ++fresh;
            if (grid[row][col] == 2) q.push({row, col});
        }
    }

    // no fresh oranges
    if (fresh == 0) return 0;
```

```
    // at each minute: pop all the queue, visit the neighbours
    // set a fresh one to rotten
    // decrease the number of fresh
    vector<pair<int, int>> directions = {{0,1},{0,-1},{1,0},{-1,0}};
    while(!q.empty()) {
        int qSize = q.size();
        // at each minute, it rots all oranges
        // therefore, fully consumes the queue
        minutesElapsed++;

        for (int i = 0; i < qSize; ++i) {
            auto [row, col] = q.front();
            q.pop();

            // visit neighbours, check boundaries
            // and if its not visited yet
            for (const auto& [dRow, dCol] : directions) {
                int nRow = row + dRow;
                int nCol = col + dCol;
                if (nRow >= 0 && nCol >= 0 && nRow < m && nCol < n && grid[nRow][nCol] == 1) {
                    grid[nRow][nCol] = 2;
                    fresh--;
                    q.push({nRow, nCol});
                }
            }
        }
    }

    // once you reach the end, count if rotten == fresh
    return (fresh == 0) ? minutesElapsed : -1;
}
```

# Backtracking

## Common pattern in backtracking

- Useful for problems like: generating all permutations / combinations
- N-Queens
- Sudoku
- Letter combinations

```
void backtrack(/* problem-specific args */) {  
    if (/* base case */) {  
        // store result  
        return;  
    }  
  
    for (/* each choice */) {  
        // make choice  
        state.push_back(choice);  
  
        // explore further  
        backtrack(/* updated args */);  
  
        // undo choice (backtrack)  
        state.pop_back();  
    }  
}
```

# Problem – 46. Permutations

Medium



LeetCode

[leetcode.com/problems/permutations](https://leetcode.com/problems/permutations)

## Problem

- You are given an array of numbers, **Example:**

`nums = [1,2]`

- Return all possible the permutations:

`output = [[1,2], [2,1]]`

# Solution – 46. Permutations

Medium



LeetCode

[leetcode.com/problems/permutations](https://leetcode.com/problems/permutations)

## Solution

- Permutations problem can be solved using backtracking
- Create a function following the template:

```
backtrack(path, result, nums) {  
    if path.size == nums.size()  
        add to the result and return  
    for i in nums  
        continue if already used this i  
        mark i as used  
        add to the current path  
        backtrack(...)  
        mark i as unused  
        remove from the current path
```

# Code – 46. Permutations

Medium



LeetCode

[leetcode.com/problems/permutations](https://leetcode.com/problems/permutations)

**Code** Time:  $O(n! * n)$  Space:  $O(n)$  **Time:** there are  $n!$  permutations. Each permutation takes  $O(n)$  time to construct. **Space:** the path and used vector grow as  $n$  grows.  $n$  represents the size of `nums`

```
void backtrack(vector<int>& path, vector<int>& nums, vector<bool>& used, vector<vector<int>>& result) {
    if (path.size() == nums.size()) {
        result.push_back(path);
        return;
    }
    for (int i = 0; i < nums.size(); ++i) {
        if (used[i]) continue;
        used[i] = true;
        path.push_back(nums[i]);
        backtrack(path, nums, used, result);
        path.pop_back();
        used[i] = false;
    }
}

vector<vector<int>> permute(vector<int>& nums) {
    vector<int> path;
    vector<vector<int>> result;
    vector<bool> used(nums.size(), false);
    backtrack(path, nums, used, result);
    return result;
}
```



# Problem – 17. Letter Combinations of a Phone Number

Medium

 [leetcode.com/problems/letter-combinations-of-a-phone-number](https://leetcode.com/problems/letter-combinations-of-a-phone-number)

## Problem

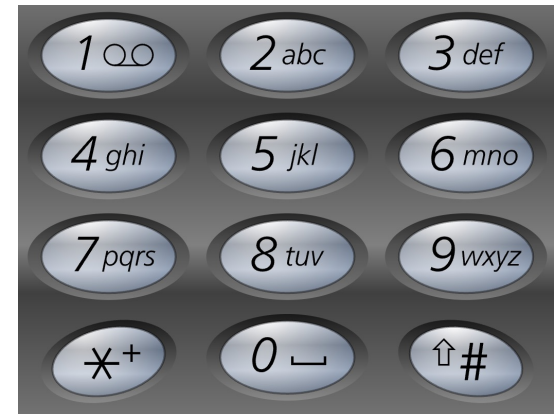
- You are given a string **digits** containing numbers such as "2" or "234" etc
- Each digit correspond to a digit of a phone number
- The digits map to a group of characters from the phone. For example, 2 → "abc", 3 → "def" ...
- Return all possible letter combinations from the digits

- **Example**

**Input:** 23

**Output:** ["ad","ae","af","bd","be","bf","cd","ce","cf"]

2 maps to "abc" and 3 maps to "def", so generate all combinations



# Solution – 17. Letter Combinations of a Phone Number

Medium

 [leetcode.com/problems/letter-combinations-of-a-phone-number](https://leetcode.com/problems/letter-combinations-of-a-phone-number)

## Solution

- Map the keyboard to a vector of strings:

```
std::vector<string> = { "", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz" }
```

First 2 characters are empty to map exactly the phone digit position

- Use backtracking to generate all combinations
- Example:** digits "2" and "3" maps to "abc" and "def":

visit "a"

visit "d"

reached the end of the digits, add "ad"

backtrack to "a"

visit "e"

reached the end of the digits, add "ae"

...

# Problem – 17. Letter Combinations of a Phone Number

Medium

 [leetcode.com/problems/letter-combinations-of-a-phone-number](https://leetcode.com/problems/letter-combinations-of-a-phone-number)

**Code** Time:  $O(4^n)$  Space:  $O(n * 4^n)$  where  $n$  is the number of digits.

For each digit, you have to generate a combination of max 4 characters (the maximum phone digits, for example, 7 represents "pqrs")

```
void backtrack(vector<string>& result, string& current,
               const vector<string>& phone, string& digits, int index) {
    if (index == digits.size()) {
        result.push_back(current);
        return;
    }
    // retrieve current digit
    char currentDigit = digits[index];
    // retrieve chars from that digit
    string chars = phone[currentDigit - '0'];

    // go over each char to backtrack
    for (const char& c : chars) {
        current.push_back(c);
        backtrack(result, current, phone, digits, index + 1);
        current.pop_back();
    }
}

vector<string> letterCombinations(string digits) {
    if (digits.empty()) return {};

    const vector<string> phone = {
        "", "", "abc", "def", "ghi",
        "jkl", "mno", "pqrs", "tuv", "wxyz"
    };
    vector<string> result;
    string current;
    backtrack(result, current, phone, digits, 0);
    return result;
}
```

# **SHORTEST PATH**

# Shortest Path Algorithms

| Algorithm            | Use Case                                                                      | Graph Type                          | Time Complexity               | Notes                                                                                    |
|----------------------|-------------------------------------------------------------------------------|-------------------------------------|-------------------------------|------------------------------------------------------------------------------------------|
| BFS                  | When all edges have <b>equal weight</b>                                       | Unweighted / same-weighted edges    | $O(V + E)$                    | Simplest and fastest when all weights are equal.                                         |
| Dijkstra             | When edge weights are <b>non-negative</b>                                     | Weighted, no negative weights       | $O((V + E) \log V)$ with heap | Greedy, efficient for SSSP (Single Source Shortest Path).                                |
| Bellman-Ford         | When edge weights can be <b>negative</b>                                      | Weighted, allows negative weights   | $O(V * E)$                    | Slower, but handles negative weights and detects negative cycles.                        |
| Floyd-Warshall       | For <b>all-pairs shortest paths</b>                                           | Dense graphs, small number of nodes | $O(V^3)$                      | Easy to implement; handles negative weights (but not negative cycles).                   |
| A* Search            | For shortest path with a <b>goal node</b> and <b>heuristic</b>                | Weighted, heuristic needed          | Depends on heuristic quality  | Often used in pathfinding (e.g. maps, games); faster than Dijkstra if heuristic is good. |
| Johnson's Algorithm  | <b>All-pairs shortest paths</b> in sparse graphs with <b>negative weights</b> | Weighted, allows negative weights   | $O(V^2 \log V + V * E)$       | Reweights graph with Bellman-Ford, then runs Dijkstra from each node.                    |
| SPFA                 | Practical variant of Bellman-Ford, often faster                               | Weighted, allows negative weights   | Avg: $O(E)$ , Worst: $O(VE)$  | Queue-based; faster in practice, not guaranteed. Handles negative weights.               |
| Bidirectional Search | When you know <b>start and target</b> , speeds up search in undirected graphs | Unweighted or uniformly weighted    | $O(b^{(d/2)})$ in best case   | Runs two simultaneous searches (from start and end); fast when goal is known.            |

# Problem – 743. Network Delay Time

Medium



LeetCode

[leetcode.com/problems/network-delay-time](https://leetcode.com/problems/network-delay-time)

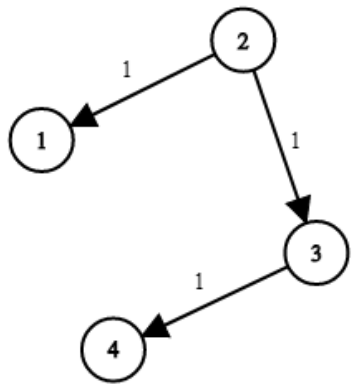
## Problem

- You are given a network of nodes  $n$  with destination and time to reach that node
- You are given a starting node  $k$  and the number of nodes in the network  $n$
- A signal is sent from node  $k$  to all nodes in the network
- Find the minimum time required for all nodes to receive the signal from  $k$

- **Example:**

**$k = 2$     $n = 4$**

**Output: 3**



# Solution – 743. Network Delay Time

Medium



LeetCode

[leetcode.com/problems/network-delay-time](https://leetcode.com/problems/network-delay-time)

## Solution

- This is solved using Dijkstra algorithm
- Build the graph by storing in the destination node and the time from a source node:  
`graph[node] = [[node, distance]]`  
`graph[2] = [[1,1], [2,3]]`
- Set up a min-heap for Dijkstra (priority\_queue) with distance and node
- Perform Dijkstra algorithm and store the shortest paths
- Check if all nodes were reached
- Return the longest distance among shortest paths

# Code – 743. Network Delay Time

Medium



LeetCode

[leetcode.com/problems/network-delay-time](https://leetcode.com/problems/network-delay-time)

**Code** Time:  $O((n + e) \log n)$  Space:  $O(n + e)$  where  $n$  is the number of nodes and  $e$  the number of edges

```
int networkDelayTime(vector<vector<int>>& times, int n, int k) {
    // node => (destination, distance)
    unordered_map<int, vector<pair<int, int>>> graph;
    for (const auto& time : times) {
        // time[0] = source node, time[1] = dest node, time[2] = time
        graph[time[0]].emplace_back(time[1], time[2]);
    }

    // min heap: distance from the origin 'k' to 'node'
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> minHeap;
    minHeap.emplace(0, k); // distance, starting node

    // shortest path from each node to the origin 'k' (node, distance)
    unordered_map<int, int> dist;

    // we start exploring the nodes from the minimum
    // distance to the origin 'k'
    while (!minHeap.empty()) {
        auto [distance, node] = minHeap.top();
        minHeap.pop();
        // already visited, skip
        if (dist.count(node)) continue;
        // set the distance
        dist[node] = distance;
        // look at the connections
        for (const auto& [n, d] : graph[node]) {
            // n[node, distance]
            // quick optimization, not necessary
            if (dist.count(n)) continue;
            // add the distance since we want
            // the distance from the origin
            minHeap.emplace(distance + d, n);
        }
    }

    // check if all nodes were visited
    if (dist.size() != n) return -1;

    // all the minimum distances are calculated
    // find the max one since we want to reach
    // all nodes
    int minTime = 0;
    for (const auto& [n, d] : dist) {
        minTime = max(minTime, d);
    }

    return minTime;
}
```



# Problem – 1306. Jump Game III

Medium



LeetCode

[leetcode.com/problems/jump-game-iii](https://leetcode.com/problems/jump-game-iii)

## Problem

- Variation of Jump Game
- You are given an array of integers and a start index (integer)
- You are initially positioned at start index of the array
- You can jump either to  **$\text{pos} + \text{array}[\text{pos}]$**  or  **$\text{pos} - \text{array}[\text{pos}]$**
- Check if you can reach any index with value 0

# Solution – 1306. Jump Game III

Medium



LeetCode

[leetcode.com/problems/jump-game-iii](https://leetcode.com/problems/jump-game-iii)

## Solution

- Once you are in any position, you have two choices:  
either go  **$i + \text{array}[i]$**  or  **$i - \text{array}[i]$**
- You should explore both directions recursively
- Once you **find 0**, return **true**
- Keep track of visited positions, **return false** once visited

# Solution – 1306. Jump Game III

Medium



LeetCode

[leetcode.com/problems/jump-game-iii](https://leetcode.com/problems/jump-game-iii)

## Solution: I like to use the following thought process:

- We know we have to explore both situations:  $\text{pos} + \text{arr}[\text{pos}]$  and  $\text{pos} - \text{arr}[\text{pos}]$ :

```
bool dfs(vector<int>& arr, int pos) {  
    return dfs(arr, pos + arr[pos]) || dfs(arr, pos - arr[pos]);  
}
```

- Add the most obvious base cases: **found zero**

```
bool dfs(vector<int>& arr, int pos) {  
    if (arr[pos] == 0) return true;  
    ...  
}
```

- And then add the other obvious base case: **out of bounds, return false**

```
bool dfs(vector<int>& arr, int pos) {  
    if (pos >= arr.size()) return false;  
    if (arr[pos] == 0) return true;  
    ...  
}
```

- Then check visited:

```
bool dfs(vector<int>& arr, int pos, vector<bool>& visited) {  
    if (pos >= arr.size()) return false;  
    if (visited[pos]) return false;  
    if (arr[pos] == 0) return true;  
    visited[pos] = true;  
    return dfs(arr, pos + arr[pos]) || dfs(arr, pos - arr[pos]);  
}
```

# Code – 1306. Jump Game III

Medium



LeetCode

[leetcode.com/problems/jump-game-iii](https://leetcode.com/problems/jump-game-iii)

**Code** Time:  **$O(n)$**  Space:  **$O(1)$**

```
bool dfs(vector<int>& arr, int pos, vector<bool>& visited) {
    if (pos >= arr.size()) return false;
    if (visited[pos]) return false;
    if (arr[pos] == 0) return true;
    visited[pos] = true;
    return dfs(arr, pos + arr[pos], visited) || dfs(arr, pos - arr[pos], visited);
}

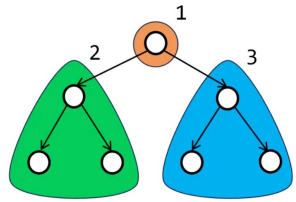
bool canReach(vector<int>& arr, int start) {
    vector<bool> visited(arr.size(), false);
    return dfs(arr, start, visited);
}
```

**TREE**

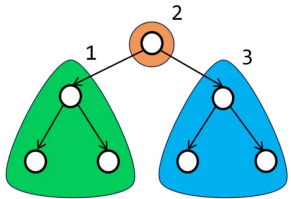
# Tree Traversals

## Depth-First Traversals

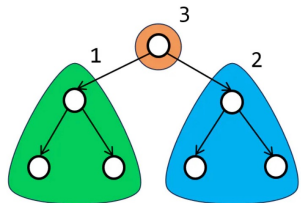
- **Pre-order:** Root - Left - Right



- **In-order:** Left - Root - Right



- **Post-order:** Left - Right - Root



## Breadth-First Traversal (Level Order Traversal)

Visit every node on a level before moving to a lower level.

# Tree Traversals

## Depth-First Traversals

Use a recursive algorithm to traverse according to the order

- **Pre-order:** Root - Left - Right



```
if (!root) return;  
doSomething();  
visit(node->left);  
visit(node->right);
```

- **In-order:** Left - Root - Right



```
if (!root) return;  
visit(node->left);  
doSomething();  
visit(node->right);
```

- **Post-order:** Left - Right - Root



```
if (!root) return;  
visit(node->left);  
visit(node->right);  
doSomething();
```

# Tree Traversals

## Example of pre-order and in-order

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

// Pre-order traversal
void preorderTraversal(TreeNode* root) {
    if (root == nullptr) return;
    cout << root->val << " ";
    preorderTraversal(root->left);
    preorderTraversal(root->right);
}

// In-order traversal
void inorderTraversal(TreeNode* root) {
    if (root == nullptr) return;
    inorderTraversal(root->left);
    cout << root->val << " ";
    inorderTraversal(root->right);
}
```



# Tree Traversals

## Example of post-order and level-order

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

// Post-order traversal
void postorderTraversal(TreeNode* root) {
    if (root == nullptr) return;
    postorderTraversal(root->left);
    postorderTraversal(root->right);
    cout << root->val << " ";
}

// Level-order traversal using a queue
void levelOrderTraversal(TreeNode* root) {
    if (root == nullptr) return;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        TreeNode* current = q.front();
        q.pop();
        cout << current->val << " ";
        if (current->left != nullptr) q.push(current->left);
        if (current->right != nullptr) q.push(current->right);
    }
}
```

# BFS Using Stack

## BFS with std::stack

- This might be useful for problems when you want to return and resume (for example, [872. Leaf-Similar Trees](#))

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

// Pre-order traversal
void bfs(std::stack<TreeNode*>& tree) {
    while(!tree.empty()) {
        TreeNode* root = tree.top();
        tree.pop();
        // do something ...
        if (root->right) tree.push(root->right);
        if (root->left) tree.push(root->left);
    }
}
```

# Problem – Maximum Depth of Binary Tree

Easy

 <https://leetcode.com/problems/maximum-depth-of-binary-tree>

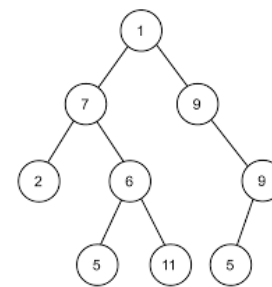
## Problem Statement

- Given the root of a binary tree, find the maximum depth

- Example:**

root = [1,7,9,2,6,null,9,null,null,5,11,5,null]

- Output:** 4



# Solution – Maximum Depth of Binary Tree

Easy

 LeetCode <https://leetcode.com/problems/maximum-depth-of-binary-tree>

## Solution

- Perform **post-order** traversal: left - right - root
- Recursively go left and right to find each value
- Return the max of each one

# Code – Maximum Depth of Binary Tree

Easy

 <https://leetcode.com/problems/maximum-depth-of-binary-tree>

```
int maxDepth(TreeNode* root) {  
    if (!root) return 0;  
    // find max left  
    int maxLeft = maxDepth(root->left);  
    // find max right  
    int maxRight = maxDepth(root->right);  
    // return max +1 (account for root)  
    return max(maxLeft, maxRight) + 1;  
}
```

# Problem – 100. Same Tree

Easy



LeetCode

[leetcode.com/problems/same-tree](https://leetcode.com/problems/same-tree)

## Problem

- You are given the root of two trees
- Write a function to check if they are the same

- **Example:**

$p = [1,2,3]$ ,  $q = [1,2,3]$

Output: true

# Solution – 100. Same Tree

Easy



LeetCode

[leetcode.com/problems/same-tree](https://leetcode.com/problems/same-tree)

## Solution

- Traverse both trees (**p** and **q**) recursively and check if the nodes are the same
- Start by the base case:  
are **p** and **q** null? return true
- One of them are null? return false, because they should be the same
- Finally, check if **p->val** is equal to **q->val** and also for both and left, recursively

# Code – 100. Same Tree

Easy



LeetCode

[leetcode.com/problems/same-tree](https://leetcode.com/problems/same-tree)

**Code** Time:  $O(n)$  where  $n$  is the number of nodes Space:  $O(h)$  where  $h$  is the height of the tree. Best case is usually  $O(\log n)$  for balanced trees, but skewed trees is usually  $O(n)$

```
bool isSameTree(TreeNode* p, TreeNode* q) {  
    // base case: leaf is null. If both are null, then return true  
    if (!p && !q) return true;  
    // if both are not NULL, then they must have value.  
    // If one of them doesn't have value, then they're different, return false  
    if (!p || !q) return false;  
    // they must have the same value  
    // as any other nodes in the tree  
    return p->val == q->val &&  
           isSameTree(p->left, q->left) &&  
           isSameTree(p->right, q->right);  
}
```



# Problem – 226. Invert Binary Tree

Easy

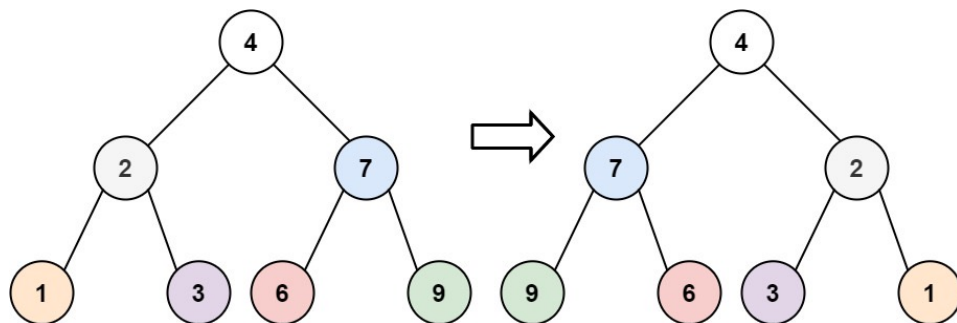


LeetCode

[leetcode.com/problems/invert-binary-tree](https://leetcode.com/problems/invert-binary-tree)

## Problem

- You are given the root of a binary tree
- Invert the tree and return the root
- **Example:**



# Solution – 226. Invert Binary Tree

Easy



LeetCode

[leetcode.com/problems/invert-binary-tree](https://leetcode.com/problems/invert-binary-tree)

## Solution

- Recursively traverse the tree
- Create a new pointer **temp** that points to **left** node
- Set **left** node to **right**
- Set **right** node to **temp**
- Call the function recursively for **left** and **right**
- Return root

# Code – 226. Invert Binary Tree

Easy



LeetCode

[leetcode.com/problems/invert-binary-tree](https://leetcode.com/problems/invert-binary-tree)

**Code** Time:  $O(n)$  Space:  $O(h)$  where  $h$  is the height of the tree

```
TreeNode* invertTree(TreeNode* root) {  
    // base case  
    if (!root) return nullptr;  
  
    // create a new pointer to left  
    TreeNode* temp = root->left;  
    // invert  
    root->left = root->right;  
    root->right = temp;  
  
    // recursively invert left and right  
    invertTree(root->left);  
    invertTree(root->right);  
  
    return root;  
}
```

# Problem – Path Sum

Easy



LeetCode

<https://leetcode.com/problems/path-sum>

## Problem Statement

- It is given the **root** of a binary tree and an integer **target sum**

- Example:**

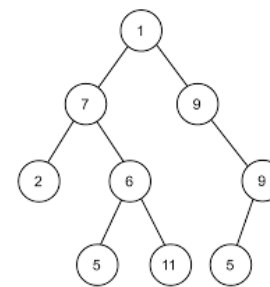
root = [1,7,9,2,6,null,9,null,null,5,11,5,null]

target sum = 10

- Return true if there is a path from root to leaf that adds up to 10

- Output:** true

Node 1 + Node 7 + Node 2 = 10



# Solution – Path Sum

Easy

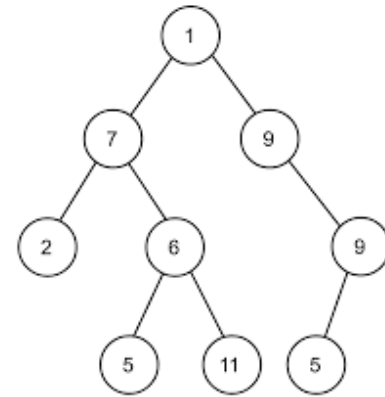


LeetCode

<https://leetcode.com/problems/path-sum>

## Solution

- Start from root node (1)
- Subtract from target number (example  $10 - 1 = 9$ )
- Continue going down the tree, until the target is 0, return true
- After visiting all nodes, if the target is not zero, return false



# Code – Path Sum

Easy



LeetCode

<https://leetcode.com/problems/path-sum>

```
bool hasPathSum(TreeNode* root, int targetSum) {
    if (!root) {
        return false;
    }
    // we want targetSum to be zero
    targetSum -= root->val;
    // if there is no left, no right, we've reached the end of the path
    // so if the targetSum is zero, then the nodes summed up to the targetSum
    if (!root->left && !root->right && targetSum == 0) {
        return true;
    }
    // propagate to left and right
    return hasPathSum(root->left, targetSum) || hasPathSum(root->right, targetSum);
}
```

Also, a small performance tweak can be made by avoiding writing *targetSum*: *targetSum -= root->val*

This will avoid a memory write access, making the calculation directly in the CPU, but also at a cost of readability

```
if (!root->left && !root->right && targetSum - root->val == 0) {
    ...
    return hasPathSum(root->left, targetSum - root->val) || hasPathSum(root->right, targetSum - root->val);
}
```

# Problem – 102. Binary Tree Level Order Traversal

Medium



LeetCode

[leetcode.com/problems/binary-tree-level-order-traversal](https://leetcode.com/problems/binary-tree-level-order-traversal)

**Problem Statement / Solution / Code**   Time:  $O(-)$    Space:  $O(-)$

■ ...

# Problem – 297. Serialize and Deserialize Binary Tree

Hard



LeetCode

[leetcode.com/problems/serialize-and-deserialize-binary-tree](https://leetcode.com/problems/serialize-and-deserialize-binary-tree)

## Problem

- Design an algorithm to serialize and deserialize a binary tree
- You have to build two interfaces: serialize that returns a string, and deserialize that returns the whole tree as `TreeNode` pointer
- The string can be represented at any format (comma-separated, space separated etc)



# Solution – 297. Serialize and Deserialize Binary Tree

Hard

 LeetCode [leetcode.com/problems/serialize-and-deserialize-binary-tree](https://leetcode.com/problems/serialize-and-deserialize-binary-tree)

## Solution

- **Serialize:** traverse the tree pre-order, and append its value to a string

Null value should also be represented

Example: [1,2,null,null,3 ...]

Call "traverse" to do it recursively

- **Deserialize:** split the string into tokens  
read each token and re-build the tree by adding a new node  
Call "buildTree" to do it recursively

# Code – 297. Serialize and Deserialize Binary Tree

Hard

 [leetcode.com/problems/serialize-and-deserialize-binary-tree](https://leetcode.com/problems/serialize-and-deserialize-binary-tree)

**Code** Time:  $O()$  Space:  $O()$

```
string serialize(TreeNode* root) {
    // traverse the tree in pre-order: root, left, right
    // generate a string with comma separator,
    // example: 1,2,N,N,3 ...
    string result;
    traverse(root, result);
    return result;
}

TreeNode* deserialize(string data) {
    // split the input data
    vector<string> tokens = split(data);
    // index to be used to access the elements from tokens recursively.
    // Hence, we need to create it here to pass by reference.
    // Note that index is bounded by the number of tokens, so it won't overflow
    int index = 0;
    TreeNode* root = buildTree(tokens, index);
    return root;
}
```

**continue...**

# Code – 297. Serialize and Deserialize Binary Tree

Hard

 [leetcode.com/problems/serialize-and-deserialize-binary-tree](https://leetcode.com/problems/serialize-and-deserialize-binary-tree)

```
TreeNode* buildTree(vector<string>& tokens, int& index) {
    // read the current token based on the index
    const string& token = tokens[index];
    // increment index before checking for null
    ++index;
    // base case: null node
    if (token == "N") {
        return nullptr;
    }
    // build root
    TreeNode* node = new TreeNode(stoi(token));
    // build left
    node->left = buildTree(tokens, index);
    // build right
    node->right = buildTree(tokens, index);
    return node;
}
```

```
// traverse in pre-order (root, left, right)
// and append the values to the string 's'
// append 'N' if it is NULL
void traverse(TreeNode* root, string& s) {
    if (!s.empty()) s += ",";
    // base case, we need to append null
    if (!root) {
        s += "N";
        return;
    }
    // visit root
    s += to_string(root->val);
    // visit left
    traverse(root->left, s);
    // visit right
    traverse(root->right, s);
}
```

```
// helper function in C++ to split string
vector<string> split(const string& s) {
    vector<string> result;
    stringstream ss(s);
    string token;
    while (getline(ss, token, ',')) {
        result.push_back(token);
    }
    return result;
}
```

# Code – 297. Serialize and Deserialize Binary Tree

Hard

 [leetcode.com/problems/serialize-and-deserialize-binary-tree](https://leetcode.com/problems/serialize-and-deserialize-binary-tree)

## Some interesting alternative to split

- C++ 23 have an interesting way to split using `std::views::split`

```
vector<string> split(string s) {  
    auto result = s |  
        views::split(',') |  
        views::transform([](auto&& subRange) {  
            return string(subRange.start(), subRange.end());  
        });  
}
```

- To understand, this follow a structure similar to unix pipes:

```
echo "123,N,556" | split | transform
```

- `std::views::split` returns ranges, something like:

```
[ range("123"), range("N"), range("556") ]
```

- `std::views::transform` converts each subrange into an actual string

# Problem – 572. Subtree of Another Tree

Easy



LeetCode

[leetcode.com/problems/subtree-of-another-tree](https://leetcode.com/problems/subtree-of-another-tree)

## Problem

- You are given the root of a binary tree `root` and the root of another binary tree `subRoot`
- Determine whether `subRoot` is a **subtree** of `root`.
- A subtree of a binary tree is a node in the tree along with all of its descendants
- The tree itself is also considered a subtree.

# Problem – 572. Subtree of Another Tree

Easy



LeetCode

[leetcode.com/problems/subtree-of-another-tree](https://leetcode.com/problems/subtree-of-another-tree)

## Solution

- Define a helper function `isSameTree(a, b)` that checks if two trees rooted at `a` and `b` are identical in both structure and node values.
- Traverse the root tree, and for each node:
- Use `isSameTree(node, subRoot)` to check if a matching subtree starts at that node.
- Return true if any such match is found; otherwise, return false.

# Problem – 572. Subtree of Another Tree

Easy



LeetCode

[leetcode.com/problems/subtree-of-another-tree](https://leetcode.com/problems/subtree-of-another-tree)

**Code** Time:  $O(m * n)$  Space:  $O(h)$  where  $n$  is the number of nodes of the tree and  $m$  the number of nodes of the subtree, and  $h$  the height of the tree.

```
bool isSame(TreeNode* q, TreeNode* r) {
    // both are null, so they're the same
    if (!q && !r) return true;
    // if they're not null, both must be not null
    if (!q || !r) return false;
    // now check the values
    if (q->val != r->val) return false;
    // check left and right
    return isSame(q->left, r->left) && isSame(q->right, r->right);
}

bool isSubtree(TreeNode* root, TreeNode* subRoot) {
    if (!root) return false;
    // Check starting from the root first
    if (isSame(root, subRoot)) {
        return true;
    }
    // they are not the same starting from the root,
    // but still subRoot may be in the middle of root. So check it recursively
    return isSubtree(root->left, subRoot) || isSubtree(root->right, subRoot);
}
```

# Problem – 105. Construct Binary Tree from Preorder and Inorder Traversal

Medium

 [leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal](https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal)

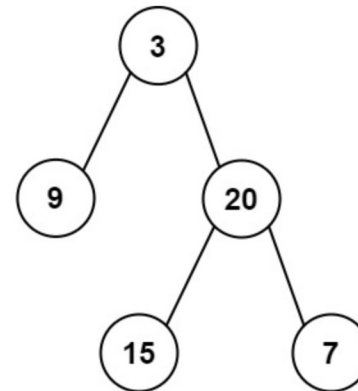
## Problem

- You are given two **array of integers: preorder and inorder**
- **preorder** is the pre-order traversal of a binary tree
- **inorder** is the in-order traversal of a binary tree
- Based on those two arrays, construct the binary tree
- **Example**

Input: preorder = [3,9,20,15,7]

inorder = [9,3,15,20,7]

Output: the head pointer of the binary tree





# Problem – 98. Validate Binary Search Tree

Medium



LeetCode

[leetcode.com/problems/validate-binary-search-tree](https://leetcode.com/problems/validate-binary-search-tree)

## Problem

- You are given the root of a **binary tree**
- Determine if the tree is a valid **binary search tree** according to the following:
- The left subtree contains all elements less than the node's key
- The right subtree contains all elements greater than the node's key
- Both left and right must also be binary search trees

# Solution – 98. Validate Binary Search Tree

Medium



LeetCode

[leetcode.com/problems/validate-binary-search-tree](https://leetcode.com/problems/validate-binary-search-tree)

## Solution

- For each node, there is a **maximum** and a **minimum** value where node lies
- The range of the root is  $(-\infty, \infty)$
- Traverse from **top to bottom**
- If you are checking the right subtree, all the elements must be higher than a minimum value
- When traversing right, make sure to update the minimum value
- The same is true for the left subtree - all the elements must be less than the maximum value

# Solution – 98. Validate Binary Search Tree

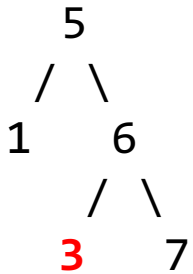
Medium



LeetCode

[leetcode.com/problems/validate-binary-search-tree](https://leetcode.com/problems/validate-binary-search-tree)

## Solution (example)



- The minimum of all elements at the right of "5" should be greater than 5
- So this tree is invalid
- For each node, the values must range between a min and a max:

node = (max, min)

node 5 =  $(-\infty, +\infty)$  can be any value

node 1 =  $(-\infty, 5)$  must be any value less than 5

node 6 =  $(5, +\infty)$  this node can be any value greater than 5

**node 3 = (5, 6)** must be greater than the subtree the node belongs and less than the immediate parent

# Code – 98. Validate Binary Search Tree

Medium



LeetCode

[leetcode.com/problems/validate-binary-search-tree](https://leetcode.com/problems/validate-binary-search-tree)

**Code** Time:  $O(n)$  Space:  $O(n)$  where  $n$  is the number of the nodes in the tree

```
// Use long long for min/max due the constraints
bool isValid(TreeNode* root, long long min, long long max) {
    // base case: if we reach the bottom, that means we've
    // checked all nodes along the way and everything is fine
    if (!root) return true;

    // compare the min/max values
    if (root->val <= min || root->val >= max) return false;
    // go right and left. When going right, the minimum should be updated and
    // when going left, the maximum should be updated by its immediate parent
    return isValid(root->right, root->val, max) && isValid(root->left, min, root->val);
}

bool isValidBST(TreeNode* root) {
    return isValid(root, LONG_MIN, LONG_MAX);
}
```

# Problem – 208. Implement Trie (Prefix Tree)

Medium



LeetCode

[leetcode.com/problems/implement-trie-prefix-tree](https://leetcode.com/problems/implement-trie-prefix-tree)

## Problem

- Implement Trie (prefix tree)
- Trie is a specialized search tree data structure used to store and retrieve strings from a dictionary set
- **Each node** represents a character of a string
- **Root node** is usually empty and does not store any character
- **Path from root to leaf node** represents the path that forms a word
- Insert/Search/Delete operations are done character by character
- Typically time complexity is  **$O(L)$**  where  $L$  is the length of the word
- The problem ask to implement a Trie class, including:

`Trie()` initialization

`void insert(string word)`

`bool search(string word)`

`bool startsWith(string prefix)`

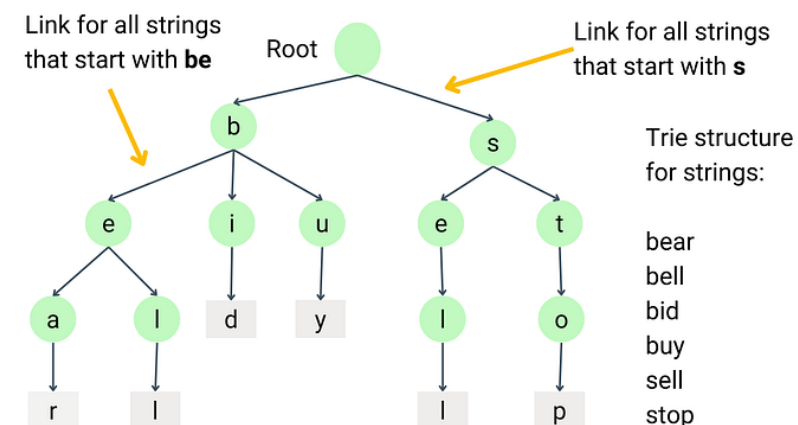


Image from <https://www.enjoyalgorithms.com/blog/introduction-to-trie-data-structure>

# Problem – 208. Implement Trie (Prefix Tree)

Medium



LeetCode

[leetcode.com/problems/implement-trie-prefix-tree](https://leetcode.com/problems/implement-trie-prefix-tree)

## Solution

- Create a struct to represent the node. Each node is connected to 26 children (size of the English alphabet)

```
struct TrieNode {  
    TrieNode* children[26] = {};  
    bool isLeaf = false;  
};
```

- **Initialization**

Create a new TrieNode object root

- **Insert**

Iterate over each character 'c' in the word

if children[c - 'a'] is nullptr, create a new TrieNode

Move to the child node

After the last character, mark the node as leaf (isLeaf = true)

# Problem – 208. Implement Trie (Prefix Tree)

Medium



LeetCode

[leetcode.com/problems/implement-trie-prefix-tree](https://leetcode.com/problems/implement-trie-prefix-tree)

## Solution

### ▪ Search

Traverse the trie character by character

If at any point the child node for a character doesn't exist, return false

After traversal, return true only if isLeaf == true

### ▪ StartsWith

Traverse the trie character by character

If any character node is missing, return false

Once reach the end of the string, return true

# Problem – 208. Implement Trie (Prefix Tree)

Medium



LeetCode

[leetcode.com/problems/implement-trie-prefix-tree](https://leetcode.com/problems/implement-trie-prefix-tree)

**Code** Time:  **$O(L)$**  Space:  **$O(L)$**  where L is the average length of each word

```
class Trie {
private:
    struct TrieNode {
        TrieNode* children[26] = {};
        bool endOfWord = false;
    };

    TrieNode* root;
public:
    explicit Trie() {
        root = new TrieNode();
    }

    void insert(string word) {
        TrieNode* node = root;
        for (const auto& c : word) {
            int i = c - 'a';
            if (!node->children[i]) {
                node->children[i] = new TrieNode();
            }
            node = node->children[i];
        }
        node->endOfWord = true;
    }

    TrieNode* findLastNode(const string& word) {
        TrieNode* node = root;
        for (const auto& c : word) {
            node = node->children[c - 'a'];
            if (!node) return nullptr;
        }
        return node;
    }

    bool search(string word) {
        TrieNode* node = findLastNode(word);
        return node && node->endOfWord;
    }

    bool startsWith(string prefix) {
        return findLastNode(prefix) != nullptr;
    }
};
```



# Problem – Kth Smallest Element in a BST

Medium



LeetCode

[leetcode.com/problems/kth-smallest-element-in-a-bst](https://leetcode.com/problems/kth-smallest-element-in-a-bst)

## Problem Statement / Solution

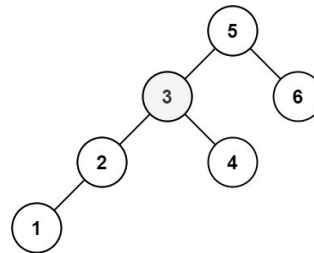
- You are given the **root** of a binary search tree and an **integer k**
- Find the  $k^{\text{th}}$  smallest value

- **Example**

From all values in the tree: 1,2,3,4,5,6

**k = 3** so find the 3<sup>th</sup> smallest value

**Output** is 3: 1,2,**3**,4,5,6 (3th)



# Solution – Kth Smallest Element in a BST

Medium

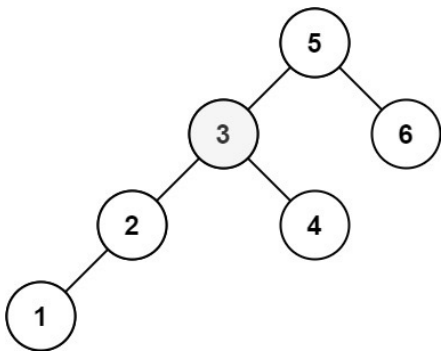


LeetCode

[leetcode.com/problems/kth-smallest-element-in-a-bst](https://leetcode.com/problems/kth-smallest-element-in-a-bst)

## Solution

- Note that the smallest element is in the left leaf
- Therefore, there is an order from small  $\rightarrow$  big values from left  $\rightarrow$  root  $\rightarrow$  right
- Perform in-order traversal **k** times and stop in the desired node



# Code – Kth Smallest Element in a BST

Medium



LeetCode

[leetcode.com/problems/kth-smallest-element-in-a-bst](https://leetcode.com/problems/kth-smallest-element-in-a-bst)

**Code** Time:  $O(k)$  Space:  $O(h)$  where  $h$  is the height of the tree

```
// in-order traversal: left, node: right
void traverse(TreeNode* node, int& k, int& result) {
    // base case
    if (!node) return;
    // visit left first
    traverse(node->left, k, result);
    // visit node
    k--;
    if (k == 0) {
        result = node->val;
        return;
    }
    // visit right
    traverse(node->right, k, result);
}

int kthSmallest(TreeNode* root, int k) {
    // perform pre-order traversal
    int result;
    traverse(root, k, result);
    return result;
}
```

# Problem – 235. Lowest Common Ancestor of a Binary Search Tree

Medium



LeetCode

[leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree](https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree)

## Problem

- You are given the **root** of a binary search tree and two nodes **p** and **q**
- Return the lowest common ancestor (LCA) node
- The LCA between two nodes **p** and **q** is the lowest node that has both **p** and **q** as descendants.  
The node itself can be a descendant of itself as well

# Solution – 235. Lowest Common Ancestor of a Binary Search Tree

Medium

 [leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree](https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree)

## Solution

- Given the binary search tree, figure out whether to traversal **right** or **left**:
- If **p** and **q** are **smaller** than the node, traverse **left**
- if **p** and **q** are **greater** than the node, traverse **right**
- if neither, return **node** which is the **LCA**

# Code – 235. Lowest Common Ancestor of a Binary Search Tree

Medium

 [leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree](https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree)

**Code** Time:  **$O(h)$**  Space:  **$O(h)$**  where  $h$  is the height of the tree. Normally,  $h = \log(n)$  where  $n$  is the number of the nodes. For skewed trees, it could be  $O(n)$

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {  
    if (p->val < root->val && q->val < root->val) {  
        return lowestCommonAncestor(root->left, p, q);  
    }  
  
    if (p->val > root->val && q->val > root->val) {  
        return lowestCommonAncestor(root->right, p, q);  
    }  
  
    return root;  
}
```

# Problem – 124. Binary Tree Maximum Path Sum

Hard



LeetCode

[leetcode.com/problems/binary-tree-maximum-path-sum](https://leetcode.com/problems/binary-tree-maximum-path-sum)

## Problem

- You are given the root node of a binary tree
- Return the **max path sum** of any path
- A path can be linear (from the root all the way down to the leaf) or the three node: root, left and right)
- A path can start at any node

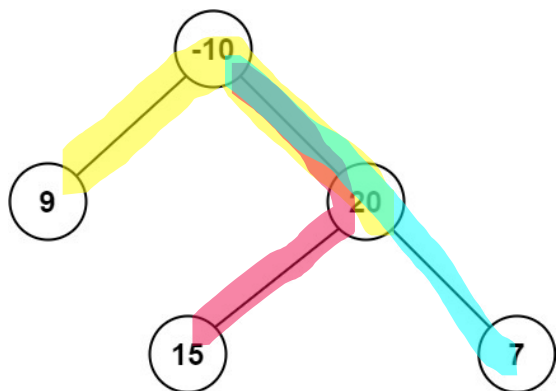
- **Example:**

$9 \rightarrow -10 \rightarrow 20$  is a valid path

$-10 \rightarrow 20 \rightarrow 15$  is a valid path

$9 \rightarrow -10 \rightarrow 20 \rightarrow 7$  is **NOT** a valid path

$20 \rightarrow 7$  is a valid path



# Solution – 124. Binary Tree Maximum Path Sum

Hard

 [leetcode.com/problems/binary-tree-maximum-path-sum](https://leetcode.com/problems/binary-tree-maximum-path-sum)

## Solution

- Use **post-order** traversal (bottom-up recursion)
- At each node:
  - Recursively compute left and right max path gains
  - Consider all 3 possible paths:
    1. Turn path: left + root + right
    2. Linear path: root + left
    3. Linear path: root + right
  - Also consider just the root (if the children is negative)
- Track the maximum path seen so far
- Only return linear path (root + one child) upward to maintain the valid structure
- Also, prune negative gain before returning



# Problem – 124. Binary Tree Maximum Path Sum

Hard

 [leetcode.com/problems/binary-tree-maximum-path-sum](https://leetcode.com/problems/binary-tree-maximum-path-sum)

**Code** Time:  $O(n)$  Space:  $O(h)$  where  $n$  is the number of nodes and  $h$  is the height of the tree.

```
int findMaxSum(TreeNode* root, int& maxSum) {
    if (!root) return 0;
    int left = findMaxSum(root->left, maxSum);
    int right = findMaxSum(root->right, maxSum);
    // 1st possible path: exactly the only 3 nodes: root, right and left
    int threeNodes = left + right + root->val;
    // 2nd possible path, linear recursive path: root + left
    int secondPath = root->val + left;
    // 3rd possible path, linear recursive path: root + right
    int thirdPath = root->val + right;

    // check if we should consider left, right or only root itself
    int bestPath = max({root->val, secondPath, thirdPath});

    // maxSum can be the accumulated 2nd and 3rd (linear path)
    // or the threeNodes path
    maxSum = max({maxSum, bestPath, threeNodes});

    // Prune subtree: we start from the bottom, so we can set 0
    // to ignore left or right path
    return max(0, bestPath);
}

int maxPathSum(TreeNode* root) {
    int maxSum = INT_MIN;
    findMaxSum(root, maxSum);
    return maxSum;
}
```

# Problem – 124. Binary Tree Maximum Path Sum

Hard

 [leetcode.com/problems/binary-tree-maximum-path-sum](https://leetcode.com/problems/binary-tree-maximum-path-sum)

**Code (compact)** Time:  $O(n)$  Space:  $O(h)$  where  $n$  is the number of nodes and  $h$  is the height of the tree.

```
int find(TreeNode *node, int& totalMax) {
    if (!node) return 0;
    int leftGain = max(0, find(node->left, totalMax));
    int rightGain = max(0, find(node->right, totalMax));
    int currentMax = node->val + leftGain + rightGain;
    totalMax = max(totalMax, currentMax);
    return node->val + max(leftGain, rightGain);
}

int maxPathSum(TreeNode* root) {
    int totalMax = INT_MIN;
    find(root, totalMax);
    return totalMax;
}
```

# Problem – 872. Leaf-Similar Trees

Easy



LeetCode

[leetcode.com/problems/leaf-similar-trees](https://leetcode.com/problems/leaf-similar-trees)

## Problem Statement

- You are given two trees
- The goal is to compare if they have the same leaves
- The leaves should be in the same order
- Example:

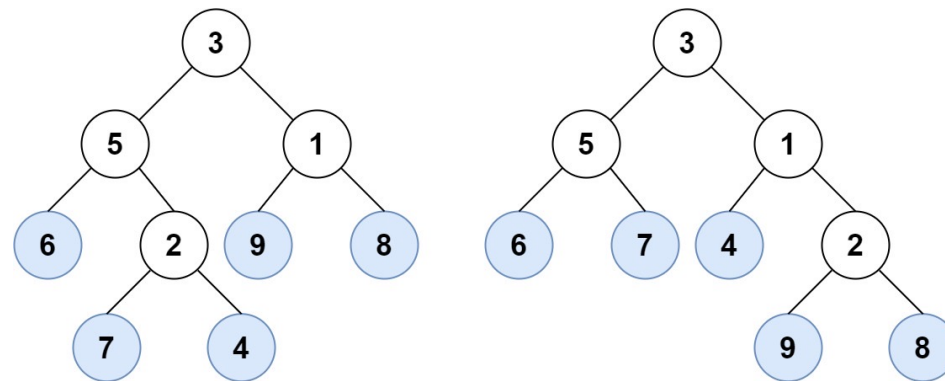
First tree:

**leaves = 6,7,4,9,8** (blue nodes)

Second tree:

**leaves = 6,7,4,9,8**

- Return true if the leaves are the same



# Solution – Leaf-Similar Trees

Easy



LeetCode

[leetcode.com/problems/leaf-similar-trees](https://leetcode.com/problems/leaf-similar-trees)

## Solution

- Get the first leaf value from tree 1
- Get the first leaf value from tree 2
- Compare, if they are different, return false immediately
- Otherwise, continue finding the next leaf value for tree 1 and 2

## Implementation

- Create two stacks **stack<TreeNode\*> left** and **stack<TreeNode\*> right**
- Add the

# Code – Leaf-Similar Trees

Easy



LeetCode

[leetcode.com/problems/leaf-similar-trees](https://leetcode.com/problems/leaf-similar-trees)

**Code** Time:  $O(n + m)$  where  $n$  and  $m$  are the numbers of nodes for trees 1 and 2 Space:  $O(h_1 + h_2)$  where  $h_1$  and  $h_2$  represents the height of the tree

```
// returns the value of the leaf, or -1 if empty
int getLeaf(stack<TreeNode*>& tree) {
    // tree is a reference, we will always pop an element from it
    while(!tree.empty()) {
        // get the top element from the stack
        TreeNode* node = tree.top();
        // already visited, so remove from stack
        tree.pop();
        // is this a leaf?
        if (!node->left && !node->right) {
            // yes, return the value
            return node->val;
        }
        // push the right FIRST to the stack
        if (node->right) tree.push(node->right);
        // left should be on top of the stack
        if (node->left) tree.push(node->left);
    }
    return -1;
}
```

```
bool leafSimilar(TreeNode* root1, TreeNode* root2) {
    // initialize the stacks, add root1 and root2
    std::stack<TreeNode*> leftTree, rightTree;
    leftTree.push(root1);
    rightTree.push(root2);

    while(true) {
        // get the leaves to compare
        int leaf1 = getLeaf(leftTree);
        int leaf2 = getLeaf(rightTree);
        // exit immediately if one leaf is different
        if (leaf1 != leaf2) return false;
        // stop when there are no leaves left
        if (leaf1 == -1 || leaf2 == -1) break;
    }
    return true;
}
```

# Code – Leaf-Similar Trees

Easy



LeetCode

[leetcode.com/problems/leaf-similar-trees](https://leetcode.com/problems/leaf-similar-trees)

**Code (another approach)** Time:  $O(n + m)$  where  $n$  and  $m$  are the numbers of nodes for trees 1 and 2 Space:  $O(h1 + h2)$  where  $h1$  and  $h2$  represents the height of the tree

```
void extractLeafs(TreeNode* node, vector<int>& leafValues) {
    // base case, return
    if (!node) return;
    // if it looks like a leaf, no left child
    // like a leaf, no right child like a leaf,
    // then it's probably a leaf
    // add to the vector
    if (!node->left && !node->right) {
        leafValues.push_back(node->val);
    }
    // continue looking at left and right
    extractLeafs(node->left, leafValues);
    extractLeafs(node->right, leafValues);
}
```

```
bool leafSimilar(TreeNode* root1, TreeNode* root2) {
    vector<int> tree1Values;
    vector<int> tree2Values;
    // extract all leafs from tree 1
    extractLeafs(root1, tree1Values);
    // extract all leafs from tree 2
    extractLeafs(root2, tree2Values);
    // compare
    return tree1Values == tree2Values;
}
```

# Problem – 1448. Count Good Nodes in Binary Tree

Medium

 [leetcode.com/problems/count-good-nodes-in-binary-tree](https://leetcode.com/problems/count-good-nodes-in-binary-tree)

## Problem Statement

- You are given a binary tree and have to find "**good**" nodes
- A **good node** is a **node** where the values in the path are always less than or equal to the **node**
- The **root node** is always a **good node**
- Example:

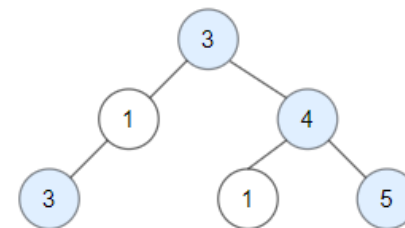
- root 3 is a good node

### left side:

- left **leaf 1** is not a good node because  $1 < 3$
- **leaf 3** is a good node because  $3 > 1$  and  $3 == 3$

### right side:

- **leaf 4** is a good node because  $4 > 3$
- **leaf 1** is not a good node because  $1 < 4$
- **leaf 5** is a good node because  $5 > 4 > 3$



# Solution – 1448. Count Good Nodes in Binary Tree

Medium

 LeetCode [leetcode.com/problems/count-good-nodes-in-binary-tree](https://leetcode.com/problems/count-good-nodes-in-binary-tree)

## Solution

- Use DFS traversal to explore the tree from the root to all leaf nodes
- As you traverse, **keep track of the maximum value** along the path from root to node
- Update max value once you find a node value greater than the max value

- **Recursive logic**

Base case: if the node is *nullptr*, return 0

At each node:

- Compare its value to max so far
- If it is a good node, increase a local count
- Recursively repeat this process for the left and right children, passing along the updated max value



# Code – 1448. Count Good Nodes in Binary Tree

Medium

 [leetcode.com/problems/count-good-nodes-in-binary-tree](https://leetcode.com/problems/count-good-nodes-in-binary-tree)

**Code** Time:  $O(n)$  Space:  $O(h)$

```
int traverse(TreeNode* root, int maxValue) {
    if (!root) return 0;
    // is this a good node?
    int count = 0;
    if (root->val >= maxValue) {
        maxValue = root->val;
        count = 1;
    }
    count += traverse(root->left, maxValue);
    count += traverse(root->right, maxValue);
    return count;
}

int goodNodes(TreeNode* root) {
    if (!root) return 0;
    return traverse(root, root->val);
}
```

**INTERVAL**

# Some Common Patterns

## Merge Intervals

- Sort by **start time**
- Merge if `current.start <= previous.end`
- Classic use: *merging calendar events, range compression*

## Interval Scheduling (Max Non-overlapping Intervals)

- Sort by **end time**
- Greedy: select interval only if `start >= last_selected.end`
- Optimal because ending earlier leaves more time for future

## Minimum Number of Arrows to Burst Balloons

- Same as interval scheduling
- Sort by **end time**
- Count how many non-overlapping intervals = minimum arrows needed

# Some Common Patterns II

## Minimum Meeting Rooms

- Sort by start and end times separately
- Use a min-heap to track active meetings
- Greedy, but uses more advanced data structure

## Can Attend All Meetings?

- Sort by start time
- Check if any overlap with previous: if `start < previous.end`, return false

# Problem – 57. Insert Interval

Medium



LeetCode

[leetcode.com/problems/insert-interval](https://leetcode.com/problems/insert-interval)

## Problem Statement

- You are given an array of **intervals**, where **intervals[i] = [start<sub>i</sub>, end<sub>i</sub>]** and **newInterval = [start, end]**
- **newInterval** must be inserted into **intervals**
- Overlapping intervals must be merged
- Example

**intervals** = [[1,2],[3,5],[6,7],[8,10],[12,16]] **newInterval** = [4,8]

**Output:** [[1,2],[3,10],[12,16]]

# Solution – 57. Insert Interval

Medium



LeetCode

[leetcode.com/problems/insert-interval](https://leetcode.com/problems/insert-interval)

## Solution

- Sort intervals by the first element (start)
- Initialize **result**
- Solve in three loops:
  1. While there is no overlap with **newInterval**, add to **intervals[i]** to **result**
  2. While it overlaps, merge **newInterval**
  3. While until the end intervals and add the remaining **intervals[i]**

# Code – 57. Insert Interval

Medium



LeetCode

[leetcode.com/problems/insert-interval](https://leetcode.com/problems/insert-interval)

**Code** Time:  $O(n)$  Space:  $O(n)$  where  $n$  is the size of intervals

```
vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int>& newInterval) {
    vector<vector<int>> result;
    int tupleIndex = 0;
    int totalTuples = intervals.size();
    // 1. check if it overlaps
    // 1 ----- 2
    //           4 ----- 8
    while (tupleIndex < totalTuples && intervals[tupleIndex][1] < newInterval[0]) {
        result.push_back(intervals[tupleIndex]);
        ++tupleIndex;
    }

    // 2. merge overlap. We already know there is an overlap here,
    // otherwise it should be sorted out in the previous step
    // 3 ---- 5
    //      4 ----- 8
    while (tupleIndex < totalTuples && intervals[tupleIndex][0] <= newInterval[1]) {
        newInterval[0] = min(newInterval[0], intervals[tupleIndex][0]);
        newInterval[1] = max(newInterval[1], intervals[tupleIndex][1]);
        ++tupleIndex;
    }
    result.push_back(newInterval);

    // 3. add remaining parts
    while (tupleIndex < totalTuples) {
        result.push_back(intervals[tupleIndex]);
        ++tupleIndex;
    }
    return result;
}
```

# Problem – 56. Merge Intervals

Medium



LeetCode

[leetcode.com/problems/merge-intervals](https://leetcode.com/problems/merge-intervals)

## Problem Statement

- You are given an array of intervals, example:

```
intervals = [[1,3],[2,6],[8,10],[15,18]]
```

- Merge all overlapping intervals. So the output should be:

```
[[1,6],[8,10],[15,18]]
```

- Interval [1,3] was merged with [2,6]



# Solution – 56. Merge Intervals

Medium



LeetCode

[leetcode.com/problems/merge-intervals](https://leetcode.com/problems/merge-intervals)

## Solution

- Sort the array based on the beginning of the interval
- In C++, when applying `sort(intervals.begin(), intervals.end())` the default comparator compares `vector<vector<int>>` lexicographically:
  - it first compares the first element [0] of each sub-vector
  - if those are equal, it compares the second element [1] and so on
- Go over each interval and compare
- `interval[i][begin] <= interval[i - 1][end]` ? then merge
- To merge, set the current `interval[i][begin]` to `interval[i - 1][begin]` and set the `interval[i][end]` to the maximum value between `interval[i][end]` and `interval[i - 1][end]`
- If no merge is necessary, push the previous interval to the result array
- Once the loop finishes, add the last element and return the result

# Code – 56. Merge Intervals

Medium



LeetCode

[leetcode.com/problems/merge-intervals](https://leetcode.com/problems/merge-intervals)

**Code** Time:  $O(n \log n)$  Space:  $O(n)$

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    if (intervals.empty()) return {};
    sort(intervals.begin(), intervals.end());
    vector<vector<int>> result;

    result.push_back(intervals[0]);

    for (int i = 1; i < intervals.size(); ++i) {
        vector<int>& current = intervals[i];
        vector<int>& previous = result.back();

        // check if they overlap, if so merge...
        // they're sorted, we know that:
        // previous[0] >= current[0]
        // 1 --- 3 (previous)
        // 2 ----- 6 (current)
        if (current[0] <= previous[1]) {
            // merge
            previous[1] = max(previous[1], current[1]);
        } else {
            result.push_back(current);
        }
    }
    return result;
}
```

# Problem – 435. Non-overlapping Intervals

Medium



LeetCode

[leetcode.com/problems/non-overlapping-intervals](https://leetcode.com/problems/non-overlapping-intervals)

## Problem Statement

- You are given an array of intervals `vector<vector<int>>` with start and end, Example:  
`intervals = [[1,2],[2,3],[3,4],[1,3]]`
- The intervals **must not** overlap each other
- You have to remove the minimum number of pairs to make it non-overlapping

# Solution – 435. Non-overlapping Intervals

Medium



LeetCode

[leetcode.com/problems/non-overlapping-intervals](https://leetcode.com/problems/non-overlapping-intervals)

## Solution

- Sort the array by the ending time:

$[[1,2],[2,3],[3,4],[1,3]] \rightarrow [[1,2],[2,3],[1,3],[3,4]]$

- In C++ a lambda function can be used with sort:

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>&b) {  
    return a[1] < b[1];  
});
```

- Note that **std::sort** is not stable (opposite of **std::stable\_sort**), so there is no guarantees that [2,3] comes before [1,3]. But for this algorithm, it doesn't matter
- Iterate over the array and check overlaps by comparing the end[i] with begin[i - 1]
- If they overlap, logically remove the current pair and count + 1
- Logically removing means just setting the end to compare to the previous element, so "skip" the current interval

# Code – 435. Non-overlapping Intervals

Medium



LeetCode

[leetcode.com/problems/non-overlapping-intervals](https://leetcode.com/problems/non-overlapping-intervals)

**Code** Time:  $O(n \log n)$  Space:  $O(1)$

```
int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    // sort by the ending time  $O(\log n)$ 
    sort(intervals.begin(), intervals.end(), [](const auto& a, const auto& b) {
        return a[1] < b[1];
    });
    int result= 0;
    int end = intervals[0][1];

    //  $O(n)$ 
    for (int i = 1; i < intervals.size(); ++i) {
        // does it overlaps?
        if (intervals[i][0] < end) {
            ++result;
        } else {
            // it doesn't overlap, just 'skip'
            // the current interval
            end = intervals[i][1];
        }
    }
    return result;
}
```

# Problem – 110. Balanced Binary Tree

Easy



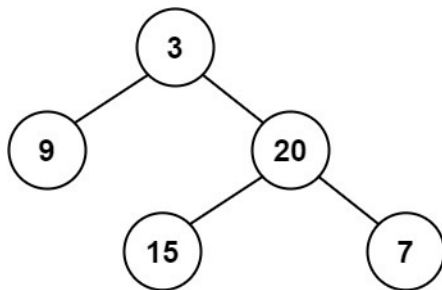
LeetCode

[leetcode.com/problems/balanced-binary-tree](https://leetcode.com/problems/balanced-binary-tree)

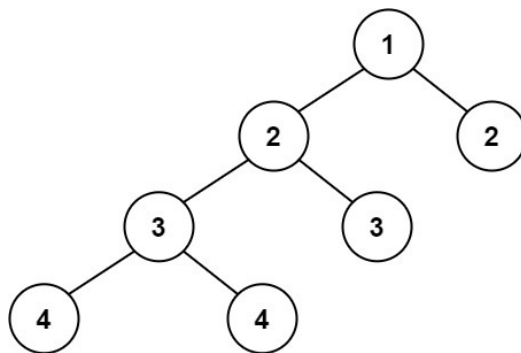
## Problem

- You are given the root of a binary tree
- Return true if it is height-balanced
- A tree is height-balanced when the height of two subtrees does not differ by two

### Height balanced



### Not Height balanced



# Problem – 110. Balanced Binary Tree

Easy



LeetCode

[leetcode.com/problems/balanced-binary-tree](https://leetcode.com/problems/balanced-binary-tree)

## Solution

- Recursive approach: go all the way down
- Calculate the height of the left subtree
- Calculate the height of the right subtree
- Compare both to check if they differ by more than one
- Continue going up the tree to check all the nodes

# Problem – 110. Balanced Binary Tree

Easy



LeetCode

[leetcode.com/problems/balanced-binary-tree](https://leetcode.com/problems/balanced-binary-tree)

**Code** Time:  $O(n)$  Space:  $O(h)$  where  $n$  is the number of the nodes and  $h$  is the height of the tree

```
int checkHeight(TreeNode* node) {
    if (!node) return 0;

    int left = checkHeight(node->left);
    // left tree is unbalanced
    if (left == -1) return -1;

    int right = checkHeight(node->right);
    // right tree is unbalanced
    if (right == -1) return -1;

    // check the different, -1 is unbalanced
    if (abs(left - right) > 1) return -1;

    return max(left, right) + 1;
}

bool isBalanced(TreeNode* root) {
    return checkHeight(root) != -1;
}
```



# LINKED LIST

# Problem – 206. Reverse Linked List

Easy

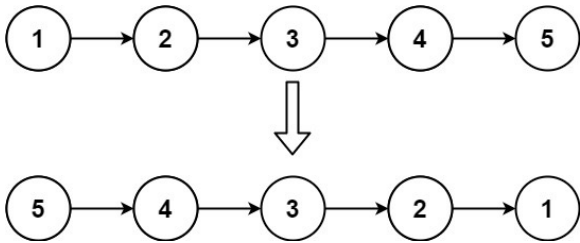


LeetCode

[leetcode.com/problems/reverse-linked-list](https://leetcode.com/problems/reverse-linked-list)

## Problem

- This is a classic problem
- Given a singly linked list, reverse its order



# Solution – 206. Reverse Linked List

Easy



LeetCode

[leetcode.com/problems/reverse-linked-list](https://leetcode.com/problems/reverse-linked-list)

## Solution

- Use recursive approach
- Looking at the pseudo-code, this recursion will return the last node:

```
reverseList(head) {  
    if (!head->next) return head  
    node = reverseList(head->next);  
    return node  
}
```

- From end to beginning, each head will be a node in the list
- Therefore, you can change this node by setting a new head:

```
head->next->next = head;  
head->next = nullptr;
```

# Code – 206. Reverse Linked List

Easy



LeetCode

[leetcode.com/problems/reverse-linked-list](https://leetcode.com/problems/reverse-linked-list)

**Code** Time:  **$O(n)$**  Space:  **$O(1)$**

```
ListNode* reverseList(ListNode* head) {  
    if (!head->next) return head;  
    ListNode* node = reverseList(head->next);  
    head->next->next = head;  
    head->next = nullptr;  
    return node;  
}
```

# Problem – 141. Linked List Cycle

Easy



LeetCode

[leetcode.com/problems/linked-list-cycle](https://leetcode.com/problems/linked-list-cycle)

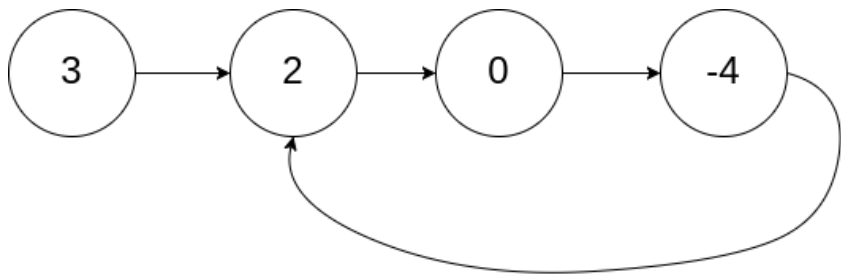
## Problem

- You are given the head of a linked list
- Return **true** if there is a cycle, false otherwise

- **Example:**

In the image below, there is a cycle (-4 to 2)

**Output:** true



# Solution – 141. Linked List Cycle

Easy



LeetCode

[leetcode.com/problems/linked-list-cycle](https://leetcode.com/problems/linked-list-cycle)

## Solution

- Have two pointers: fast and slow
- Slow will go over each item in the linked list
- Fast will go twice as fast as slow (`fast = fast->next->next`)
- If fast reach at the end, there is no cycle
- If fast encounter slow, there is a cycle, return true

# Code – 141. Linked List Cycle

Easy



LeetCode

[leetcode.com/problems/linked-list-cycle](https://leetcode.com/problems/linked-list-cycle)

**Code** Time: **O(n)** Space: **O(1)**

```
bool hasCycle(ListNode *head) {  
    if (!head || !head->next) return false;  
    ListNode* slow = head;  
    ListNode* fast = head;  
    while (fast && fast->next) {  
        slow = slow->next;  
        fast = fast->next->next;  
        if (slow == fast) return true;  
    }  
    return false;  
}
```

# Problem – 21. Merge Two Sorted Lists

Easy

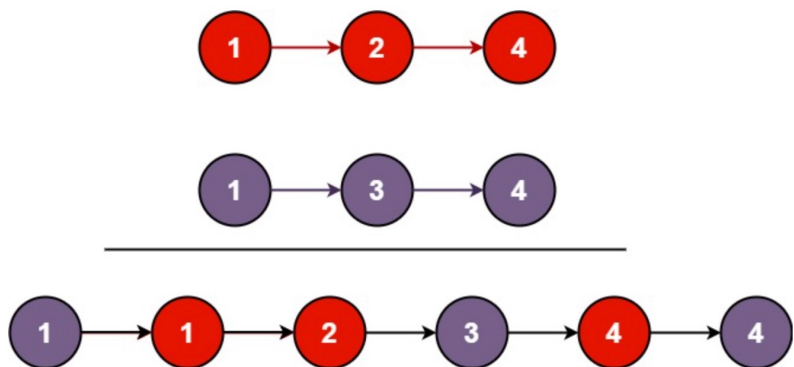


LeetCode

[leetcode.com/problems/merge-two-sorted-lists](https://leetcode.com/problems/merge-two-sorted-lists)

## Problem

- You are given the head of two linked lists (list1 and list2)
- Merge the two lists into one **sorted** list





# Solution – 21. Merge Two Sorted Lists

Easy



LeetCode

[leetcode.com/problems/merge-two-sorted-lists](https://leetcode.com/problems/merge-two-sorted-lists)

## Solution

- Recursively explore the two lists. Base case:

```
if (!list1) return list2;
```

```
if (!list2) return list1;
```

- Compare the value of the current node of list 1 and list 2

```
if (list1->val > list2->val) { ...
```

- Set the next node of the node with the minimum value:

assume the previous condition is true, so

```
list2->next = mergeTwoLists(list1, list2->next);
```

```
return list2;
```

meaning, we want list2->next to come before list1. But we do this recursively since we need the next result

# Code – 21. Merge Two Sorted Lists

Easy



LeetCode

[leetcode.com/problems/merge-two-sorted-lists](https://leetcode.com/problems/merge-two-sorted-lists)

**Code** Time:  $O(n + m)$  Space:  $O(n + m)$  where  $n$  is the length of list1 and  $m$  is the length of list2

```
ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {  
    if (!list1) return list2;  
    if (!list2) return list1;  
  
    if (list1->val < list2->val) {  
        list1->next = mergeTwoLists(list1->next, list2);  
        return list1;  
    } else {  
        list2->next = mergeTwoLists(list2->next, list1);  
        return list2;  
    }  
}
```

# Problem – 23. Merge k Sorted Lists

Hard



LeetCode

[leetcode.com/problems/merge-k-sorted-lists](https://leetcode.com/problems/merge-k-sorted-lists)

## Problem

- You are given an **array of k linked lists**
- Each linked list is **sorted** in ascending order
- Merge all linked lists into one **sorted** linked-lists

# Solution – 23. Merge k Sorted Lists

Hard



LeetCode

[leetcode.com/problems/merge-k-sorted-lists](https://leetcode.com/problems/merge-k-sorted-lists)

## Solution

- Create a function to merge two lists
- Go over the lists and merge with each over; **or**
- Use divide and conquer to merge (more optimal)
- Divide and conquer is more efficient because it avoids merging a big list with a small one multiple times

# Code – 23. Merge k Sorted Lists

Hard



LeetCode

[leetcode.com/problems/merge-k-sorted-lists](https://leetcode.com/problems/merge-k-sorted-lists)

**Code** Time:  $O(N \log k)$  Space:  $O(\log k)$  where **N** is the total number of nodes across all lists and **k** is the number of lists

```
ListNode* mergeKLists(vector<ListNode*>& lists) {
    if (lists.empty()) return nullptr;
    return divideAndConquer(lists, 0 /* left */, lists.size() - 1 /* right */);
}

ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    if (!l1) return l2;
    if (!l2) return l1;

    if (l1->val < l2->val) {
        l1->next = mergeTwoLists(l1->next, l2);
        return l1;
    } else {
        l2->next = mergeTwoLists(l2->next, l1);
        return l2;
    }
}

ListNode* divideAndConquer(vector<ListNode*> lists, int left, int right) {
    if (left == right) return lists[right];

    int mid = left + (right - left) / 2;
    ListNode* l1 = divideAndConquer(lists, left, mid);
    ListNode* l2 = divideAndConquer(lists, mid + 1, right);
    return mergeTwoLists(l1, l2);
}
```

# Problem – 146. LRU cache

Medium



LeetCode

[leetcode.com/problems/lru-cache](https://leetcode.com/problems/lru-cache)

## Problem

- **LRU (Least Recently Used)** is a cache eviction policy where when a cache is full and a new item needs to be added, the least accessed item is removed.
- When an element is accessed, the item is marked as recent accessed
- More information: [https://en.wikipedia.org/wiki/Cache\\_replacement\\_policies#LRU](https://en.wikipedia.org/wiki/Cache_replacement_policies#LRU)
- You are asked to design a LRU data structure
- The data structure has a given capacity
- You must implement **get** to retrieve a cache value
- You must implement **put** to add or update a new value
- Both **get** and **put** should run in **O(1)** average time complexity

# Problem – 146. LRU cache

Medium



LeetCode

[leetcode.com/problems/lru-cache](https://leetcode.com/problems/lru-cache)

## Solution

- Store the values in a data structure and keep the least accessed at the end
- You have to store your cache in some data structure that is **update-friendly**
- **Doubly linked** list is a good candidate. However, accessing an element costs  **$O(n)$**
- To solve this issue, you can create a **hash map to index** each key from the doubly linked list
- You end up with two main data structures:

```
std::list<pair<int, int>> data;  
std::unordered_map<int, std::list<pair<int, int>>::iterator> index;
```

- **data** is where the key values are stored
- **index** contains the key and the iterator pointer to the data element
- Invalid capacity handling is not implemented (capacity  $\leq 0$ )

# Problem – 146. LRU cache

Medium



LeetCode

[leetcode.com/problems/lru-cache](https://leetcode.com/problems/lru-cache)

**Code** Time: **O(1)** Space: **O(capacity)**

```
class LRUCache {
private:
    std::list<std::pair<int, int>> data;
    std::unordered_map<int, std::list<std::pair<int,
int>>::iterator> index;
    int capacity;

    void moveToFront(std::list<std::pair<int,
int>>::iterator dataIt) {
        data.splice(data.begin(), data, dataIt);
    }

public:
    LRUCache(int capacity) : capacity(capacity) {}

    int get(int key) {
        auto it = index.find(key);
        if (it != index.end()) {
            auto dataIt = it->second;
            moveToFront(dataIt);
            return dataIt->second;
        }
        return -1;
    }
};
```

```
void put(int key, int value) {
    auto it = index.find(key);
    if (it != index.end()) {
        // key exists
        auto dataIt = it->second;
        dataIt->second = value;
        moveToFront(dataIt);
        return;
    }

    // key doesn't exist: check capacity and add new element
    if (data.size() >= capacity) {
        // evict least recently used (back element)
        auto last = data.back();
        index.erase(last.first);
        data.pop_back();
    }

    // add new element to front
    data.push_front({key, value});
    index[key] = data.begin();
};
```

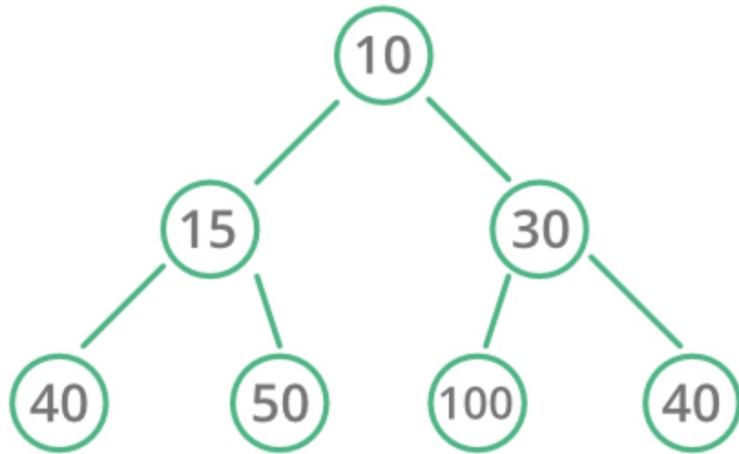


# HEAP / PRIORITY QUEUE

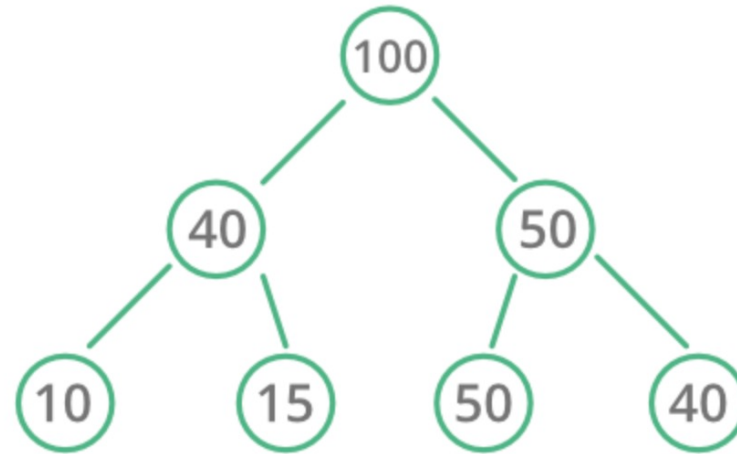
# Heap

- **Heap** is a complete binary tree that satisfy the heap property (max or min)
- **Min heap**: root node contains the minimum value
- **Max heap**: root node contains the maximum value

Min Heap



Max Heap



# Heap in C++

## Two main ways to implement:

### 1. Using **std::make\_heap** from **<algorithm>**

```
std::make_heap(RandomIt first, RandomIt last)
```

```
std::push_heap(RandomIt first, RandomIt last)
```

```
std::pop_heap(RandomIt first, RandomIt last)
```

```
std::sort_heap(RandomIt first, RandomIt last)
```

### 2. Using **std::priority\_queue** from **<queue>** **(recommended)**

```
std::priority_queue<T, Container, Compare>
```

# Heap in C++ – `std::priority_queue` example

## Min heap

```
std::priority_queue<int, std::vector<int>, std::greater<int>>>
```

## Max heap

```
std::priority_queue<int> or
```

```
std::priority_queue<int, std::vector<int>, std::less<int>>>
```

```
// Min heap
std::priority_queue<int, std::vector<int>, std::greater<int>>> minHeap;

minHeap.push(3);
minHeap.push(6);
minHeap.push(4);
// remove top element (3)
minHeap.pop();
// root node (top) is now 4
std::cout << minHeap.top();
```

# Problem – 215. Kth Largest Element in an Array

Medium



LeetCode

[leetcode.com/problems/kth-largest-element-in-an-array](https://leetcode.com/problems/kth-largest-element-in-an-array)

## Problem

- You are given an array of integers **nums** and an integer **k**
- Find the **k<sup>th</sup>** largest element
- **Example:**

### Input

`nums = [3, 2, 1, 5, 6, 4]`

`k = 2`

**Output:** 5

# Solution – 215. Kth Largest Element in an Array

Medium



LeetCode

[leetcode.com/problems/kth-largest-element-in-an-array](https://leetcode.com/problems/kth-largest-element-in-an-array)

## Solution

- Start with a **min heap**
- Loop through the **nums** array:
  - Add the element to the min heap
  - Check if the size of the heap is always less than **k**. If the size is greater, pop the minimum element
- Return the element from the top: the  $k^{\text{th}}$  largest element

# Code – 215. Kth Largest Element in an Array

Medium



LeetCode

[leetcode.com/problems/kth-largest-element-in-an-array](https://leetcode.com/problems/kth-largest-element-in-an-array)

**Code** Time:  $O(n \log k)$  Space:  $O(k)$

```
int findKthLargest(vector<int>& nums, int k) {
    std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;
    // O(n)
    for (const auto& num : nums) {
        // O(log k) since the heap is bounded to k elements
        minHeap.push(num);
        if (minHeap.size() > k) {
            minHeap.pop();
        }
    }
    return minHeap.top();
}
```

# Problem – 347. Top K Frequent Elements

Medium



LeetCode

[leetcode.com/problems/top-k-frequent-elements](https://leetcode.com/problems/top-k-frequent-elements)

## Problem

- You are given an **array** of numbers and an **integer**  $k$
- Return an array with the  **$k$**  most frequent elements

Example

**Input:**

$\text{nums} = [1, 1, 1, 2, 2, 3], k = 2$

**Output:**

$[1, 2]$



# Solution – 347. Top K Frequent Elements

Medium



LeetCode

[leetcode.com/problems/top-k-frequent-elements](https://leetcode.com/problems/top-k-frequent-elements)

## Solution (1) - hashmap + array sort

- Go over the array, count the numbers and store them in an *unordered\_map*

### Example:

`nums = [1,1,1,2,2,3], k = 2`

`freq[1] = 3`

`freq[2] = 2`

`...`

- Go over the *unordered\_map*, add to an array and sort descending
- Create another array adding the **k** first elements and return

# Code – 347. Top K Frequent Elements

Medium



LeetCode

[leetcode.com/problems/top-k-frequent-elements](https://leetcode.com/problems/top-k-frequent-elements)

**Code (1)** Time:  $O(n \log n)$  Space:  $O(n)$

```
vector<int> topKFrequent(vector<int>& nums, int k) {  
    // 1. Create the number's frequency map  
    // O(n)  
    unordered_map<int, int> freq;  
    for (const auto& num : nums) {  
        freq[num] += 1;  
    }  
    // 2. Create an array with the frequencies  
    vector<pair<int, int>> freqVec(freq.begin(), freq.end());  
    // 3. Sort by the frequency  $O(n \log n)$   
    sort(freqVec.begin(), freqVec.end(), [](auto& a, auto& b) {  
        return a.second > b.second;  
    });  
    // 4. Create the result with the k first elements  
    // O(k)  
    vector<int> result;  
    for (int i = 0; i < k; ++i) {  
        result.push_back(freqVec[i].first);  
    }  
    return result;  
}
```

# Solution – 347. Top K Frequent Elements

Medium



LeetCode

[leetcode.com/problems/top-k-frequent-elements](https://leetcode.com/problems/top-k-frequent-elements)

## Solution (2) - hashmap + min heap

- Go over the array, count the numbers and store them in an *unordered\_map*

### Example:

`nums = [1,1,1,2,2,3], k = 2`

`freq[1] = 3`

`freq[2] = 2`

`...`

- Go over the frequencies, add to a min heap. If the size of the heap exceeds **k**, remove the top one (the minimum value)
- Create another array result adding all elements from the heap and return it

# Code – 347. Top K Frequent Elements

Medium



LeetCode

[leetcode.com/problems/top-k-frequent-elements](https://leetcode.com/problems/top-k-frequent-elements)

**Code (2)** Time:  $O(n \log k)$  Space:  $O(n)$

```
vector<int> topKFrequent(vector<int>& nums, int k) {  
    // 1. Create the number's frequency map  
    // O(n)  
    unordered_map<int, int> freq;  
    for (const auto& num : nums) {  
        freq[num] += 1;  
    }  
    // 2. Create the min heap with priority queue  
    // O(n log k)  
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> minHeap;  
    for (const auto& [num, count] : freq) {  
        minHeap.push({count, num});  
        if (minHeap.size() > k) minHeap.pop();  
    }  
    // 3. build the result  
    vector<int> result;  
    while (!minHeap.empty()) {  
        auto num = minHeap.top().second;  
        minHeap.pop();  
        result.push_back(num);  
    }  
    return result;  
}
```

# Solution – 347. Top K Frequent Elements

Medium



LeetCode

[leetcode.com/problems/top-k-frequent-elements](https://leetcode.com/problems/top-k-frequent-elements)

## Solution (3) - hashmap + bucket sort

- Go over the array, count the numbers and store them in an *unordered\_map*

### Example:

`nums = [1,1,1,2,2,3], k = 2`

`freq[1] = 3`

`freq[2] = 2`

...

- Create buckets for each frequency and add the corresponding numbers:

`bucket[1] = [3]` → 3 only appears once in nums

`bucket[2] = [2]` → 2 appears twice

`bucket[3] = [1]` → 1 appears three times

- Go over each bucket, add to the result and return it

# Code – 347. Top K Frequent Elements

Medium



LeetCode

[leetcode.com/problems/top-k-frequent-elements](https://leetcode.com/problems/top-k-frequent-elements)

**Code (3)** Time:  $O(n)$  Space:  $O(n)$

```
vector<int> topKFrequent(vector<int>& nums, int k) {  
    // Create the number's frequency map  
    unordered_map<int, int> freq;  
    for (const auto& num : nums) {  
        freq[num]++;  
    }  
    // create the buckets  
    // e.g. [[1,2,3],[4,5,6]] ...  
    vector<vector<int>> buckets(nums.size() + 1);  
    for (const auto& [num, count] : freq) {  
        buckets[count].push_back(num);  
    }  
    // go over each bucket to build the result  
    vector<int> result;  
    for (int i = buckets.size() - 1; i >= 0; --i) {  
        for (const auto& num : buckets[i]) {  
            result.push_back(num);  
            if (result.size() == k) return result;  
        }  
    }  
    return result;  
}
```

# Problem – 347. Top K Frequent Elements

Medium



LeetCode

[leetcode.com/problems/top-k-frequent-elements](https://leetcode.com/problems/top-k-frequent-elements)

## Some considerations

- Theoretically, bucket sort should be the fastest solution  $O(n) < O(n \log k)$
- In practice, min heap end up being faster:
  - fewer allocations: priority\_queue stores flat pairs rather than inner vectors
  - better cache locality: heap is built over a single array (binary heap)
  - if **k** is small, heap touches fewer elements

**MATRIX**



# Problem – 73. Set Matrix Zeroes

Medium



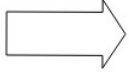
LeetCode

[leetcode.com/problems/set-matrix-zeroes](https://leetcode.com/problems/set-matrix-zeroes)

## Problem

- You are given a **matrix m x n**
- When an element in the matrix is **0**, set the whole column and row to **zero**
- **You must do it in place**

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |



|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 0 | 0 |
| 1 | 0 | 1 |

# Solution – 73. Set Matrix Zeroes

Medium



LeetCode

[leetcode.com/problems/set-matrix-zeroes](https://leetcode.com/problems/set-matrix-zeroes)

## Solution

- Iterate through the matrix top-down
- For each column (from  $\text{col} = 1$  to  $n - 1$ ), when you **find 0**, set:  
 **$\text{matrix}[\text{row}][0] = 0$**  → marks that the row should be zeroed  
 **$\text{matrix}[0][\text{col}] = 0$**  → marks that the col should be zeroed
- If  **$\text{matrix}[\text{row}][0] == 0$** , set a variable  **$\text{col0} = \text{true}$**  to remember if column 0 must be zeroed later
- Iterate bottom-top, right-left
- For each cell, if  **$\text{matrix}[\text{row}][0] == 0$**  or  **$\text{matrix}[0][\text{col}] == 0$**  set  **$\text{matrix}[\text{row}][\text{col}] = 0$**
- It must be bottom-top, right-left to not set the first row to zero first
- Also check if col0 is true. If true, set the first column to zero:  
 **$\text{matrix}[\text{row}][0] = 0$**

# Code – 73. Set Matrix Zeroes

Medium



LeetCode

[leetcode.com/problems/set-matrix-zeroes](https://leetcode.com/problems/set-matrix-zeroes)

**Code** Time:  $O(m \times n)$  Space:  $O(1)$  where  $m$  is the number of rows and  $n$  the number of columns. As this is an in-place implementation, there is no additional space being allocated, therefore space complexity is  $O(1)$

```
void setZeroes(vector<vector<int>>& matrix) {
    int m = matrix.size();
    int n = matrix[0].size();
    bool col0 = false;

    for (int row = 0; row < m; ++row) {
        if (matrix[row][0] == 0) col0 = true;
        for (int col = 1; col < n; ++col) {
            if (matrix[row][col] == 0) {
                matrix[row][0] = 0;
                matrix[0][col] = 0;
            }
        }
    }

    for (int row = m - 1; row >= 0; --row) {
        for (int col = n - 1; col >= 1; --col) {
            if (matrix[row][0] == 0 || matrix[0][col] == 0) {
                matrix[row][col] = 0;
            }
        }
        if (col0) {
            matrix[row][0] = 0;
        }
    }
}
```

# Problem – 54. Spiral Matrix

Medium



LeetCode

[leetcode.com/problems/spiral-matrix](https://leetcode.com/problems/spiral-matrix)

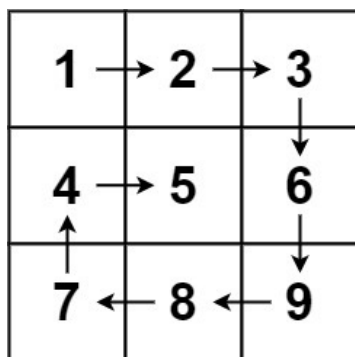
## Problem

- You are given a matrix **m x n**
- Return the elements in a flat array, the same order as the image

- **Example**

Input: matrix = `[[1,2,3],[4,5,6],[7,8,9]]`

Output: matrix = `[1,2,3,6,9,8,7,4,5]`



# Solution – 54. Spiral Matrix

Medium



LeetCode

[leetcode.com/problems/spiral-matrix](https://leetcode.com/problems/spiral-matrix)

## Solution

- **Traverse the matrix in spiral order following four directions:** right across top row, down the right column, left across bottom row, and up the left column, then repeat inward
- **Maintain four boundary variables that shrink after each direction:** rowStart, rowEnd, colStart, and colEnd get updated after completing each side of the spiral to move the boundaries inward
- **Boundary handling is the main challenge:** It's easy to introduce bugs when determining when to stop traversal or when to skip certain directions, requiring careful condition checks
- **Non-square matrices require special boundary checks:** The conditional statements if (rowStart <= rowEnd) and if (colStart <= colEnd) prevent adding duplicate elements when dealing with rectangular matrices or edge cases like single rows/columns

# Code – 54. Spiral Matrix

Medium



LeetCode

[leetcode.com/problems/spiral-matrix](https://leetcode.com/problems/spiral-matrix)

**Code** Time:  $O(m \times n)$  Space:  $O(1)$

```
vector<int> spiralOrder(vector<vector<int>>& matrix) {
    vector<int> result;
    int rowStart = 0;
    int colStart = 0;
    int colEnd = matrix[0].size() - 1;
    int rowEnd = matrix.size() - 1;

    while(rowStart <= rowEnd && colStart <= colEnd) {
        for (int col = colStart; col <= colEnd; ++col) {
            result.push_back(matrix[rowStart][col]);
        }
        ++rowStart;

        for (int row = rowStart; row <= rowEnd; ++row) {
            result.push_back(matrix[row][colEnd]);
        }
        --colEnd;

        // if matrix is not square condition
        if (rowStart <= rowEnd) {
            for (int col = colEnd; col >= colStart; --col) {
                result.push_back(matrix[rowEnd][col]);
            }
            --rowEnd;
        }

        if (colStart <= colEnd) {
            for (int row = rowEnd; row >= rowStart; --row) {
                result.push_back(matrix[row][colStart]);
            }
            colStart++;
        }
    }
    return result;
}
```

# **DYNAMIC PROGRAMMING**

# Dynamic Programming

**Dynamic Programming (DP)** is an algorithm technique used to solve problems that can be broken down into **simpler, overlapping subproblems**.

## Key Concepts of Dynamic Programming

- **Overlapping subproblems:** a problem has overlapping subproblems if it can be broken down into subproblems.
- **Memoization (Top-Down Approach):** store the results in a cache (typically a dictionary or array) to avoid recalculation – recursion and caching approach.
- **Tabulation (Bottom-Up Approach):** first solve all possible subproblems iteratively, and store them in a table.



# Common Patterns in Dynamic Programming

- **Toy example (Fibonacci):** Climbing Stairs, N-th Tribonacci Number, Perfect Squares
- **Constant Transition:** Min Cost Climbing Stairs, House Robber, Decode Ways, Minimum Cost For Tickets, Solving Questions With Brainpower
- **Grid:** Unique Paths, Unique Paths II, Minimum Path Sum, Count Square Submatrices with All Ones, Maximal Square, Dungeon Game
- **Dual-Sequence:** Longest Common Subsequence, Uncrossed Lines, Minimum ASCII Delete Sum for Two Strings, Edit Distance, Distinct Subsequences, Shortest Common Supersequence
- **Interval:** Longest Palindromic Subsequence, Stone Game VII, Palindromic Substrings, Minimum Cost Tree From Leaf Values, Burst Balloons, Strange Printer
- **Longest Increasing Subsequence:** Count Number of Teams, Longest Increasing Subsequence, Partition Array for Maximum Sum, Largest Sum of Averages, Filling Bookcase Shelves
- **Knapsack:** Partition Equal Subset Sum, Number of Dice Rolls With Target Sum, Combination Sum IV, Ones and Zeroes, Coin Change, Coin Change II, Target Sum, Last Stone Weight II, Profitable Schemes
- **Topological Sort on Graphs:** Longest Increasing Path in a Matrix, Longest String Chain, Course Schedule III
- **DP on Trees:** House Robber III, Binary Tree Cameras
- **Other problems:** 2 Keys Keyboard, Word Break, Minimum Number of Removals to Make Mountain Array, Out of Boundary Paths

## Credits

[https://www.youtube.com/watch?v=9k31KcQmS\\_U](https://www.youtube.com/watch?v=9k31KcQmS_U)

<https://algo.monster/problems/dp-list>

# Dynamic Programming – Example – Fibonacci Sequence

## Naive Recursive Approach

$O(2^n)$

```
int fib(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    return fib(n - 1) + fib(n - 2);  
}
```

## Memoization (Top-Down DP)

$O(n)$

```
std::unordered_map<int, int> memo;  
  
int fib(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    if (memo.find(n) != memo.end()) {  
        return memo[n];  
    }  
    memo[n] = fib(n - 1) + fib(n - 2);  
    return memo[n];  
}
```

## Tabulation (Bottom-up DP)

$O(n)$

```
int fib(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    int dp[n + 1];  
    dp[0] = 0;  
    dp[1] = 1;  
    for (int i = 2; i <= n; i++) {  
        dp[i] = dp[i - 1] + dp[i - 2];  
    }  
    return dp[n];  
}
```

# Problem – Climbing Stairs

Easy



LeetCode

[leetcode.com/problems/climbing-stairs](https://leetcode.com/problems/climbing-stairs)

## Problem

- You need to climb a staircase with **n steps**
- Each time, you can only climb either **1 step** or **2 steps**
- Find out how **many different ways** you can **climb to the top**
- **Example:**

## Input

$n = 2$

**Output:** 2

## Explanation:

1. Climb 1 step, then climb 1 step again
2. Climb 2 steps in one go

# Solution – Climbing Stairs

Easy



LeetCode

[leetcode.com/problems/climbing-stairs](https://leetcode.com/problems/climbing-stairs)

## Solution

- **You need to climb a staircase with  $n$  steps** and want to find the total number of distinct ways to reach the top
- **At each step, you can either take 1 step or 2 steps** - this gives you two choices at most positions
- **This follows the Fibonacci sequence pattern** - the number of ways to reach step  $n$  equals ways to reach  $(n-1)$  plus ways to reach  $(n-2)$
- **Use dynamic programming to avoid recalculating subproblems** - either bottom-up tabulation or memoized recursion works well
- **Base cases are crucial**: typically  $f(1) = 1$  and  $f(2) = 2$ , representing the ways to reach the first and second steps

# Solution – Climbing Stairs

Easy



LeetCode

[leetcode.com/problems/climbing-stairs](https://leetcode.com/problems/climbing-stairs)

## Code

```
std::unordered_map<int, int> memo;
```

```
int climbStairs(int n) {  
    // Identify the sequence, when:  
    // n = 0 (0 way), there is no way to get up  
    // n = 1 (1 way): only one way : 1-step  
    // n = 2 (2 ways): 1s + 1s | 2s  
    // n = 3 (3 ways): 1s + 1s + 1s | 1s + 2s | 2s + 1s  
    // n = 4 (5 ways): 1s + 1s + 1s + 1s | 1s + 1s + 2s | 1s + 2s + 1s | 2s + 1s + 1s | 2s + 2s |  
  
    if (n <= 2) {  
        return n;  
    }  
  
    if (memo.find(n) != memo.end()) {  
        return memo[n];  
    }  
  
    memo[n] = climbStairs(n - 1) + climbStairs(n - 2);  
    return memo[n];  
}
```

# Problem – 1143. Longest Common Subsequence

Medium



LeetCode

<https://leetcode.com/problems/longest-common-subsequence>

## Problem

- You are given two strings, example:  
text1 = "abcd"  
text2 = "ace"
- Find the longest subsequence between them

# Problem – 62. Unique Paths

Medium



LeetCode

<https://leetcode.com/problems/unique-paths>

## Problem

- The robot is placed in a  $m \times n$  grid
- It starts at the top-left cell  $(0,0)$  and must reach the bottom-right  $(m - 1, n - 1)$
- The robot can only move right or down at any point
- Return the number of unique paths the robot can take to reach the destination



# Solution – 62. Unique Paths

Medium



LeetCode

<https://leetcode.com/problems/unique-paths>

## Solution 1 (recursive)

- Define a recursive function `countPaths(m, n)`

- **Base case**

If  $m == 1$  or  $n == 1$ , there's only one way to reach that cell (either all downs or all rights).

- **Recursive case**

To reach cell  $(m, n)$  the robot must come from:

Cell  $(m - 1, n) \rightarrow$  from above

Cell  $(m, n - 1) \rightarrow$  from left

So the number of of paths to  $(m, n)$  is the **sum** of the paths to those two cells

- **Memoization**

Use a 2D vector  $[m + 1][n + 1]$



# Code – 62. Unique Paths

Medium



LeetCode

<https://leetcode.com/problems/unique-paths>

**Code** Time:  $O(m * n)$  Space:  $O(m * n)$

```
int countPaths(int m, int n, vector<vector<int>>& memo) {
    if (m == 1 || n == 1) return 1;
    if (memo[m][n] != -1) return memo[m][n];
    memo[m][n] = countPaths(m, n - 1, memo) + countPaths(m - 1, n, memo);
    return memo[m][n];
}
int uniquePaths(int m, int n) {
    /*
        count(m, n) = count(m, n + 1) + count(m + 1, n)
        same as (imagine robot going from m,n to 0,0 up and left)
        count(m, n) = count(m, n - 1) + count(m - 1, n)
    */
    vector<vector<int>> memo(m + 1, vector<int>(n + 1, -1));
    return countPaths(m, n, memo);
}
```

# Solution – 62. Unique Paths

Medium



LeetCode

<https://leetcode.com/problems/unique-paths>

## Solution 2 (iterative)

- Create a 2D vector `dp` of size  $(m+1) \times (n+1)$  to store intermediate results
  - Set `dp[1][1] = 1` because there is exactly one way to stand on the starting cell
  - Iterate through each cell `(row, col)` from `(1, 1)` to `(m, n)`:
    - Skip `(1, 1)` since it's already initialized
    - For every other cell, the number of unique paths to it is the sum of:
      - Paths from the cell above: `dp[row-1][col]`
      - Paths from the cell to the left: `dp[row][col-1]`
- `dp[row][col] = dp[row - 1][col] + dp[row][col - 1]`**

# Code – 62. Unique Paths

Medium



LeetCode

<https://leetcode.com/problems/unique-paths>

**Code** Time:  $O(m * n)$  Space:  $O(m * n)$

```
int uniquePaths(int m, int n) {
    vector<vector<int>> dp(m + 1, vector<int>(n + 1));
    dp[1][1] = 1;
    for (int row = 1; row <= m; ++row) {
        for (int col = 1; col <= n; ++col) {
            if (row == 1 && col == 1) continue;
            dp[row][col] = dp[row-1][col] + dp[row][col-1];
        }
    }
    return dp[m][n];
}
```

```
// Optimized 1DP
int uniquePaths(int m, int n) {
    vector<int> dp(n, 1); // base case: first row is all 1s
    for (int row = 1; row < m; ++row) {
        for (int col = 1; col < n; ++col) {
            dp[col] = dp[col] + dp[col - 1];
        }
    }
    return dp[n - 1];
}
```

# Solution – 62. Unique Paths

Medium



LeetCode

<https://leetcode.com/problems/unique-paths>

## Solution 3 (combinatorics)

- **Grid size:**  $m \times n$
- **Start:** top-left cell  $(0, 0)$
- **End:** bottom-right cell  $(m - 1, n - 1)$
- To get from the top-left to the bottom-right:

You must move exactly  $m - 1$  times down

And exactly  $n - 1$  times right

These two types of moves must be made in some order, with a total of:

$$(m - 1) + (n - 1) = m + n - 2 \text{ moves}$$

- **Hence**, from a sequence of  $m + n - 2$  moves, choose  $m - 1$  of them to be down moves (the rest will be right), or vice versa

$$\text{Number of unique paths} = \binom{m + n - 2}{m - 1} = \frac{(m + n - 2)!}{(m - 1)! (n - 1)!}$$

# Code – 62. Unique Paths

Medium



LeetCode

<https://leetcode.com/problems/unique-paths>

**Code** Time:  $O(\min(m,n))$  Space:  $O(1)$

```
int uniquePaths(int m, int n) {
    // we will compute the binomial coefficient:
    // (m + n - 2) choose (m - 1) => total moves choose down moves
    // = (m + n - 2)! / ((m - 1)! * (n - 1)!)

    long long res = 1;

    // we compute the result iteratively to avoid large factorials
    // res = (n) * (n+1) * ... * (m+n-2) / (1 * 2 * ... * (m - 1))

    for (int i = 1; i <= m - 1; ++i) {
        // multiply numerator: (n - 1 + i)
        // divide by denominator: i
        res = res * (n - 1 + i) / i;
    }

    return (int)res;
}
```

# Problem – 198. House Robber

Medium



LeetCode

[leetcode.com/problems/house-robber](https://leetcode.com/problems/house-robber)

## Problem

- You are robbing houses lined up in a row
- Each house has a cash value **nums[i]**
- You **cannot rob two adjacent houses**
- Goal: maximize total money robbed without triggering alarms

# Solution – 198. House Robber

Medium



LeetCode

[leetcode.com/problems/house-robber](https://leetcode.com/problems/house-robber)

## Solution

- Solve as a DP problem
- `dp` represents the maximum value you can get once rob only `i` or the previous houses
- At each “house” `i`, you can either rob or skip

- **Example**

`nums = [1,2,3,1]`

at position `i = 0`, you can have two options:

**rob**: you end up with 1; or **skip**: you end up with 0

at position `i = 1`

**rob**: you get 2 + total of two houses before; or **skip**: total of robbed before

- Therefore:

**rob** = `nums[0] + dp[i - 2]`

**skip** = `dp[i - 1]`

# Problem – 198. House Robber

Medium



LeetCode

[leetcode.com/problems/house-robber](https://leetcode.com/problems/house-robber)

**Code** Time:  $O(n)$  Space:  $O(n)$

```
int rob(vector<int>& nums) {
    int n = nums.size() + 1;
    // base case: start dp[0] = 0 (no rob)
    // dp[1] = nums[0]
    vector<int> dp(n, 0);
    dp[1] = nums[0];
    for (int i = 2; i < n; ++i) {
        int numPos = i - 1;
        // if I rob
        int rob = nums[numPos] + dp[i - 2];
        // if I skip
        int skip = dp[i - 1];
        dp[i] = max(rob, skip);
    }
    return dp[n - 1];
}
```



# Problem – 198. House Robber

Medium



LeetCode

[leetcode.com/problems/house-robber](https://leetcode.com/problems/house-robber)

**Code (space optimized)** Time:  $O(n)$  Space:  $O(1)$

```
int rob(vector<int>& nums) {
    int prev = nums[0];
    int prev2 = 0;

    for (int i = 2; i <= nums.size(); ++i) {
        int numPos = i - 1;
        // if I rob
        int rob = nums[numPos] + prev2;
        // if I skip
        int skip = prev;
        prev2 = prev;
        prev = max(rob, skip);
    }
    return prev;
}
```

# Problem – 213. House Robber II

Medium



LeetCode

[leetcode.com/problems/house-robber-ii](https://leetcode.com/problems/house-robber-ii)

## Problem

- Similar to House Robber I but with a new constraint:
- You are robbing houses **lined up in a circle**
- Each house has a cash value **nums[i]**
- You **cannot rob two adjacent houses**
- You **cannot rob the last and first houses at the same time**
- Goal: maximize total money robbed without triggering alarms

# Problem – 55. Jump

Medium



LeetCode

<https://leetcode.com/problems/jump-game>

## Problem

- You are given an **array of integers**
- Each element represents the **maximum number of jumps** you can go
- **Example:**  
[2,1,1]  
at index 0, element is 2, so you can go 2 elements further (index 2)  
at index 1, element is 1, so you can go 1 element further from there (index 2)
- **Return true** if you can reach the **end of the array**, false otherwise



LeetCode

<https://leetcode.com/problems/jump-game>

## Solution

- Go over the **array of integers**
- For **each position**, you can go from your **current position + the value of the current element**

- Example:

[2,1,0,4]

At position 0, you can go to position  **$0 + 2 = 2$**  where the element is **0**

- Keep track of the **maximum distance** you can go
- If the maximum distance you can go is less than your current position, **return false**
- If reached the end of the loop, the jumps are completed, so **return true**

# Code – 55. Jump

Medium



LeetCode

<https://leetcode.com/problems/jump-game>

**Code** Time: **O(n)** Space: **O(1)**

```
bool canJump(vector<int>& nums) {
    int dist = 0;
    int n = nums.size();
    if (nums.size() == 1) return true;
    for (int i = 0; i < n; ++i) {
        dist = max(dist, i + nums[i]);
        if (i > dist) return false;
    }
    return true;
}
```

**EOF**